CSE_3523

Segment-4
Interrupt system

Microprocessor, microcontroller and embedded system

Course code: CSE-3523
Credit Hour: 3

Md. Mujibur Rahman Maruf
Lecturer, Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

interrupt

- **An interrupt** is either a hardware generated call (externally derived from a hardware signal) or software generated CALL (internally derived from the execution of an instruction or by some other internal event)
- Interrupt is the method of creating a temporary halt during program execution and allows peripheral devices to access the microprocessor.
- The microprocessor responds to that interrupt with an ISR (Interrupt Service Routine), which is a short program to instruct the microprocessor on how to handle the interrupt

Types of interrupt

- **Hardware**
- an external signal is applied to the NMI input pin or the INTR input pin
- NMI - non-maskable interrupt
- INTR – interrupt used to deal with I/O devices that need attention
- **Software**
- Occurs when INT instruction is executed
- When some error condition is produced by the execution of an instruction such as divide by zero

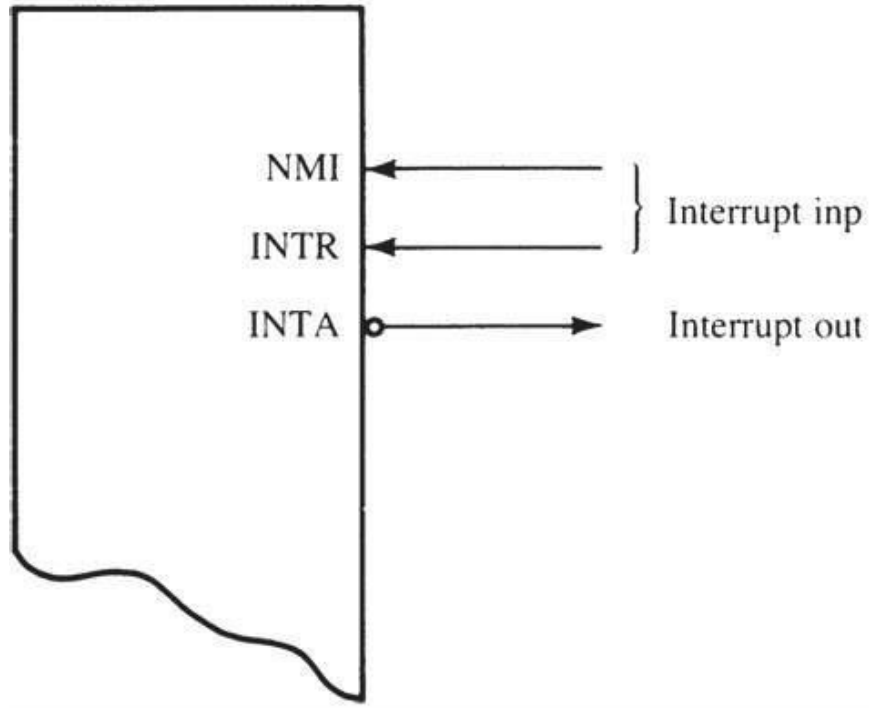
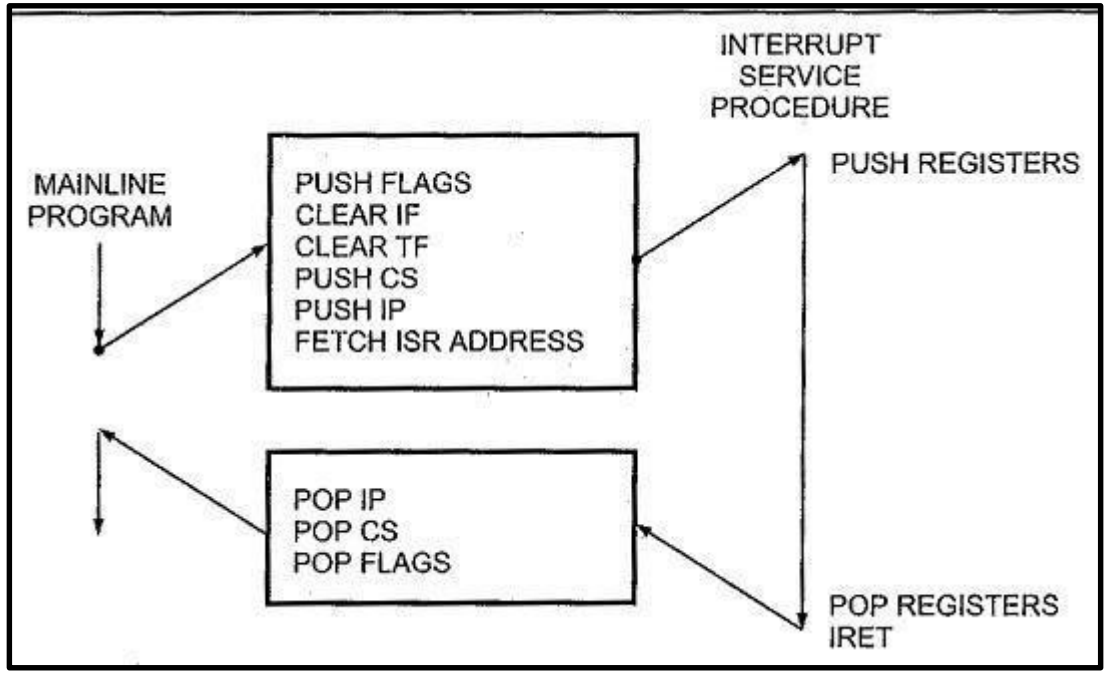


Figure : The interrupt pins on all versions of the Intel microprocessor.

interrupt Handling in 8086



- When an interrupt is requested…
  - the CPU finishes executing current instruction
  - pushes flag register onto stack
  - disables the INTR input by clearing the IF (interrupt flag) in the flag register
  - clears the TF (trap flag) in the flag register
  - pushes the current CS contents onto the stack
  - pushes the current IP contents onto the stack
  - does an indirect far jump to the start of the ISR (Interrupt Service Routine)

interrupt Handling in 8086

- Processor executes Interrupt Service Routine (ISR) like a normal procedure
- Only difference is normal procedures use RET at the end of a procedure, ISRs use an IRET instruction instead of RET
- This pops IP, CS, and the Flag registers
- ISRs can be interrupted!

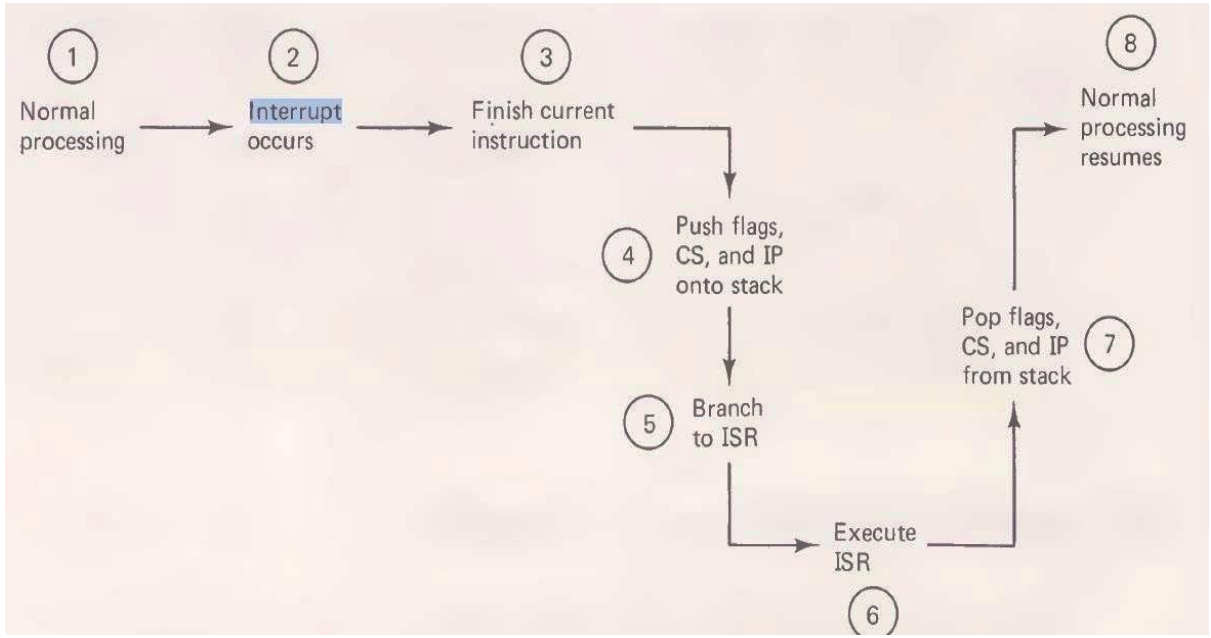
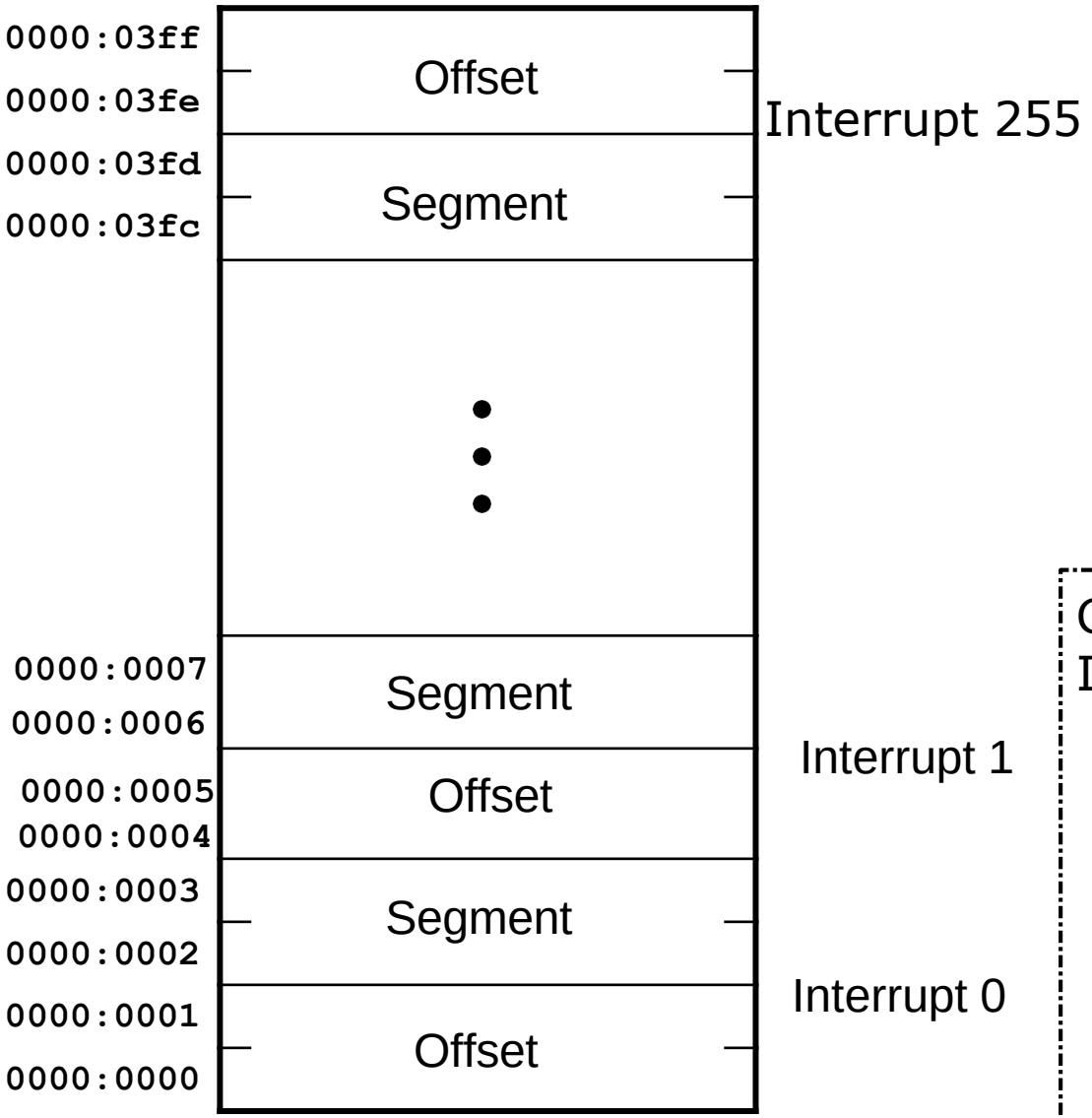


Fig:Interrupt Handling Sequence

interrupt Vector Table

- In 8086, 1st 1k bytes of memory 00000-003ffh is set aside as a table for storing the starting addresses of interrupt service procedures(routines)
- Since 4 bytes are required to store the CS & IP values for each interrupt service procedure, the table can hold the starting addresses of up to 256 interrupts
- The starting address of an interrupt service procedure is called interrupt vector or interrupt pointer and the table is referred to as interrupt vector table or the interrupt pointer table

IVT Format



CS MSB
CS LSB
IP MSB
IP LSB

Given a Vector, where is the ISR address stored in memory ?

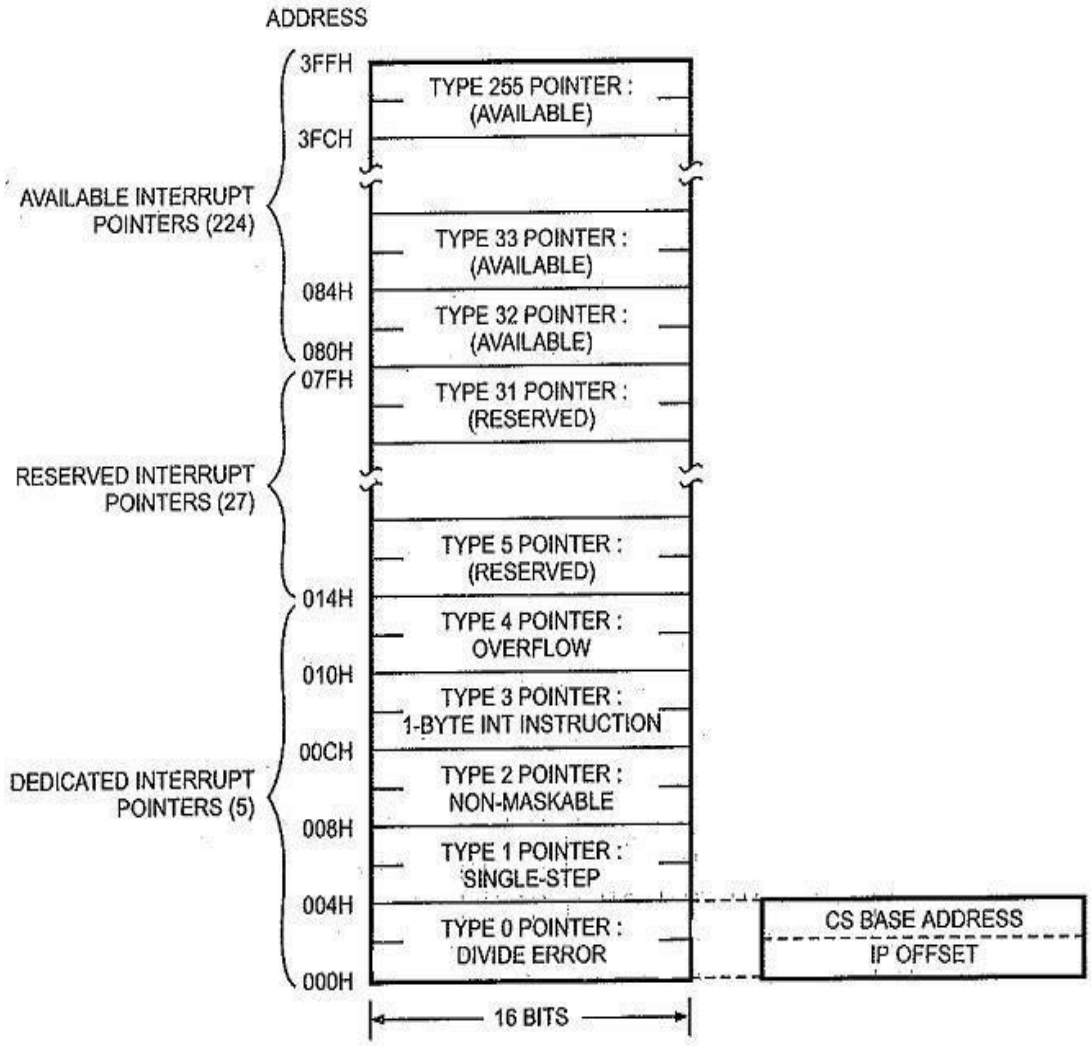
$$Offset = Type \times 4$$

Example: **`int 36h`**

Offset = (54×4) = 216
= **`00d8h`**

interrupt Vector Table

- Each double word interrupt vector is identified by a number from 0 to 255.
- Intel calls this number, the type of interrupt
- The lowest 5 interrupts are dedicated to specific interrupts such as divide by 0, single step, NMI, interrupt caused by an instruction and overflow interrupt.



interrupt Vector Table

- Interrupts from 5-31 are reserved by Intel for more complex processors
- The upper 224 interrupts from type 32 to type 255 are available for users to use for hardware or software interrupts
- The vector for each interrupt requires 4 memory locations.
- Therefore, when the 8086 responds to a particular type of interrupt, it automatically multiplies the type by 4 to produce the desired address in the table to get the starting address of The Interrupt Service Procedure (ISP)

Type 0 interrupt

Occurs automatically when the result of a DIV or IDIV instruction is too large to fit in the destination register (division by zero or overflow).

```
MOV AX, 05H
MOV BL, 00H
DIV BL     ; Division by zero → Type 0 interrupt
```

Operation:

- The CPU pushes the Flags, CS (Code Segment), and IP (Instruction Pointer) onto the stack.
- It then loads CS and IP from memory addresses 0002h and 0000h respectively in the Interrupt Vector Table (IVT).
- The corresponding Interrupt Service Routine (ISR) handles the divide error.

Use Case:
Prevents the system from executing invalid arithmetic operations that could crash programs.

Type 1 interrupt(Single Step)

Cause:

Occurs when the Trap Flag (TF) in the Flag Register is set (TF=1).
Used for single-step debugging — the CPU executes one instruction, then generates a Type 1 interrupt.

Operation:

- After executing each instruction, the processor automatically generates the interrupt.
- CS, IP, and Flags are pushed onto the stack.
- The ISR address is taken from 0004h–0007h in the IVT.

```
STI
MOV TF,1  ; Enable single-step
; After every instruction, CPU triggers Type 1 interrupt
```

Use Case:
Used by debuggers to monitor instruction execution one at a time.

Type 2 interrupt:Non-Maskable Interrupt (NMI)

Cause:

Triggered by a low-to-high signal on the NMI hardware pin.
Cannot be disabled by software (non-maskable).

Operation:

The CPU completes the current instruction.

Pushes Flags, CS, and IP on the stack.

Loads new CS and IP from 0008h–000Bh in the IVT.

Executes the NMI ISR

Example – Power Failure Protection:

A sensor circuit detects a power drop and sends a pulse to the NMI pin.

The ISR quickly saves important data to battery-backed memory.

Type 2 interrupt (example)

The **Non-Maskable Interrupt (NMI)** in 8086 is often used for critical system protections.

In a **boiler system**, a pressure sensor is connected to the NMI input. If the boiler pressure exceeds a preset limit, the sensor sends an interrupt signal. The corresponding Interrupt Service Routine (ISR) can immediately turn off the fuel supply, open a pressure relief valve, and sound an alarm to prevent accidents.

Another common application is power failure protection. External circuits detect the loss of AC power and send an interrupt to the NMI input. Since DC power remains available briefly (around 50 ms due to large capacitors), the ISR quickly saves critical program data to battery-backed RAM. Once AC power is restored, the system retrieves the saved data and resumes program execution seamlessly.

Type 3 interrupt

Cause:
Generated by the INT 3 instruction, used by software debuggers.

Operation:

- When the processor executes INT 3, it automatically calls the ISR located at 000Ch–000Fh in the IVT.
- Used to pause execution at a specific point for debugging.

```
MOV AX, 5
INT 3       ; Breakpoint – stops here for debugging
MOV BX, 6
```

Use Case:
Used in debugging tools to inspect register or memory contents at a specific code line.

Type 4 interrupt:Overflow Interrupt

Cause:
Triggered when the Overflow Flag (OF) is set and the INTO (Interrupt on Overflow) instruction is executed.

Operation:

- If OF=1, a Type 4 interrupt occurs.
- CS, IP, and Flags are pushed on the stack.
- CS and IP are loaded from 0010h–0013h in the IVT.
- If OF=0, execution continues normally (no interrupt).

```
MOV AL,108d
MOV BL,81d
ADD AL, BL    ; May set Overflow Flag
INTO          ; Type 4 interrupt if OF=1
```

Adding two signed 8-bit numbers:
+108d (01101100) + 81d (01010001) ---------- 10111101 (Overflow)
The overflow flag is set because result exceeds signed range, triggering Type 4 interrupt

Use Case:
Used to detect arithmetic overflow in signed operations.

Type 5 interrupt: BOUND

BOUND

It compares a register value with a specified upper and lower bound, raising an interrupt if the value is out of bounds.

How it Works:

The BOUND instruction compares the contents of a register (usually AX or BX) with a specified lower bound and upper bound (stored in memory).

If the value in the register is outside the range defined by the bounds, a BOUND interrupt (Interrupt 5) is triggered.

Type 5 interrupt: BOUND

Cause:
Generated when the BOUND instruction detects a register value outside specified bounds.

Operation:

- BOUND compares a register value with lower and upper limits in memory.
- If the value is outside the range, a Type 5 interrupt occurs.
- ISR address is stored at 0014h–0017h in IVT.

```
DATA SEGMENT
LOWER DW 10
UPPER DW 20
DATA ENDS

MOV AX, 25
BOUND AX, LOWER  ; AX=25 > 20 ⇒ Type 5 interrupt
```

Use case:
Used for array index checking or ensuring variable values stay within valid limits.

interrupt

Type	Interrupt Name	Trigger Source	ISR Address in IVT	Purpose / Example
0	Divide Error	DIV/IDIV overflow or divide by zero	0000h–0003h	Division by zero check
1	Single Step	Trap Flag (TF=1)	0004h–0007h	Step-by-step debugging
2	Non-Maskable Interrupt	NMI hardware pin (low→high)	0008h–000Bh	Power failure, safety shutdown
3	Breakpoint	INT 3 instruction	000Ch–000Fh	Software breakpoint for debugging
4	Overflow	INTO instruction when OF=1	0010h–0013h	Detect signed arithmetic overflow
5	Bound Range Exceeded	BOUND instruction (out of range)	0014h–0017h	Array boundary checking

IRET – Interrupt Return Instruction

Cause:

Executed at the end of every Interrupt Service Routine (ISR) to return control back to the main program that was interrupted.It is not an interrupt type, but a special instruction used to end an interrupt handler.

Operation:

When an interrupt occurs, the processor automatically performs these actions:

- Pushes Flags, CS (Code Segment), and IP (Instruction Pointer) onto the stack.

.

IRET – Interrupt Return Instruction

- Then it jumps to the corresponding ISR (Interrupt Service Routine).
- When the ISR finishes, it executes the IRET instruction.

IRET performs the reverse steps:

- Pops IP from the stack (restores the next instruction address).
- Pops CS from the stack (restores original code segment).
- Pops Flags from the stack (restores all status and control flags).

This restores the exact state of the CPU before the interrupt occurred and allows the interrupted program to continue normally

interrupt Sources

8086 interrupt can originate from three sources:

Hardware Interrupt:

two hardware pins (INTR and NMI) that request interrupts…

And one hardware pin ($\overline{\text{INTA}}$) to acknowledge the interrupt requested through INTR.

Software Interrupt:

The processor also has software interrupts INT, INTO, INT 3, and BOUND.

Flag bits IF (interrupt flag) and TF (trap flag), are also used with the interrupt structure and special return instruction IRET

Internal Interrupt (Exception):

Generated by an error during instruction execution.(divide-by-zero).

Example: If an operand is divided by zero, 8086 automatically interrupts the current program

Hardware interrupt:NMI

- **NMI (Non-Maskable Interrupt):**
- The Non-Maskable Interrupt (NMI) has the highest priority among external interrupts in the 8086 activated on a positive edge (low-to-high transition) and is treated like a Type 2 interrupt (equivalent to executing INT 02).
- When an NMI occurs, the processor completes the current instruction before servicing the interrupt. If an internal interrupt is being serviced, it will be suspended, and the NMI will be honored, even if it is of lower priority.

Hardware interrupt:NMI

- cannot be disabled (or "masked") by the program.
- Because NMI cannot be ignored, it’s used only for very critical situations, like:
  - Hardware memory errors.
  - Impending power failures.
- If NMIs were triggered casually, they could disrupt important operations — like disk read/write processes where interrupting midway could corrupt data.

.

Power failure detector

NMI as Power Failure detector

This circuit detects a power failure and sends an interrupt (NMI - Non-Maskable Interrupt) to the processor, allowing the system to quickly save important data before complete shutdown.

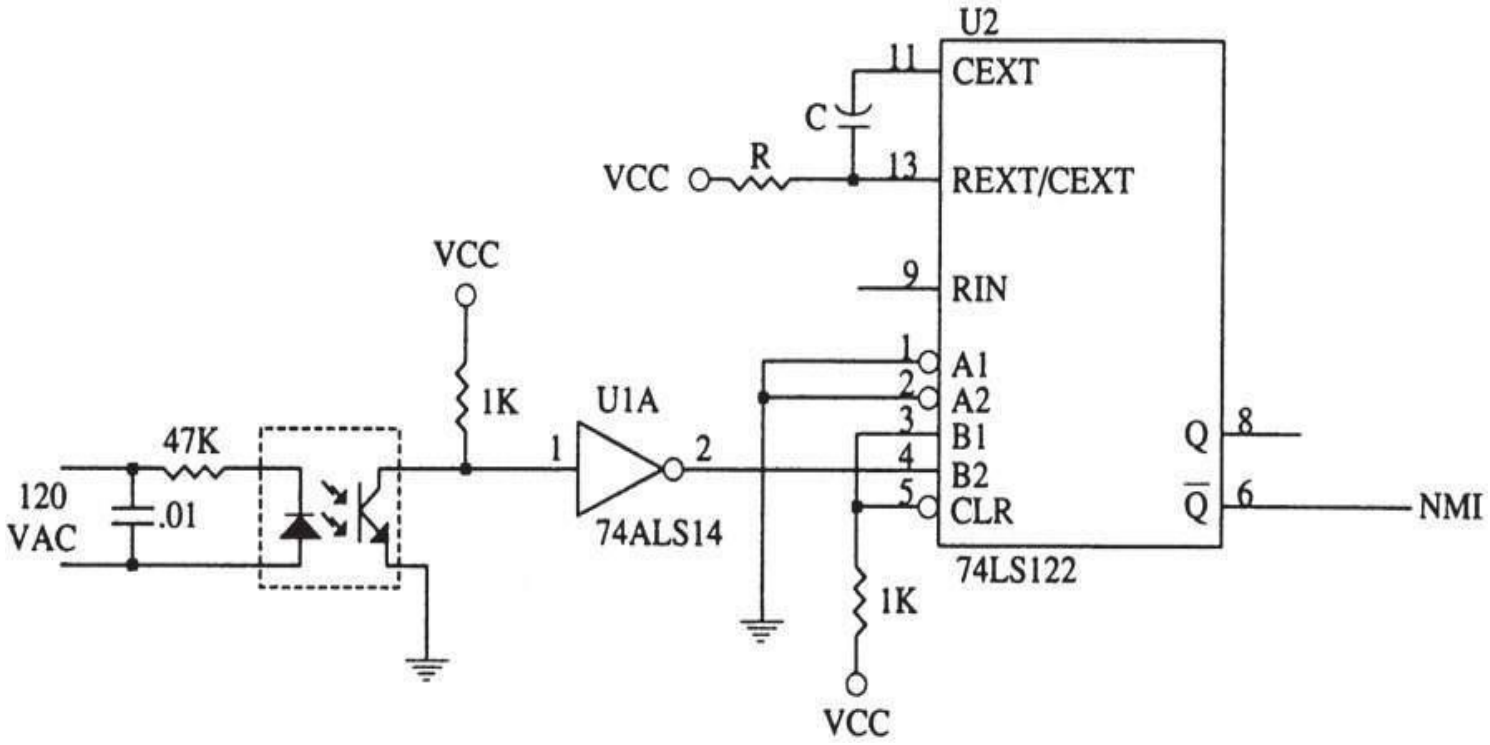


Fig:A power failure detection circuit.

Power failure detector

AC Power Sensing (Optical Isolator):

120V AC powers an optical isolator; when AC is present, the internal LED turns ON, causing the output transistor to conduct.

Inverter Stage (74ALS14 - Schmitt Trigger):

The isolator output feeds into a 74ALS14 Schmitt Trigger inverter (U1A), which sharpens the signal. Its output goes **LOW** when AC power is present and **HIGH** when AC fails.

Pulse Generation using 74LS122 (Monostable Multivibrator):

The clean signal is fed into a 74LS122 one-shot (monostable) multivibrator.

When AC power fails (inverter output goes HIGH), it triggers the one-shot.

The one-shot generates a fixed-width pulse even if the AC fails completely.

Power failure detector

Non-Maskable Interrupt (NMI):

Output Q’ of the 74LS122 sends a pulse to the NMI pin of the processor.

This interrupts the processor immediately (even if interrupts are disabled) to execute emergency routines (like saving important data to memory).

Timing Components:

The timing of the pulse width is set by R (resistor) and C (capacitor) connected to the 74LS122.

This ensures the NMI pulse is long enough for the processor to start its emergency service routine.

Microprocessor Interrupts: INTR, INTA & Data Bus

The **INTR (Interrupt Request)** signal is sent by an external device to the microprocessor, requesting it to halt its current task and handle a more urgent operation. Upon receiving the INTR signal, the microprocessor responds by sending an INTA (Interrupt Acknowledge) signal back to the device, confirming that it has recognized the interrupt. The data bus plays a crucial role in this process by transferring the interrupt vector type number(which points to the Interrupt Service Routine or ISR) on data bus connections D_7–D_0.

Interrupt vector type generation

Interrupt Vector Generation Using Fixed Data

The circuit demonstrates a simple hardware method to generate a fixed interrupt vector type (FFH) during an INTR (Interrupt Request) cycle in an 8086 microprocessor system.

- An 8-bit DIP switch (labeled 1–8) is connected to the data bus lines **D0 to D7**.

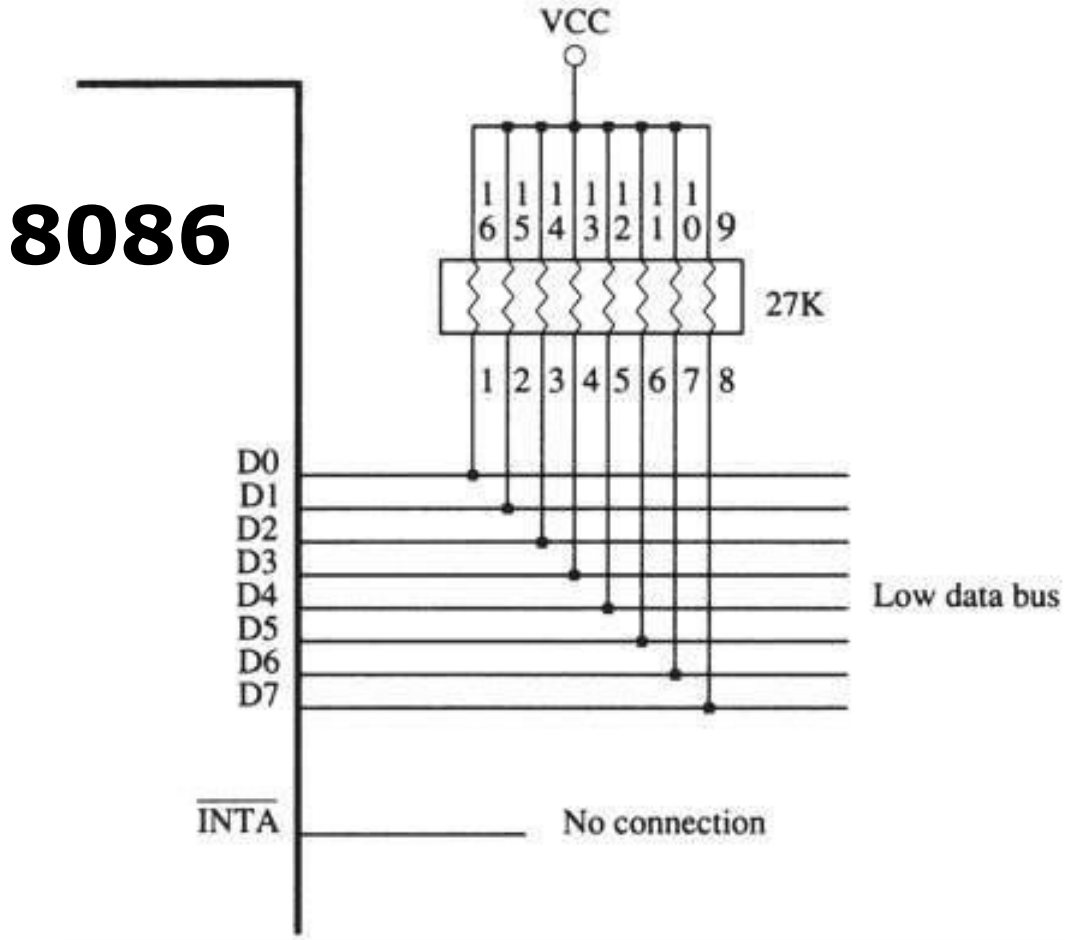


Fig: shows a circuit to appy interrupt vector type number FFH to the data bus in response to an INTR

Interrupt vector type generation

- Each switch connects through a 27kΩ pull-down resistor to ground, ensuring that when the switch is open, the corresponding data line reads as 0.

When the switches are closed, they connect the line to VCC, forcing a logic high (1).

- In the configuration shown, all switches are closed, so all data lines are at logic high, producing the FFH value (11111111) on the data bus.
- The output is enabled in response to the INTA (Interrupt Acknowledge) signal, placing the vector on the bus only during the interrupt acknowledge cycle.
- The system does not use a buffer or decoder, so this setup always delivers the same fixed vector

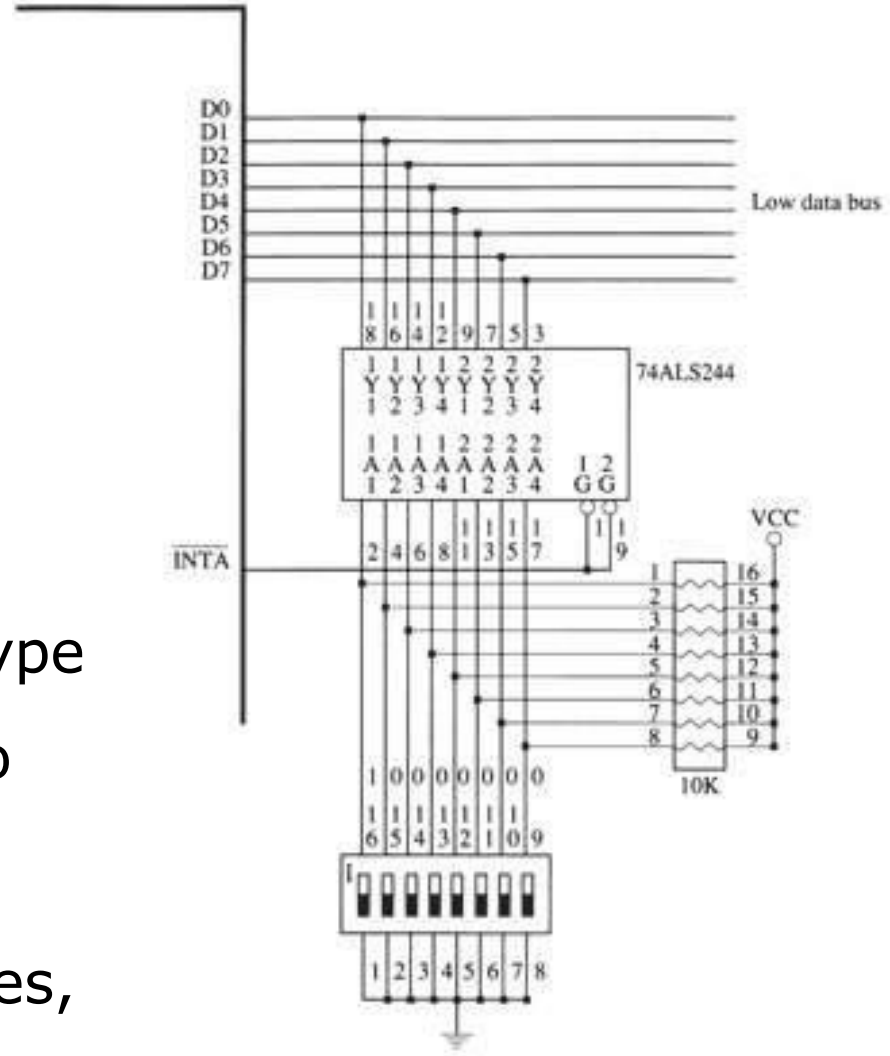
Interrupt vector type generation

Fixed Interrupt Vector Generator for 8086 (Using 74ALS244 + DIP Switches)

This circuit provides a fixed 8-bit interrupt vector (e.g., 80H) to the 8086 during an INTR interrupt using a 74ALS244 buffer and DIP switches, controlled by the INTA signal.

1.DIP Switches: Set the desired 8-bit interrupt type (e.g., 1000 0000 for 80H); each switch connects to VCC or GND via 10kΩ pull-down resistors.

2.74ALS244 Buffer: Takes input from DIP switches, outputs to the data bus only when enabled.



A circuit that supplies a fixed interrupt vector (e.g., 80H) to the 8086 in response to INTA.

Interrupt vector type generation

3.INTA (Interrupt Acknowledge) Signal:

This signal from the processor is connected to the enable pin (G1 or G2) of the 74ALS244. It enables the buffer to put the DIP switch value onto the data bus

8086 Action:

•. When INTR is asserted, the 8086 responds with INTA, which enables the buffer. The DIP switch value is then placed on the data bus (D0–D7) and read by the processor as the interrupt type. Each switch sets one bit via pull-down resistors. When INTA is inactive, the buffer enters high-impedance mode, leaving the bus unaffected.

•On INTR → sends INTA → reads vector from data bus → jumps to corresponding ISR.

Expanding Interrupt

Expanding the INTR input from one to seven interrupt request lines.

the circuit expands one INTR input to support multiple (up to 7) interrupt request lines using a 74ALS244 octal buffer and an 8-input NAND gate. Each IR line connects to the buffer and to the NAND gate. The NAND gate monitors all IR lines simultaneously.

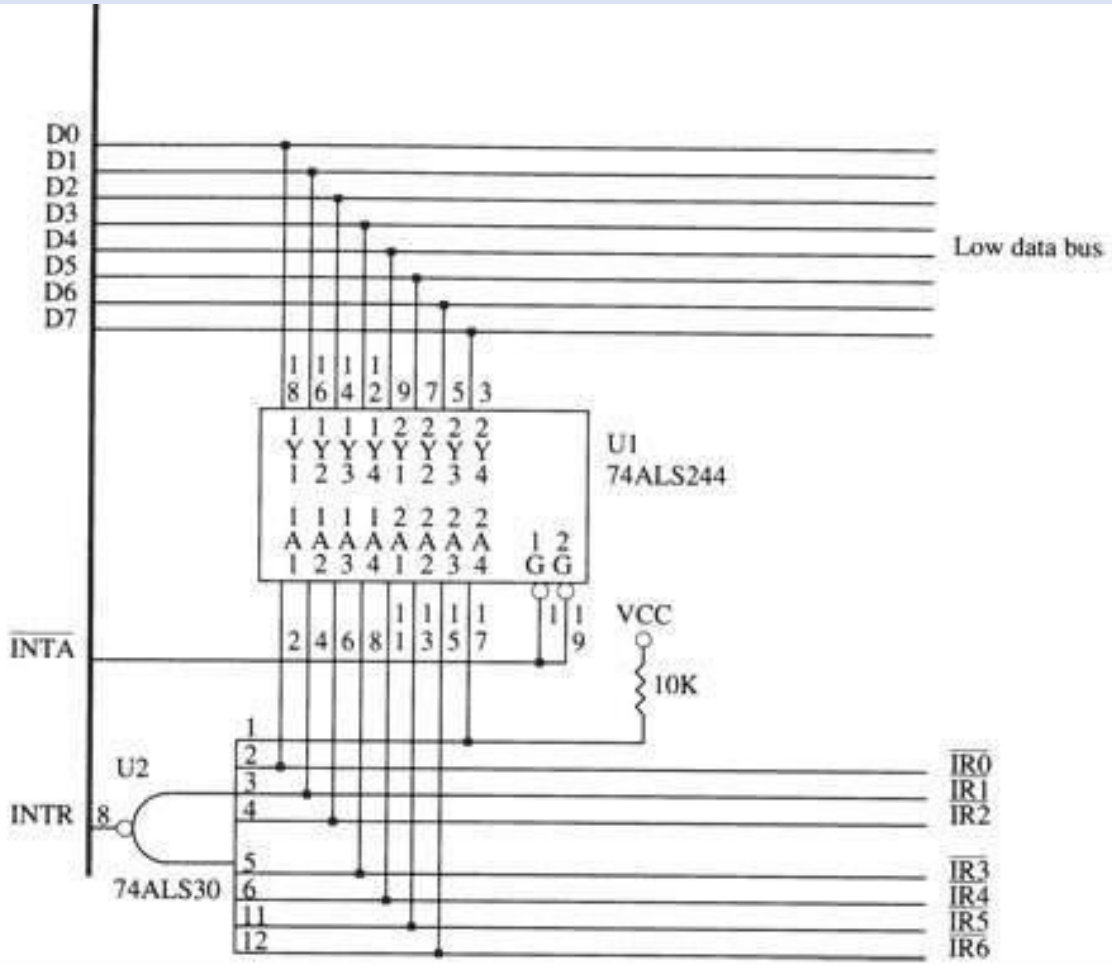


Fig: Expanding the INTR input from one to seven interrupt request lines.

Expanding Interrupt

Operation:

When any IR input goes low (logic 0), the NAND gate output goes high (logic 1), triggering the INTR line of the microprocessor. The processor responds to this interrupt and reads the data from the 74ALS244 buffer to determine which IR line caused the interrupt. The appropriate interrupt vector is then fetched and executed.

Expanding Interrupt:Daisy chain

In this setup, two 82C55 chips are used, each generating 2 interrupt signals, for a total of 4 interrupt sources. These interrupts are connected in a chain (daisy-chained), with the final signal going to the processor's single interrupt pin (INTR).

Since all interrupt outputs are combined into one line, the CPU sees only one interrupt signal and uses a single interrupt vector. To find out which device caused the interrupt, it polls each 82C55 in software. This method is hardware-efficient, using only one wire and vector, while still supporting multiple interrupt sources.

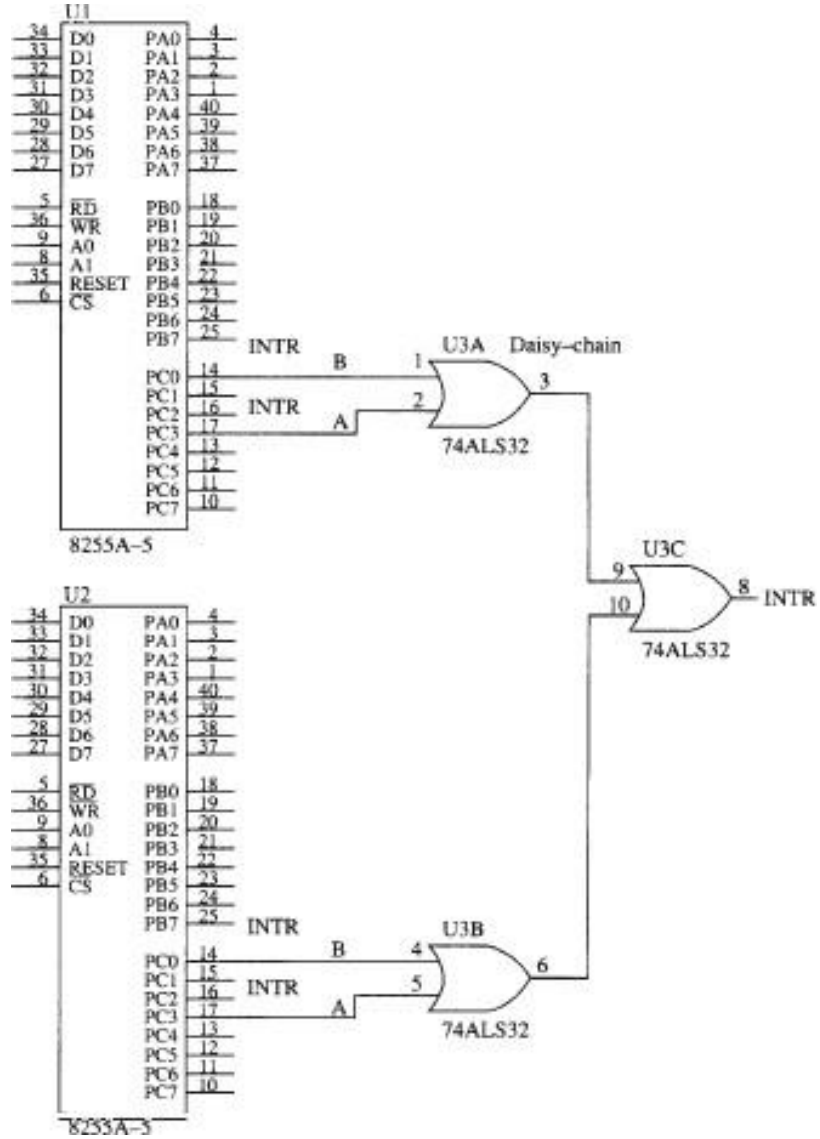


Fig:Two 82C55 PIAs connected to the INTR outputs are daisy-chained to produce an INTR signal

Ref:

Chapter 12:12-1,12-2,12-3

THE INTEL MICROPROCESSORS

BARRY B. BREY

Eighth Edition

CSE_3523

Segment-5

Microprocessor, microcontroller and embedded system

Course code: CSE-3523

Credit Hour: 3

Md. Mujibur Rahman Maruf

Lecturer

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Fundamentals of Computing

Multitasking

Process and thread

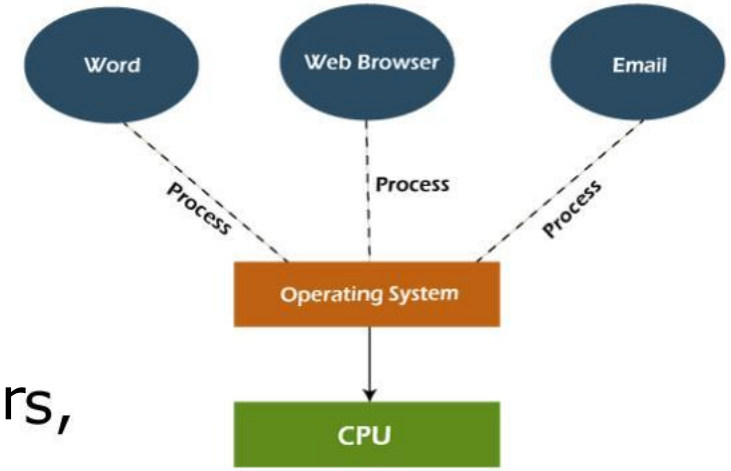
Multi-user System

VOID

Multitasking:

•**Multitasking** is the ability of an operating system to execute multiple tasks (also called *processes* or *programs*) seemingly at the same time.

•This doesn't mean all tasks run *exactly* at the same time, but the CPU switches between them so quickly that it appears simultaneous to the user.



Key Concepts:

•**Context Switching:**

- •The CPU saves the state of the current task (registers, memory pointers) and loads the state of the next task.
- •Enables efficient use of CPU time.

VOID

Multitasking:

- **Automatic Task Handling:**
  - The system interrupts the current program automatically, saves its state, and resumes another task.
  - This switching is handled by the OS without user interference.

multitasking systems, the CPU gives each process a small time slice. It switches tasks so efficiently that users don't notice interruptions. It's like juggling — only one ball is in hand at a time, but all seem active.

Real-World Examples:

Downloading a file while watching a video

Updating software while browsing the internet

Running antivirus scans while working on spreadsheets

VOID

Multi user System

A **multi-user system** is a computer system that allows **multiple users** to access and **use computing resources simultaneously or in turns**. These systems manage and isolate user activities, ensuring that one user's actions do not interfere with others.

Types of Multi-User Systems:

Time-Sharing Systems:CPU time is divided into small time slices and distributed among users.

Networked Systems: Multiple users connect to a central server over a network

Examples:

UNIX/Linux servers used in universities or companies

Mainframe computers in banking or government sectors

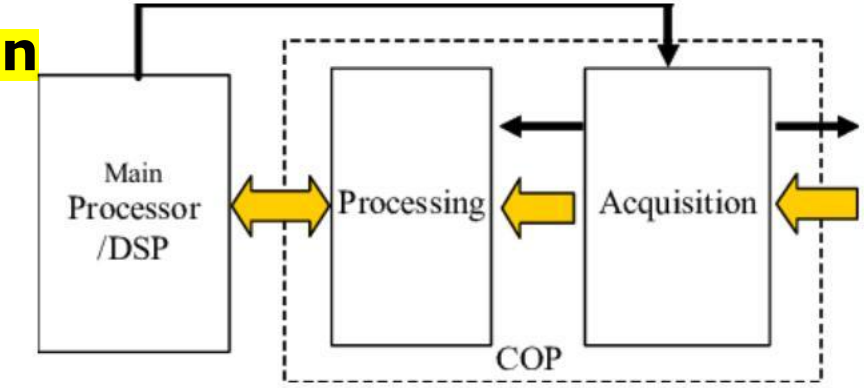
Remote desktop systems or virtual machines (VMs)

Performance Enhancers in Modern CPUs

- **Coprocessor**
- **Dual core Processor**
- **Cache memory**

Coprocessor:

A **coprocessor** is a *helper processor* that works with the **main CPU** to handle specific jobs more efficiently.



Why It's Used:

Think of the CPU as a *team leader* — it manages many tasks. But when tasks become *too complex or repetitive*, the CPU hands them off to a **coprocessor** — a *specialist* that does the job faster and better.

Main Purpose:

To **speed up performance** by handling:

1.Heavy calculations 2.Graphics rendering 3.Signal processing

Real Benefit:

Makes computer **run faster and smoother** by letting the CPU focus on general work, while the coprocessor handles the heavy lifting.

dual core Processor

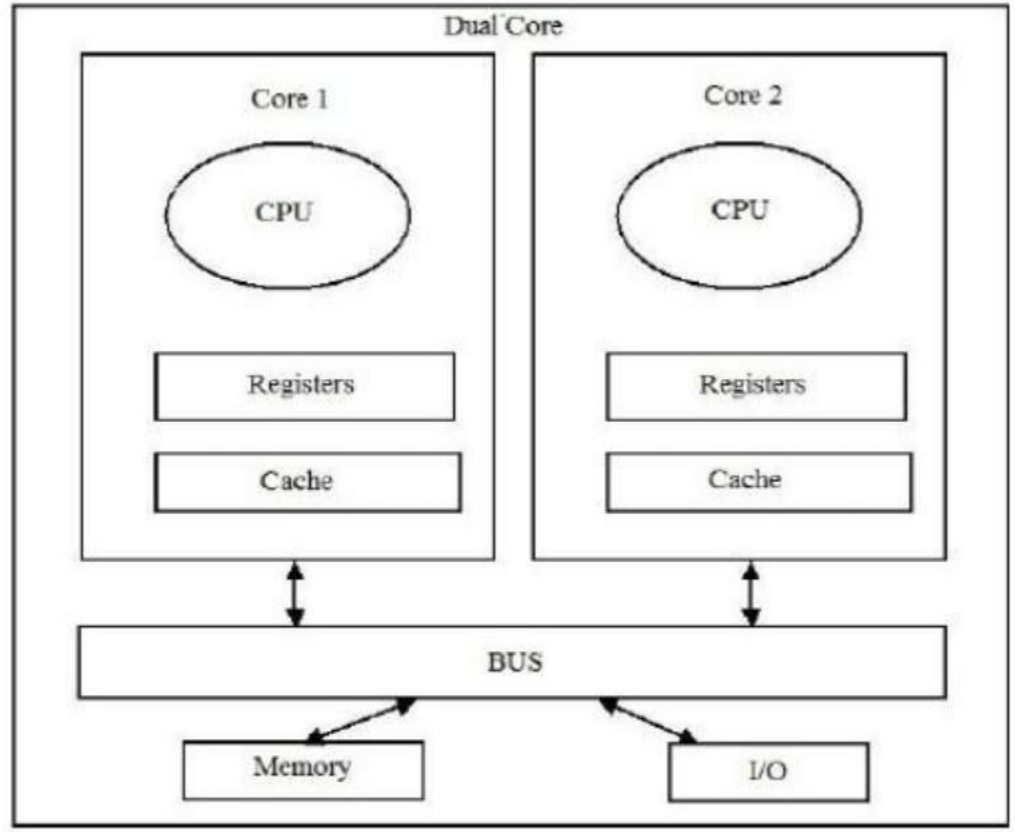
Dual Core Processor

A **dual-core processor** is a single chip that contains **two independent CPU cores**.

Each core can **execute tasks separately**, enabling **parallel processing** and improved **multitasking performance**.

Example:

Intel Core i3-2100: Dual-core processor from Intel's 2nd generation lineup



Cache memory

Cache Memory is a **small, high-speed memory** located close to the CPU that stores **frequently accessed data and instructions**.

Why Is Cache Memory Important?

Modern processors operate **much faster than main memory (RAM)**. If the CPU had to wait for data to be fetched from RAM every time, it would **slow down system performance significantly**. Cache memory reduces this delay by storing copies of frequently used data, allowing the CPU to access it **almost instantly**.

Characteristics:

Much faster than RAM

Smaller in size (usually measured in KB or MB)

Volatile memory (data is lost when power is off)

Types of Cache Memory

L1 Cache (Level 1):

Fastest and smallest

Integrated directly into the CPU chip

Stores very frequently accessed data

Size: Typically 16 KB to 128 KB

L2 Cache (Level 2):

Larger but slightly slower than L1

May be on the CPU chip or a separate chip

Stores recently used data

Size: 256 KB to 1 MB+

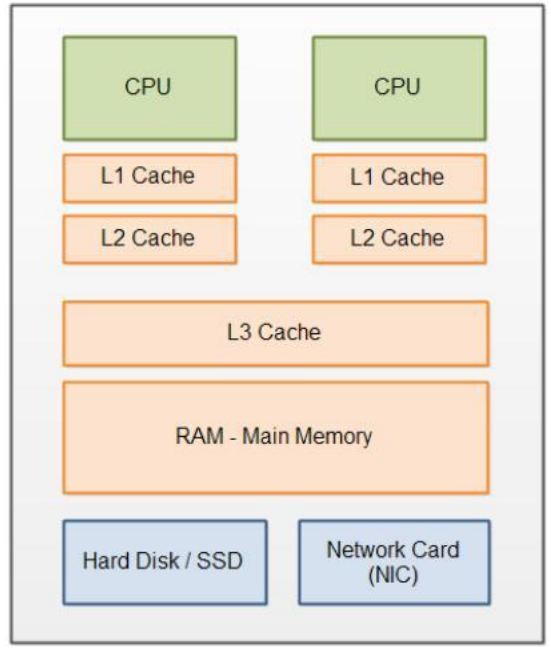
L3 Cache (Level 3):

Shared among cores in multi-core processors

Slower than L1 and L2 but faster than RAM

Helps improve overall performance

Size: 2 MB to 32 MB+

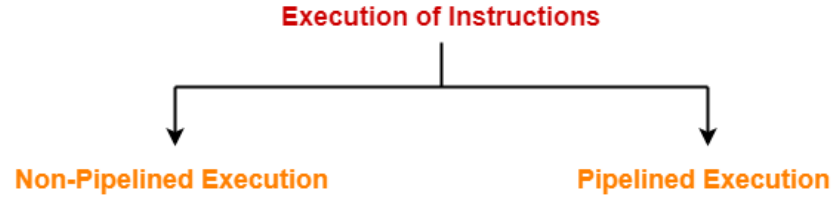


Instruction Execution Types

Pipelining

Instruction Execution Types

In computer architecture, instruction execution is classified into **two types** based on how the CPU processes instructions: **Non-Pipelining** and **Pipelining**. This depends on whether instructions are executed **sequentially** or **overlapped**, directly affecting CPU performance and efficiency.



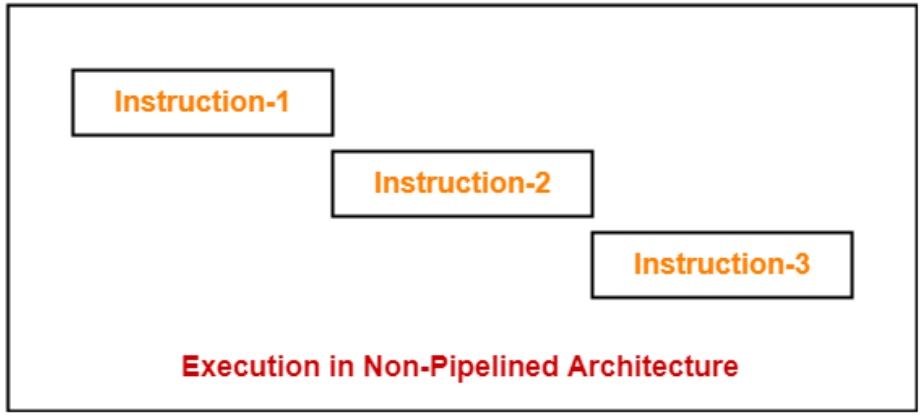
1. Non-Pipelining Architecture

Executes **one instruction at a time**

All stages (fetch → decode → execute → memory → write-back) must **finish before the next instruction starts**

Example:

If one instruction takes **5 clock cycles**, the next instruction starts **only after** those 5 cycles finish (non-pipelined).



What is Pipelining?

- Pipelining refers to dividing the execution of an instruction into multiple stages and executing these stages in parallel for different instructions. This approach reduces idle time of processor components and increases instruction throughput.

Key points:

- Instructions are divided into multiple stages
- Next instruction starts before previous finishes
- Improves hardware utilization
- **Pipeline Structure**
- Each stage has **input register + logic**
- Registers store **intermediate results**
- Output of one stage → **input of next**
- Ensures **synchronized data flow**

- **Example:**
- Factory: one car is painted, another assembled, and another inspected simultaneously.

Instruction Pipeline Stages

A typical instruction pipeline consists of five stages.

Each stage performs a specific function and allows different instructions to be processed concurrently.

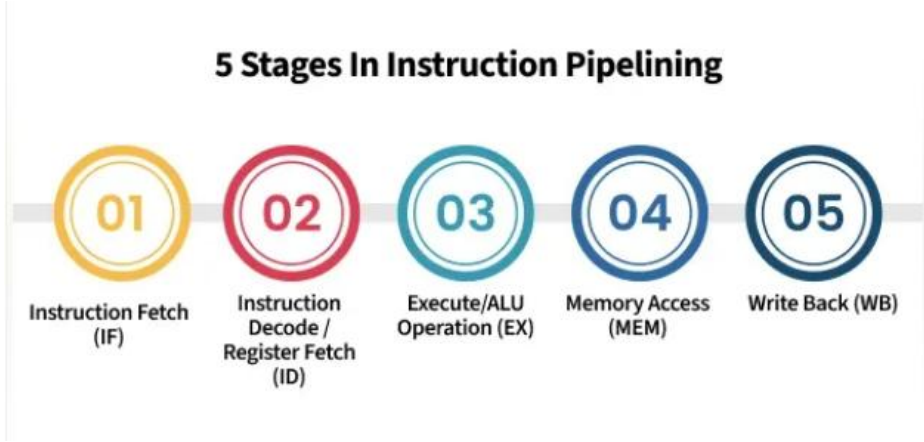
IF (Instruction Fetch): Fetch instruction from memory

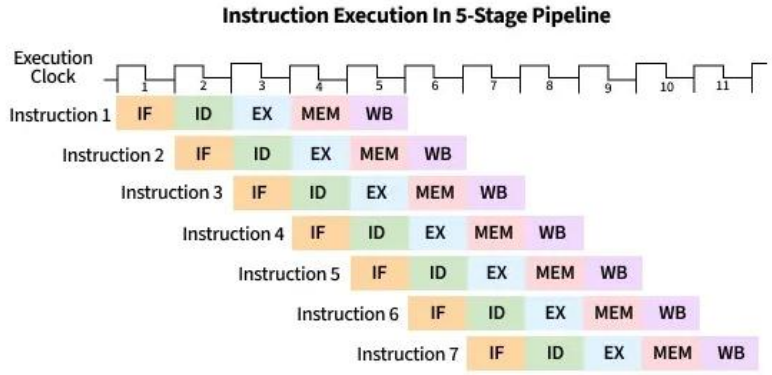
ID (Instruction Decode): Decode instruction, read registers

EX (Execute): Perform arithmetic or logic operation

MEM (Memory Access): Accesses memory if needed (for load or store instructions).(*No memory access needed, proceed to WB.)*

WB (Write Back): The result of the instruction is written back to the register file.





Types of Pipeline

Pipelining is mainly classified into two types based on the nature of operations being performed. Both types aim to improve performance through parallel execution.

- **Arithmetic Pipeline**
- **Instruction Pipeline**

Arithmetic Pipeline

An arithmetic pipeline divides an arithmetic problem into various sub problems for execution in various pipeline segments. It is used for floating point operations, multiplication and various other computations.

Floating point addition using arithmetic pipeline

The following sub operations are performed in this case

- Exponent comparison
- Mantissa alignment(Mantissa:significant digit of floating point numbers),
- Addition/Subtraction
- Normalization

example:ADD 2 number X and Y (X=0.3214*10^3 and Y=0.4500*10^2)

First of all the two exponents are compared and the larger of two exponents is chosen as the result exponent. The difference in the exponents then decides how many times we must shift the smaller exponent to the right. Then after shifting of exponent, both the mantissas get aligned. Finally the addition of both numbers take place followed by normalisation of the result in the last segment

Instruction Pipeline

Instruction Pipeline

Instruction pipelining overlaps different phases of instruction execution such as fetch, decode, and execute. This ensures continuous utilization of CPU components.

Overlaps **IF, ID, EX stages**

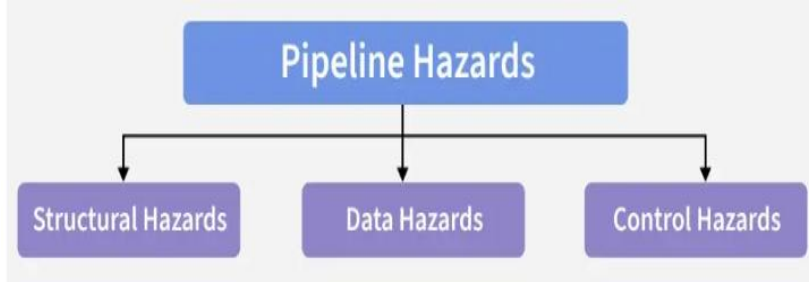
Executes multiple instructions **simultaneously**

Increases **CPU efficiency**

Common in **general-purpose processors**

Pipeline Hazards

Pipeline hazards are conditions that cause the pipeline to stall or produce incorrect results. These hazards prevent smooth execution of instructions.



- Occur due to **dependency or conflicts**
- Cause **pipeline stalls**
- Reduce performance if frequent
- Must be handled carefully

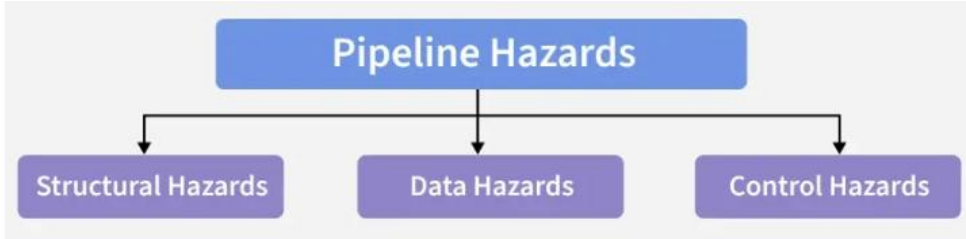
Types of Pipeline Hazards

There are different types of pipeline hazards depending on the source of the problem.

Data Hazard: A data hazard occurs when instructions exhibit data dependencies such that one instruction depends on the result of a previous instruction that has not yet completed in the pipeline

Pipeline Hazards

Control Hazard: A control hazard (also known as a branch hazard) arises when the flow of instruction execution is altered by branch or jump instructions. Since fetching the next instruction depends on the outcome of a branch, the pipeline may fetch the wrong instructions before knowing the actual branch direction



Structural Hazard: A structural hazard, also called a resource conflict, occurs when two or more instructions require access to the same hardware resource simultaneously, and the hardware cannot support the required parallel access

Advantages and Disadvantages of Pipelining

VOID

Advantages of Pipelining

Pipelining significantly improves system performance by increasing instruction throughput and better utilization of hardware resources.

- Higher instruction throughput
- Reduced processor cycle time
- Efficient use of CPU components
- Supports parallel processing

Disadvantages of Pipelining:pipelining also has some limitations related to complexity and control.

- Complex processor design
- Increased instruction latency
- Performance affected by branching
- Pipeline stalls reduce efficiency

Specialized Microprocessors

- **Embedded Microprocessor**
- **bit-slice microprocessor**
- **Arithmetic Microprocessor**

Embedded Microprocessor

Definition:

A microprocessor designed to perform specific control tasks within embedded systems.

Characteristics:

- Low power consumption
- Real-time operation
- Small size and cost-efficient
- Specialized instruction sets

Examples:

Intel 8051 — widely used microcontroller in embedded devices

ARM Cortex-M series — popular in modern IoT and mobile devices

Applications:

Home appliances (microwaves, washing machines)

Automotive control systems

Industrial automation

bit-slice microprocessor

A **bit-slice microprocessor** is a CPU design method that builds a processor by combining smaller "slice" modules—typically 4-bit Arithmetic Logic Units (ALUs)—to form a CPU of desired word length (like 8-bit, 16-bit, 32-bit, or more).

How It Works:

The processor is constructed from multiple identical bit-slice modules connected in parallel.

Each module processes a portion (slice) of the full data word (e.g., 4 bits).

Control logic manages the coordination between slices to execute operations across the full word size

Advantages:

Flexible CPU word size

Enables custom control logic

Good for specialized or high-performance CPUs

Examples:

AMD 2901 — 4-bit slice ALU used to build 16- or 32-bit processors

Arithmetic Microprocessor

What is an Arithmetic Microprocessor?

Specialized microprocessor or unit focused on arithmetic operations

Core components:

- **Arithmetic Logic Unit (ALU):** Performs addition, subtraction, and logic operations
- **Floating Point Unit (FPU):** Handles floating-point calculations (decimals)

Handles complex math operations like multiplication, division, square root

Essential for scientific, multimedia, and engineering applications

Importance:

- Speeds up mathematical computations compared to software-only methods
- Critical in digital signal processing, graphics, scientific simulations, cryptography

Examples:

- **Intel 8087:** Early math coprocessor for floating-point operations

CSE_3523

Segment-6
I/O operation

Microprocessor, microcontroller and embedded system

Course code: CSE-3523
Credit Hour: 3

Md. Mujibur Rahman Maruf

Lecturer

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Introduction to I/O Interface

VOID

The I/O interface allows communication between the microprocessor and external devices . ***IN*** and **OUT Instruction** transfer data between an I/O device and the microprocessor's accumulator (AL, AX or EAX).

IN/OUT Operation Examples Based on Data and Address Size

#	Address Type	Data Size	Example Instruction	Meaning
1	8-bit fixed	8-bit	IN AL, 60H OUT 60H, AL	I/O with fixed port 60H using AL
2	16-bit via DX	8-bit	MOV DX, 03F8H IN AL, DX OUT DX, AL	I/O with port in DX using AL
3	16-bit via DX	16-bit	MOV DX, 0378H IN AX, DX OUT DX, AX	I/O with port in DX using AX
4	8-bit fixed	16-bit	IN AX, 19H OUT 19H, AX	I/O with fixed port 0019H using AX

VOID

ressing Techniques for I/O Instructions

There are two ways to specify the address of an I/O port:

- **Fixed I/O Addressing (8-bit)**
  - The port address is within the instruction.(e.g. IN AL, 60H)
  - It appears on address lines A7–A0, while A15–A8 are 00000000b.
  - Used for accessing **first 256 ports** (00H–FFH).
  - used to access system board device,(e.g. keyboard.)
- **Variable I/O Addressing (16-bit)**
  - The port address is stored in the **DX register**.
  - The full 16-bit address appears on address lines A15–A0.
  - Enables access to **full 64K I/O space**.
  - used to access serial and parallel ports, harddrives, etc.

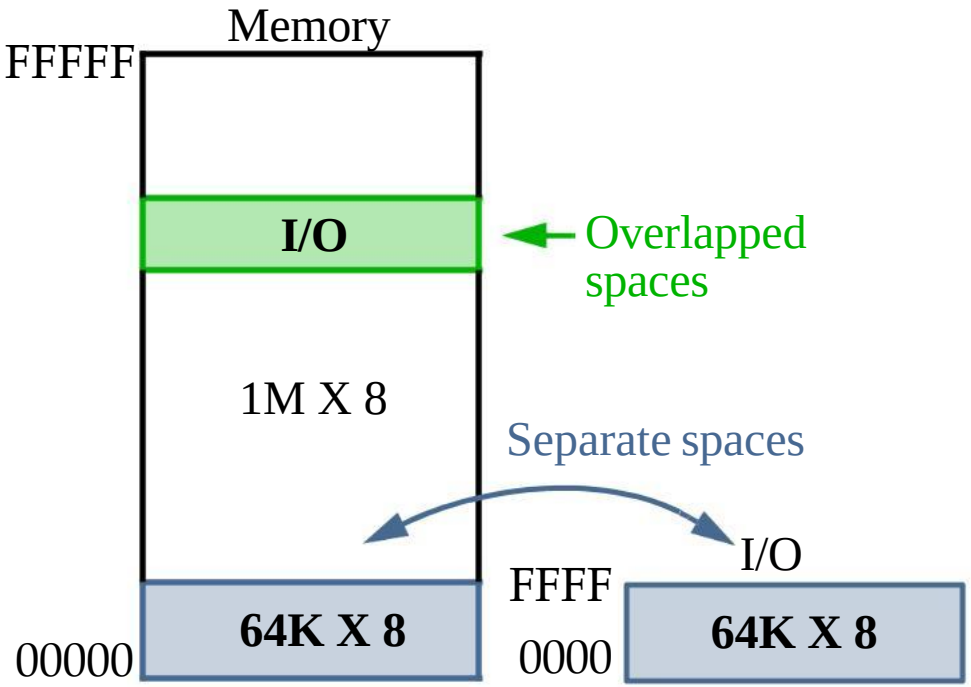
```
MOV DX, 0378H
IN AX, DX
```

Isolated Vs Memory Mapped

Isolated I/O

- Uses **separate address space** for I/O.
- Requires **special instructions**: IN, OUT.
- Hardware using M/IO andW/R needed to develop signals IORC
- and IOWC.

$\overline{RD}$	$\overline{WR}$	$IO/\overline{M}$	Signal
0	1	0	$MEMR$
1	0	0	ME**MW**
0	1	1	$\overline{IOR}$
1	**0**	**1**	$\overline{IOW}$



Memory-Mapped I/O

- I /O shares **memory address space**.

Uses **normal instructions**: MOV, ADD, etc.

IORC and IOWC not required.

- 8-bit port addresses used to access system board device, e.g. timer and keyboard.

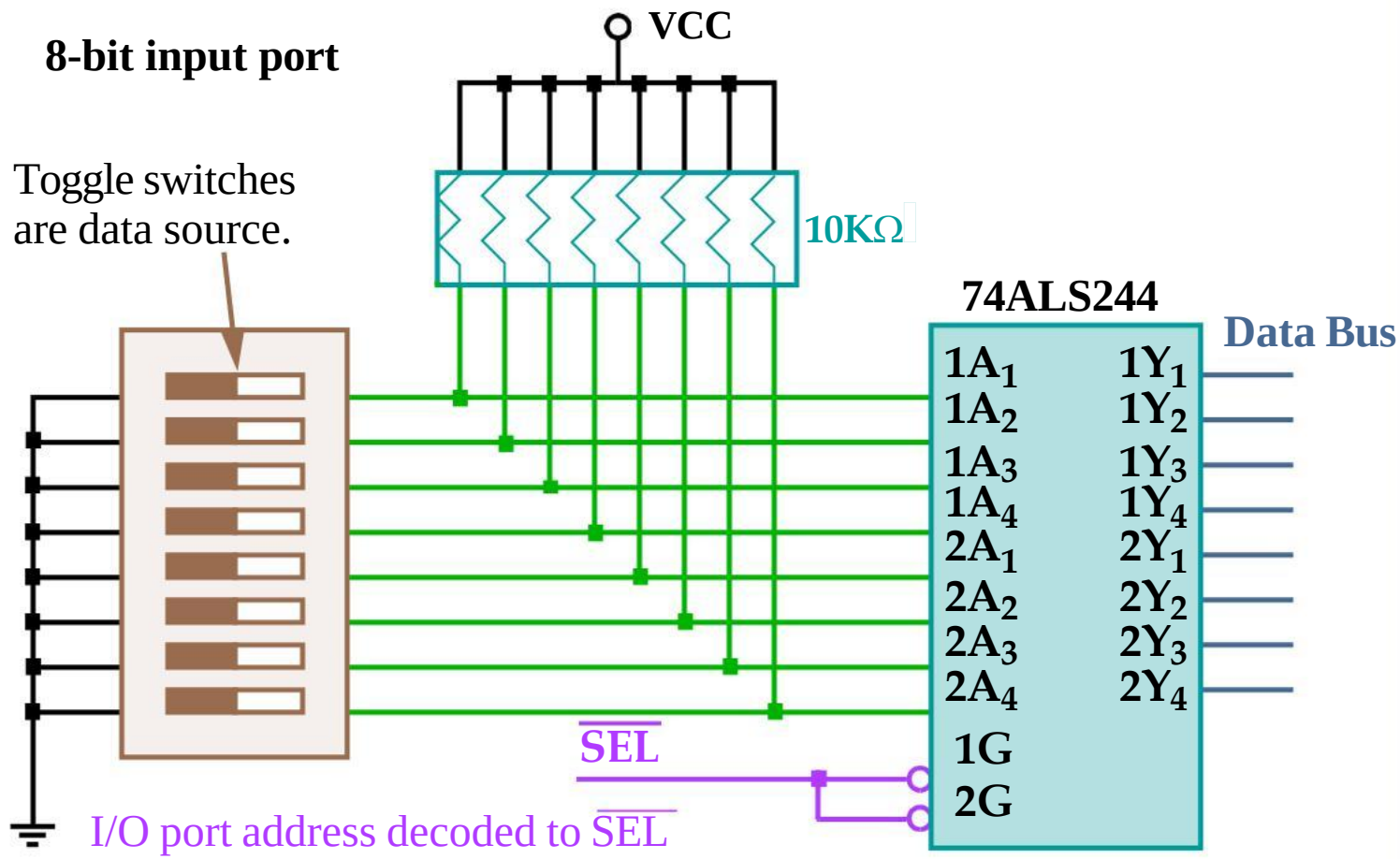
Basic I/O Interface

Basic Input Interface: The basic input device (to the microprocessor) is a set of tri-state buffers(**74ALS244**)

The **74ALS244 tri-state buffer** reads input from **toggle switches** and places 8-bit data on the CPU data bus.

A **decoded SEL signal** enables the buffer to transfer data during an IN instruction.

10kΩ pull-up resistors ensure proper logic levels for stable input.



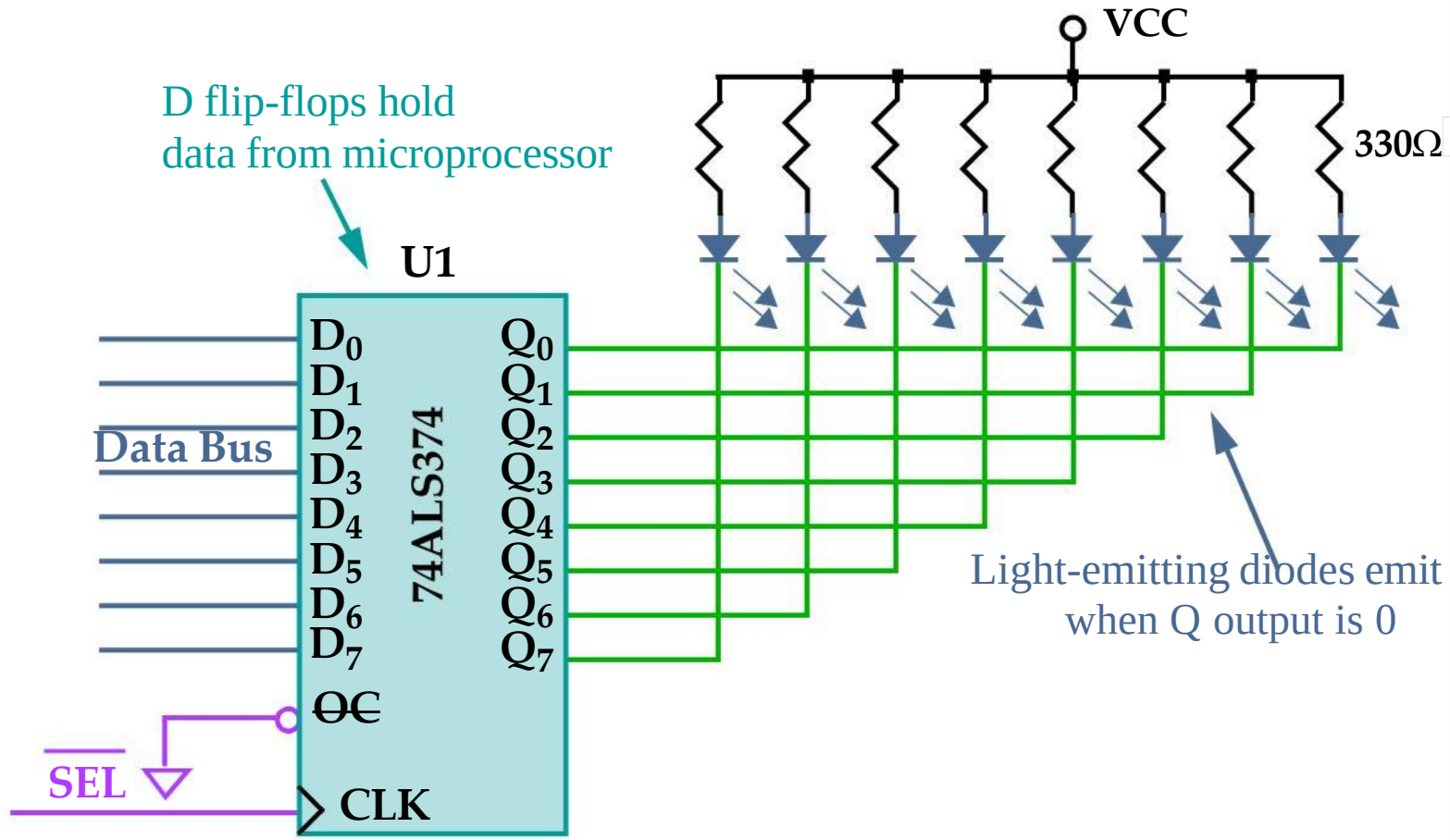
Basic I/O Interface

Basic Output Interface: The basic output device (from the microprocessor) is a set of latches.

ca ptures output data from the CPU using the OUT instruction.

A **SEL signal** triggers the latch to store data, which controls **LEDs** connected to its outputs.

330Ω resistors limit current, and LEDs turn ON when output is logic 0



Minimum Mode Memory Interfacing

In minimum mode, the **address bus and data bus are multiplexed** on the same lines (AD0 to AD15).

- During the first clock cycle, these lines act as a **memory/IO address bus**.
- For the second and third clock cycles, they function as a **data bus**, carrying data.
- **Demultiplexing** is the process of separating these address and data signals for read/write operations.

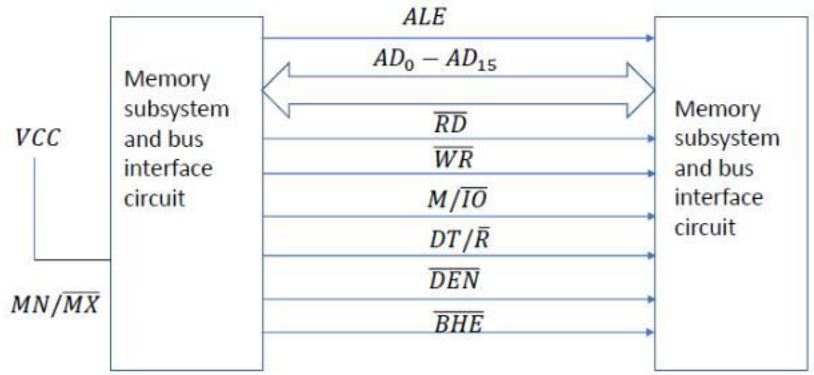


Figure (2) block diagram for minimum mode 8086 memory interface

The control signals in minimum mode are listed in table below

$\overline{RD}$	$\overline{WR}$	$IO/\overline{M}$	Signal
0	1	0	$\overline{MEMR}$
1	0	0	$\overline{MEMW}$
0	1	1	$\overline{IOR}$
1	0	1	$\overline{IOW}$

8086 Memory Organization

The 8086 microprocessor's memory address space has distinct logical and physical organizations.

- **Logical Organization**: The memory is seen as a **single 1M×8 memory bank**. Byte-wide storage locations are assigned consecutive addresses from 00000H to FFFFFH.

- **Physical Organization:** Physically, memory is implemented as **two independent 512 Kbyte banks**: the **low (even) bank** and the **high (odd) bank**

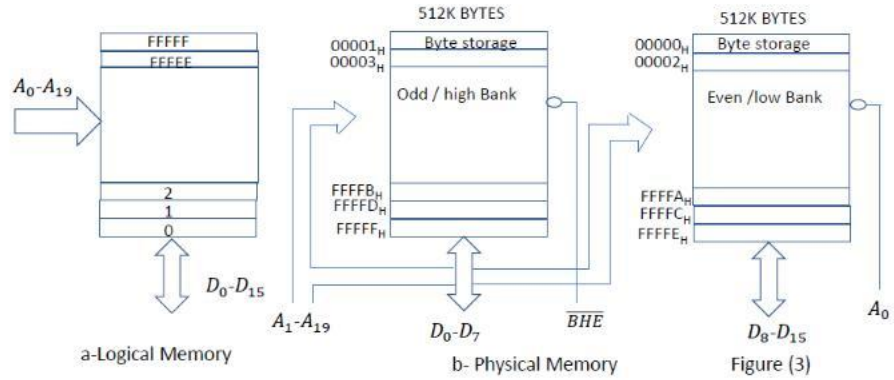


Figure (3)

$\overline{BHE}$	A_0	Function
0	0	Choose both odd and even memory bank
0	1	Choose only odd memory bank
1	0	Choose only even memory bank
1	1	None is chosen

Address Decoding

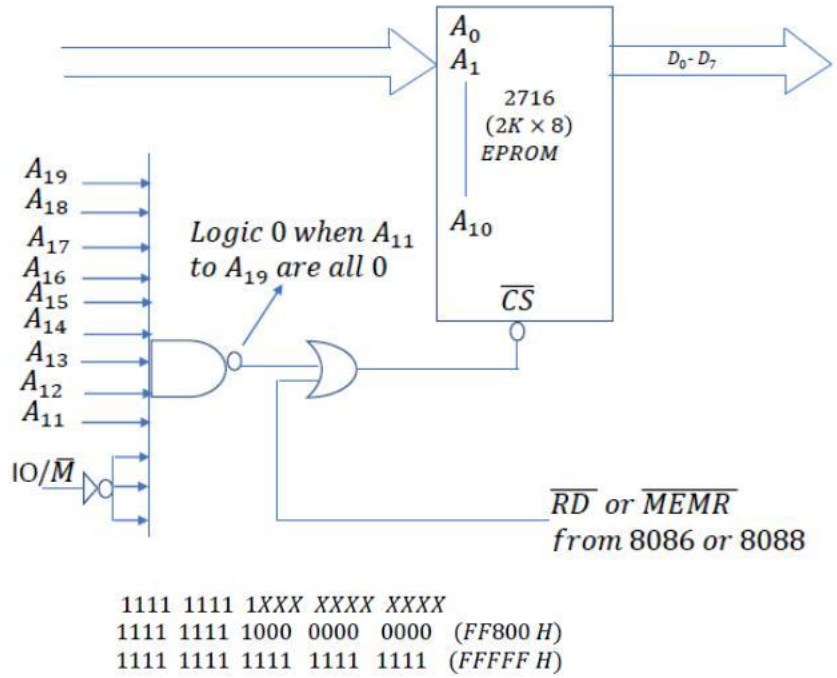
A microprocessor can typically address a much larger memory space than a single memory chip can cover. This is where **address decoding** becomes essential.

- **Purpose of Decoding**: Decoding ensures that a memory device functions at a **unique section or partition of the memory map**.
- **Without a Decoder**: Only one memory device could be connected to a microprocessor, severely limiting its utility.
- **Correcting Mismatches**: A decoder corrects mismatches when the microprocessor's address pins exceed the memory component's address pins. For instance, the 8088 has 20 address connections, while a 2716 EPROM has only 11.

NAND Gate Decoder

a 2K × 8 EPROM, like the 2716, being used with an 8088 microprocessor.

- The 8088's address connections A10–A0 are directly connected to the EPROM's address inputs A10–A0.
- The remaining nine address pins (A19–A11) are connected to a **NAND gate decoder**.

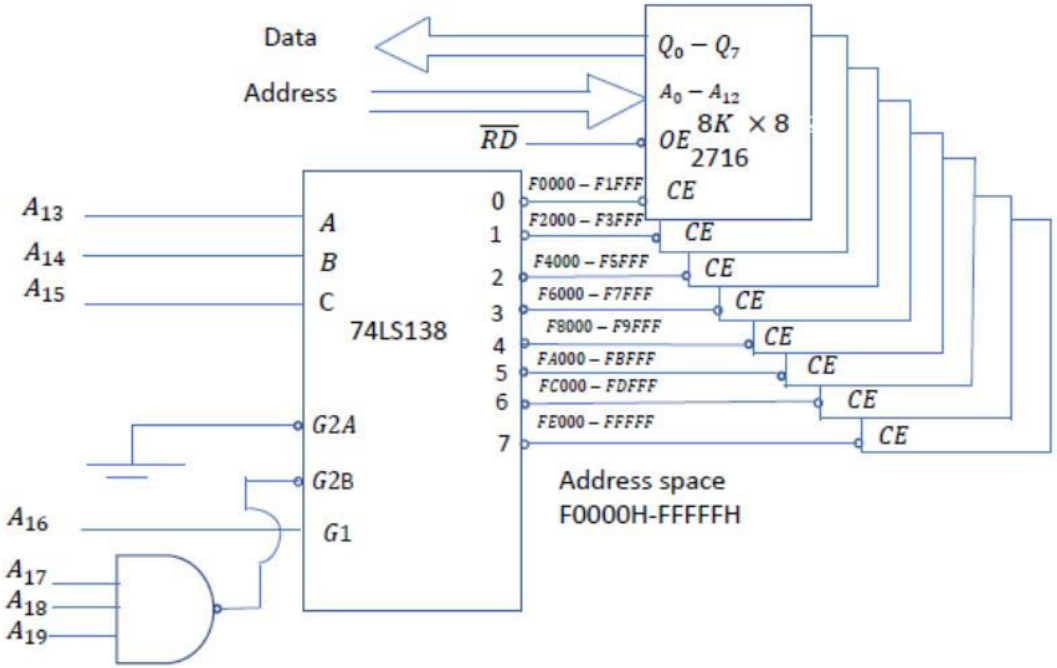


- This decoder selects the EPROM from one of the 2K-byte sections within the 8088's 1M-byte memory system.

A simple NAND gate decoder can be used to map a memory device to a specific address range, such as FF800H to FFFFFH

Using a 3-to-8 Decoder (74LS138)

For more complex memory systems, a **3-to-8 decoder** like the 74LS138 is often used.In a system with eight 2764 EPROMs a 64K × 8 memory section in an 8088 microprocessor:

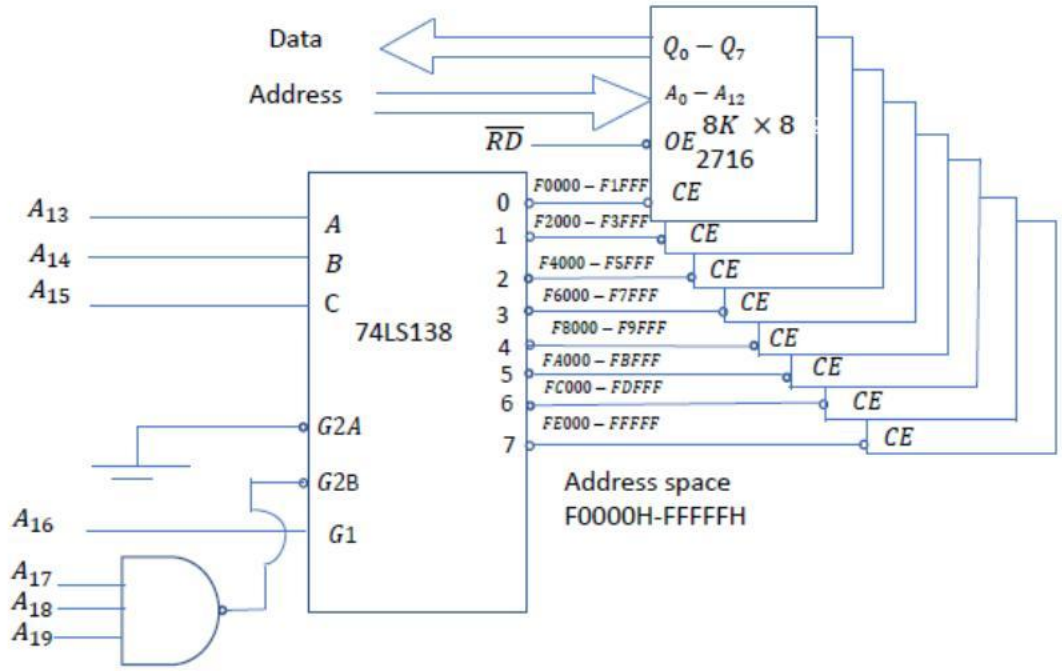


A circuit that uses eight 2764 EPROMs for a 64K × 8 section of memory in an 8088 microprocessor-based system. The addresses selected in this circuit are F0000H–FFFFFH.

- Address bits A19–A17 are connected to a three-input NAND gate. When all three are high, the NAND gate output goes low, enabling input G2B of the 74LS138.
- Input G1 is directly connected to A16.
- For this decoder to be enabled, the first four address connections (A19–A16) must all be high

Using a 3-to-8 Decoder (74LS138)

- Address inputs C, B, and A of the 74LS138 connect to microprocessor address pins A15–A13. These inputs determine which of the decoder's output pins goes low, selecting a specific EPROM.



Module 0
1111 000X XXXX XXXX XXXX
1111 0000 0000 0000 0000 =F0000H
1111 0001 1111 1111 1111 =F1FFFH
Module 7
1111 111X XXXX XXXX XXXX
1111 1110 0000 0000 0000 =FE000H
1111 1111 1111 1111 1111 =FFFFFH

Memory Expansion

There are two primary reasons to expand memory capacity beyond a single device:

1. The **byte-wide length is insufficient**.
2. The **total storage capacity isn't enough bytes**.

Both needs can be met by interconnecting multiple Integrated Circuits (ICs).

Memory Expansion

Example 1: Implementing 32Kx16 EPROM using 32Kx8 EPROMs

To implement a 32K × 16 EPROM using 32K × 8 EPROMs, you'd use two 32K × 8 chips in parallel.

- **A 32K × 8 EPROM** has 15 address lines (A0–A14) (2^{15}=32K).
- **A 32K × 16 EPROM** also needs 15 address lines (A0–A14).
- One 32K × 8 EPROM handles data lines D0–D7, and the other handles D8–D15 to achieve the 16-bit wide data path.

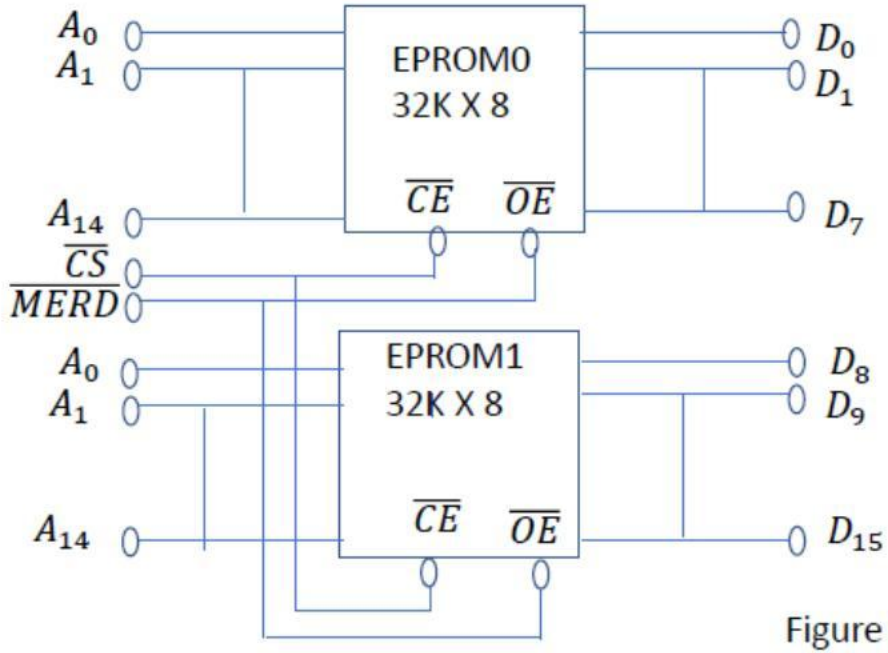


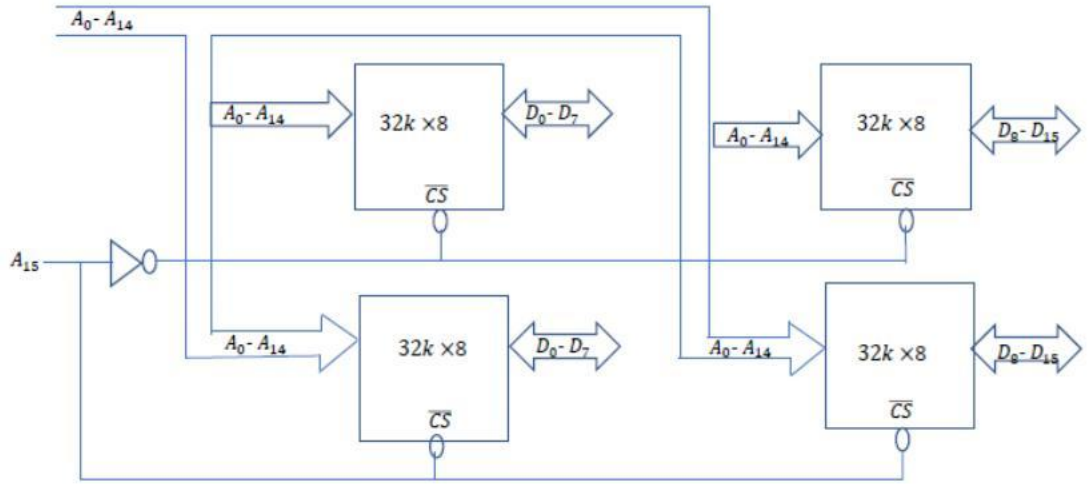
Figure (9)

Arithmetic Microprocessor

Example 2: Implementing 64Kx16 EPROM using 32Kx8 EPROMs

To implement a 64K × 16 EPROM using 32K × 8 EPROMs:

- A 32K × 8 bit EPROM has 15 address lines (A0–A14).



Fig

- A 64K × 16 bit EPROM requires 16 address lines (A0–A15) (2^{16}=64K).
- You would need **four** 32K × 8 EPROMs. Two chips form a 32K × 16 bank, and the A15 address line is used to select between two such 32K × 16 banks, effectively creating a 64K × 16 memory.

82C55 Programmable Peripheral Interface (PPI)

The **82C55 Programmable Peripheral Interface (PPI)** is a versatile interfacing component that connects I/O devices to a microprocessor. commonly found in PCs, often as part of an integrated chipset, for interfacing with the keyboard and parallel printer port

Group A

Port A (PA7-PA0) and upper half of port C (PC7 - PC4)

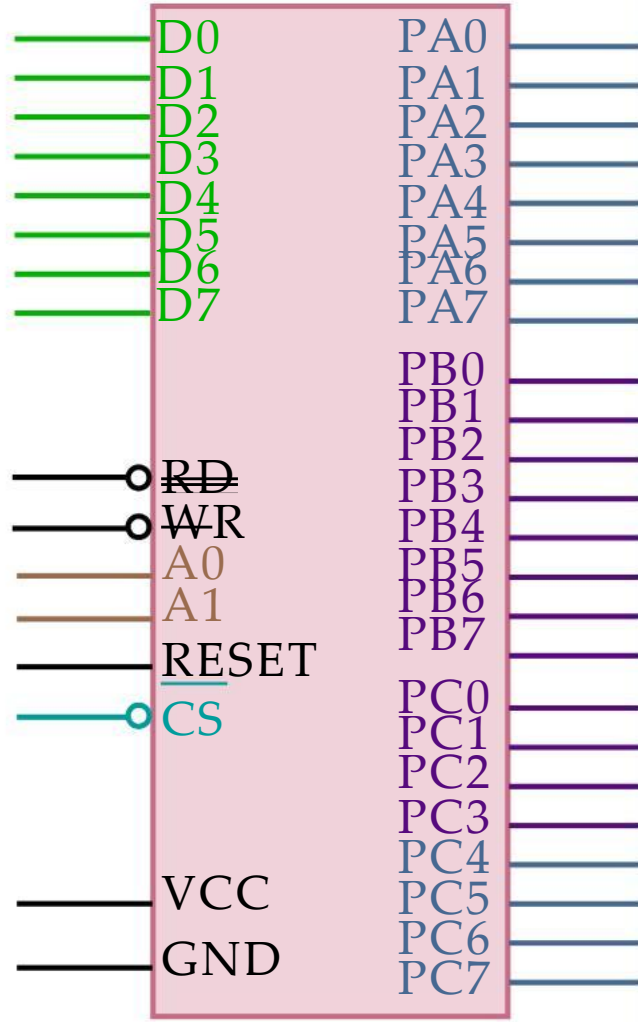
Group B

Port B (PB7-PB0) and lower half of port C (PC3 - PC0)

I/O Port Assignments

A_1	A_0	*Function*
0	0	Port A
0	1	Port B
1	0	Port C
1	1	Command Register

Pinout of 82C55 PPI



Operation Modes of 82C55

The 82C55 PPI operates in:

- **Mode 0 (Simple I/O)**: Provides basic, unhandshaked input/output for general-purpose parallel data transfer.
- **Mode 1 (Strobed I/O)**: Enables handshaked input/output for synchronized data transfer, using Port C for control signals.
- **Mode 2 (Bidirectional Strobed I/O)**: Configures Port A for bidirectional data flow with handshaking, suitable for bused communication.

Mode / Port	Port A (PA0-PA7)	Port B (PB0-PB7)	Port C (PC0-PC7)
Mode 0	8-bit Input or Output (simple I/O)	8-bit Input or Output (simple I/O)	Divided into two 4-bit nibbles, each Input or Output.
Mode 1	8-bit Strobed Input or Output (with handshaking)	8-bit Strobed Input or Output (with handshaking)	Provides handshaking signals for Port A & B.
Mode 2	8-bit Bidirectional Strobed I/O (Port A only)	Cannot be used as Mode 2	Provides handshaking & interrupt control for Port A.

Programming the 82C55

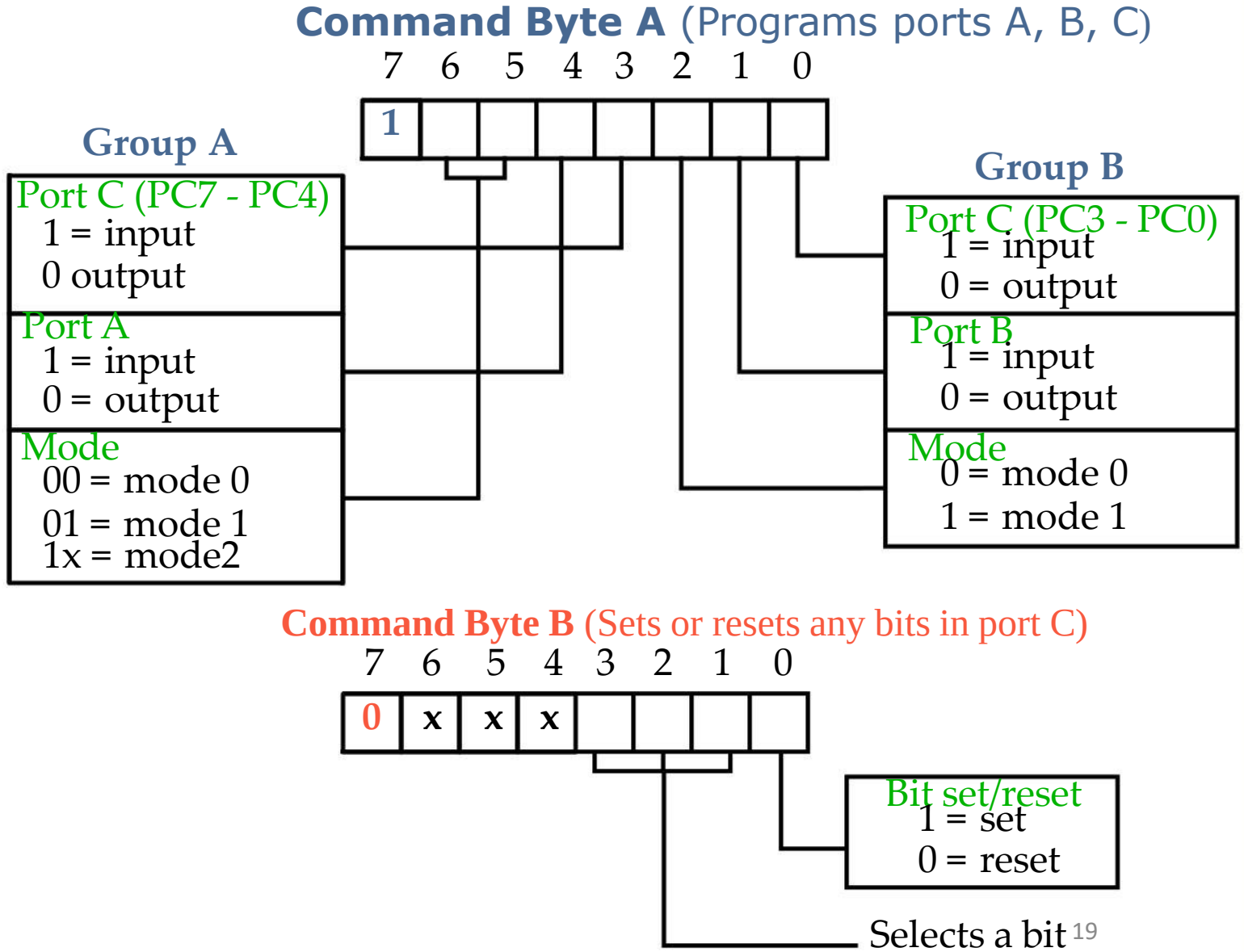
VOID

The command byte configures ports A, B, and C as **input or output** and selects Mode 0, 1, or 2.

It controls both **Group A (Port A + upper Port C)** and **Group B (Port B + lower Port C)**.

BSR mode is used to **set or reset individual bits** of **Port C only**, without affecting other ports.

It is selected when bit 7 of the command byte is **0**.



Handshaking

Handshaking is a technique used to **coordinate** data transfer between the CPU and slower I/O devices.It uses **control signals** to confirm that both the sender and receiver are ready before data is sent or received.This avoids data corruption, especially when devices operate at different speeds

Input Handshaking Example –Keyboard

When you press a key, the **keyboard** sends data to the 82C55 and signals it's ready.The 82C55 stores the key data and notifies the **CPU**, which then reads it safely.

Output Handshaking Example –Printer

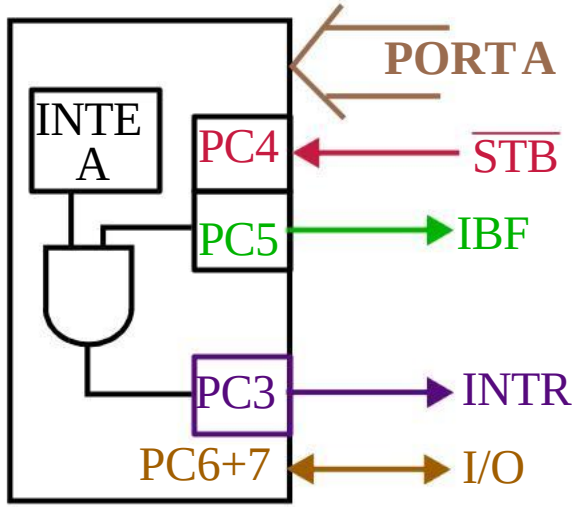
The **CPU** sends a character to the **printer** via 82C55.

The printer sends an **ACK** after receiving it, telling the CPU it's ready for the next one.

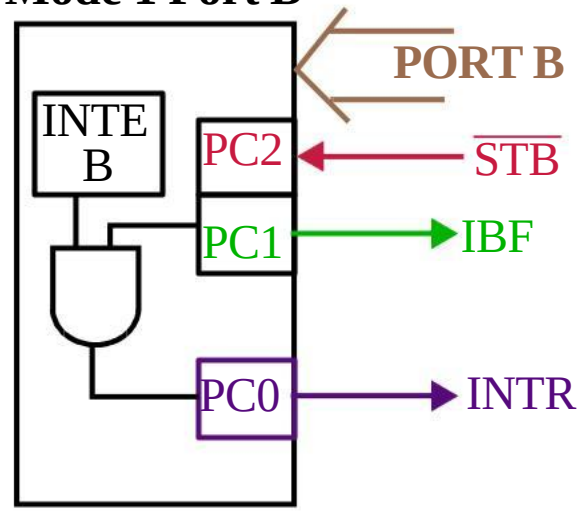
•Handshaking Input (Mode 1 Strobe Input)

82C55: Mode 1 Strobed Input

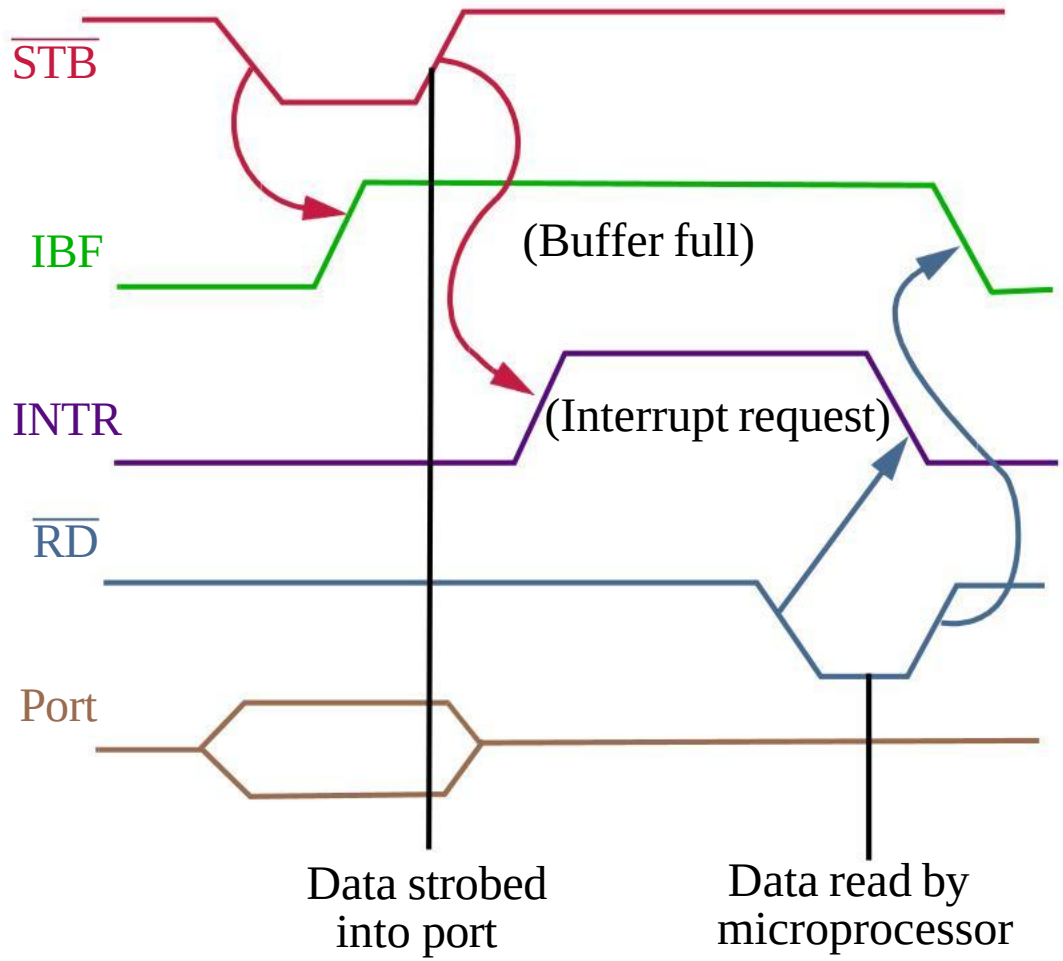
Mode 1 Port A



Mode 1 Port B



Timing Diagram



•Handshaking Input (Mode 1 Strobe Input)

1.STB (Strobe):An external device pulls **STB** low →this tells the 82C55 to latch the incoming data.Data must be valid before STB goes low and held until just after it returns high.

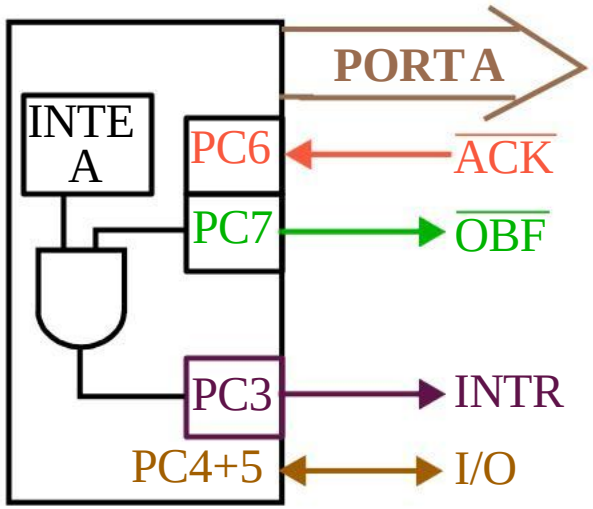
2.IBF (Input Buffer Full):After STB rises, **IBF** goes high to indicate that **valid data is latched** in the input port.It stays high until the CPU reads the data.

3.INTR (Interrupt Request):If interrupts are enabled (via **INTE**), the rising edge of IBF causes **INTR** to go high.This signals the CPU that input data is ready to be read.

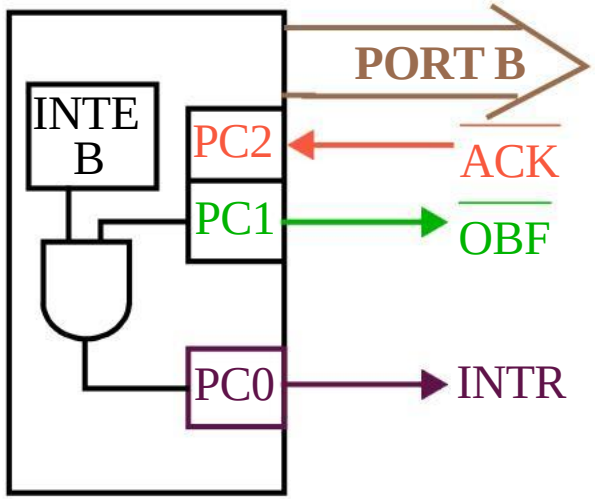
4.RD (Read):The CPU responds by pulling **RD** low to read the data from the port. After the read, IBF and INTR go low, completing the cycle.

•Handshaking Output (Mode 1 Strobe output)

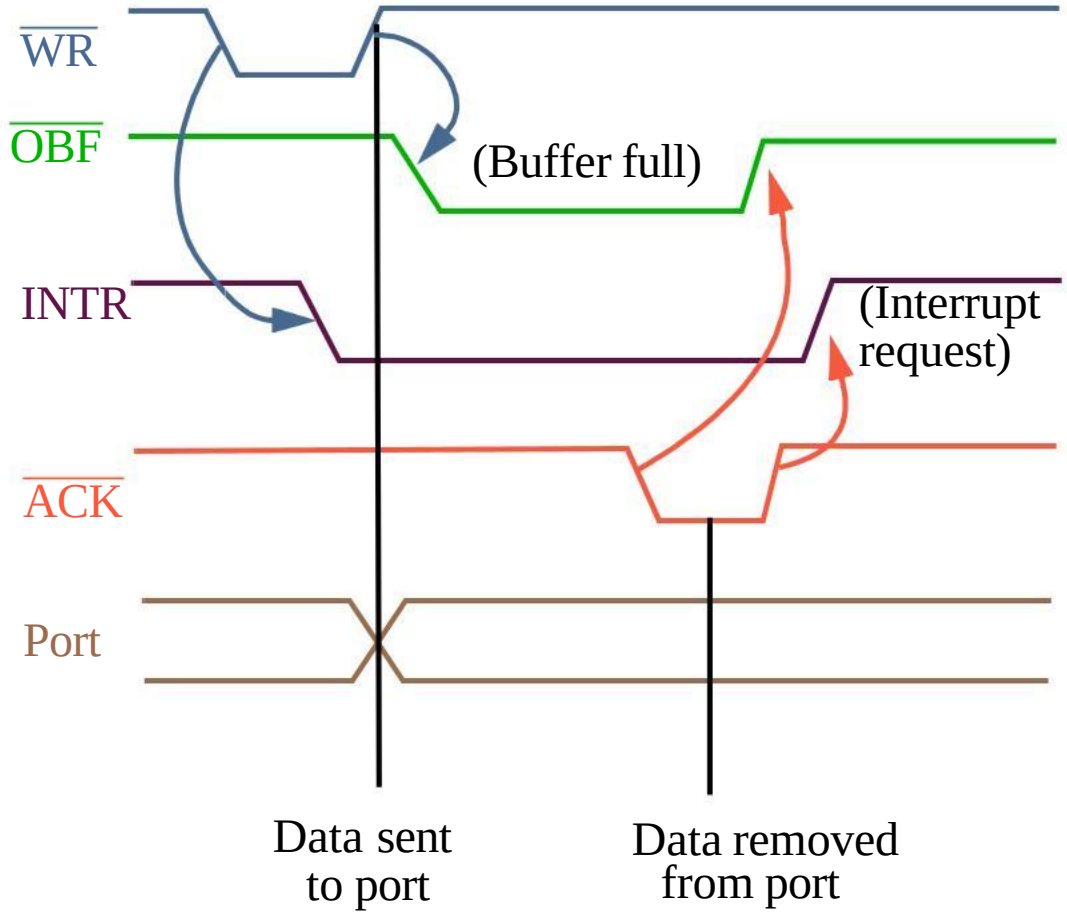
Mode 1 Port A



Mode 1 Port B



Timing Diagram



•Handshaking Output (Mode 1 Strobe output)

1.WR (Write):The CPU places valid data on the bus and pulls **WR** low to latch data into the output port.

2.OBF (Output Buffer Full):After WR goes high, **OBF** goes low to indicate that the port holds **valid output data**.It remains low until the external device acknowledges the data.

3.ACK (Acknowledge):When the external device receives the data, it pulls **ACK** low.This tells the 82C55 that the data was accepted.

4.INTR (Interrupt Request):If **interrupts are enabled** (via INTE), when ACK occurs and OBF goes high again, **INTR** is triggered.

This notifies the CPU that the device is ready for the next data.

CSE_3523

Segment-7 (Part-1)
Microcontroller

Microprocessor, microcontroller and embedded system

Course code: CSE-3523
Credit Hour: 3

Md. Mujibur Rahman Maruf

Lecturer

Dept. of CSE,IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Introduction to the 8051 Microcontroller

What is a Microcontroller?

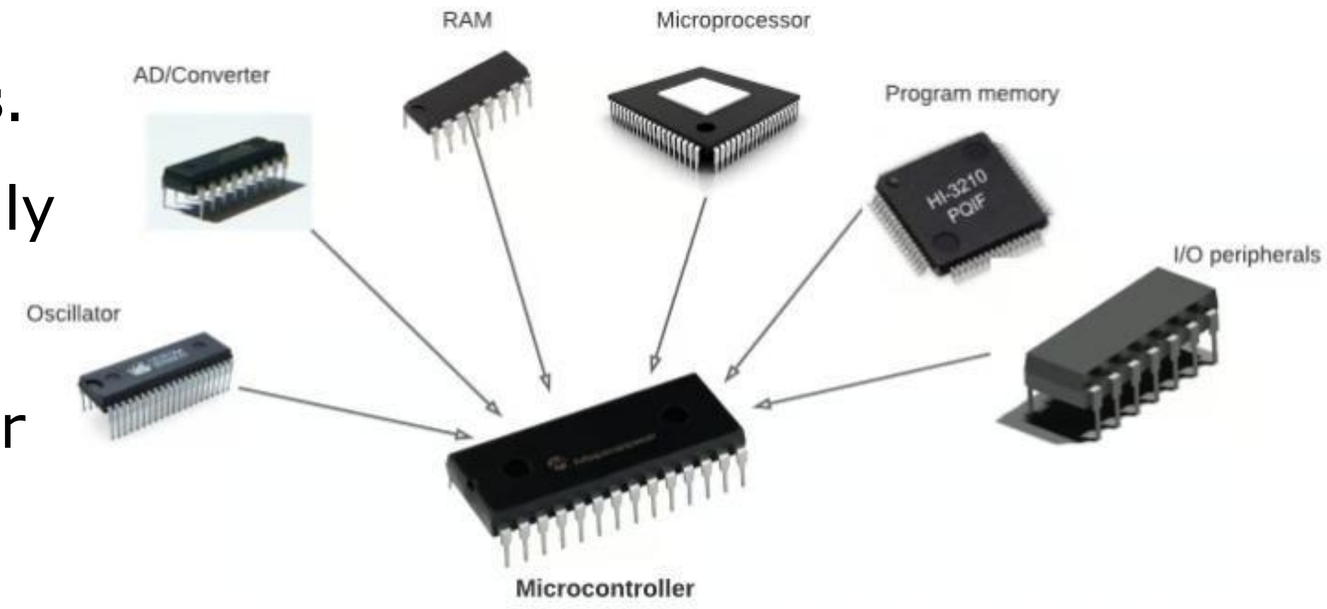
- A microcontroller is a **computer on a single chip**.
- It includes a **CPU, RAM, ROM, I/O ports, timers, and serial communication** interfaces—all built-in.
- Designed for **embedded applications** (washing machines, cameras, robotics).

Why 8051?

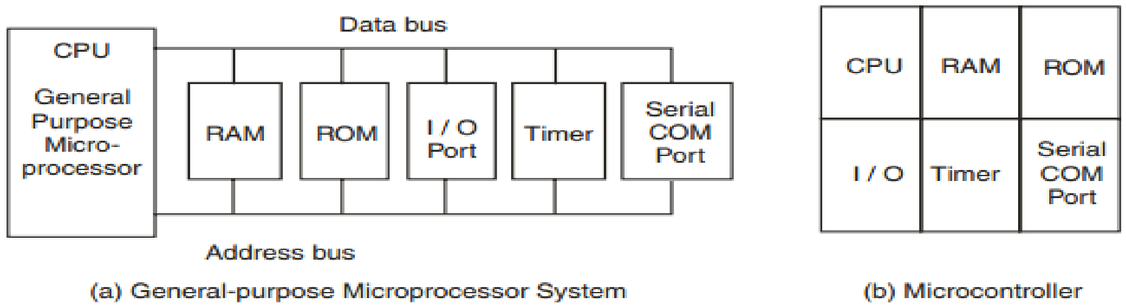
Developed by **Intel in the late 1970s**.

Most popular 8-bit microcontroller family (**MCS-51 series**).

Dominates ~40% of the microcontroller market.



Microprocessor Vs Microcontroller



(a) General-purpose Microprocessor System (b) Microcontroller

Fig. 1.1 Structure of microprocessor and microcontroller

Aspect	Microprocessor	Microcontroller (8051)
Integration Level	CPU only – external components needed (RAM, ROM, I/O, etc.)	CPU + RAM + ROM + I/O + Timers + Serial Port (on-chip)
System Design	Needs external interfacing for full functionality	All essential components are embedded in one chip
Memory	External RAM and ROM must be interfaced	128B RAM and 4KB ROM are inbuilt
Input/Output Ports	Must interface external devices like 8255 for I/O	Four 8-bit I/O ports available internally
Timers and Serial	External chips required (e.g., 8253 for timer, USART for serial)	Built-in timers and serial communication pins (T0, T1, TXD, RXD)
Cost and Size	Higher cost and larger board size	Lower cost, compact size ("System on Chip")
Application Suitability	Suitable for complex, high-processing systems	Best for small, real-time control applications
Example	Intel 8085, 8086	Intel 8051

Application of 8051

VOID

Energy Management

- Used in energy meters for homes and industries
- Enables precise energy consumption measurement
- Enhances smart meter performance

Touch Screens

- Integrated into portable devices (e.g., phones, media players, gaming devices)
- Many microcontroller brands include touch sensing
- Enables interactive and responsive UIs

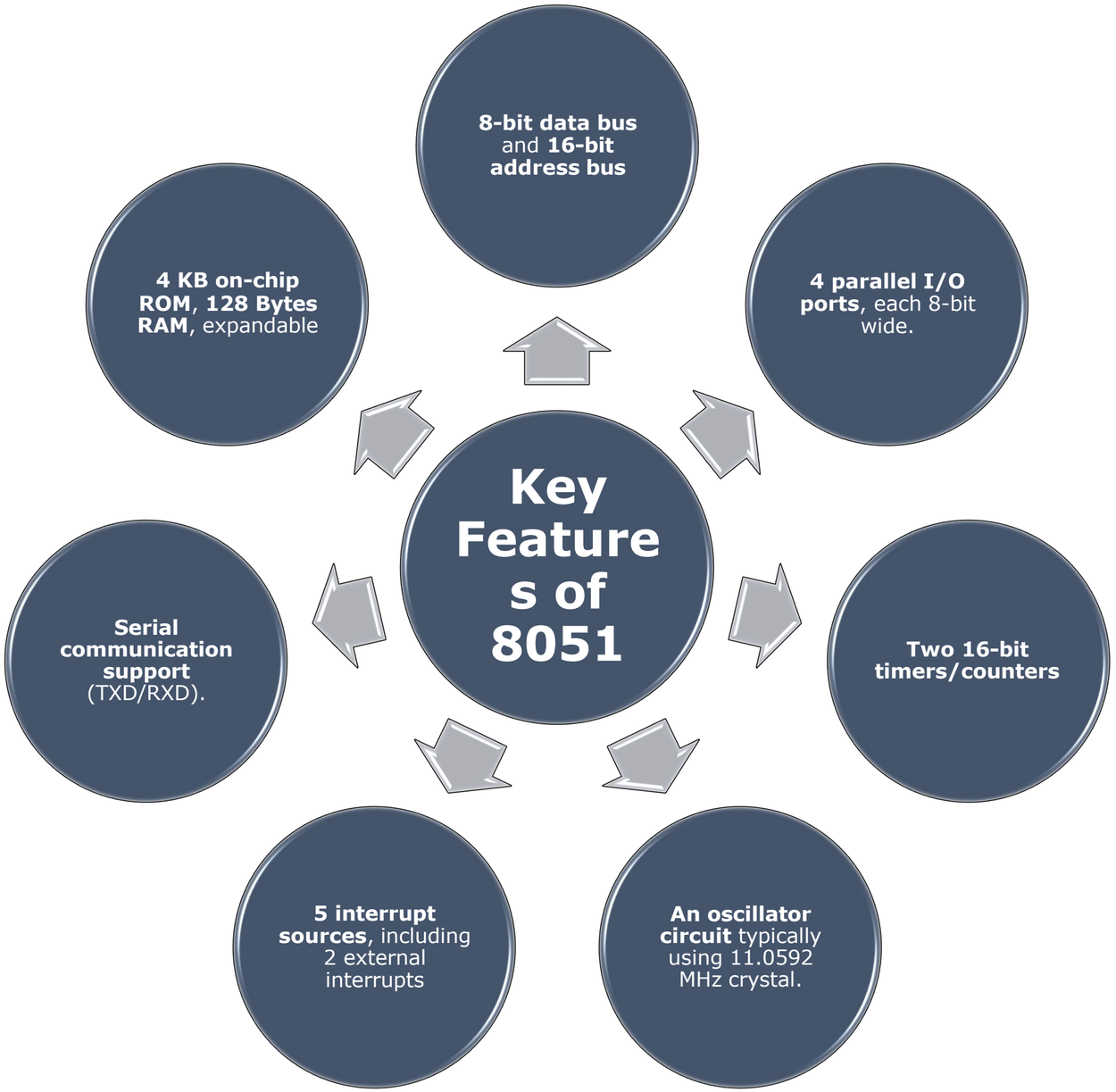
Automobiles

- Controls engine functions in hybrid vehicles
- Powers cruise control and anti-lock braking systems
- Offers efficiency and reliability

Medical Devices

- Embedded in devices like glucose and blood pressure monitors
- Displays accurate readings
- Ensures dependable healthcare diagnostics

Key Features of 8051 Microcontroller



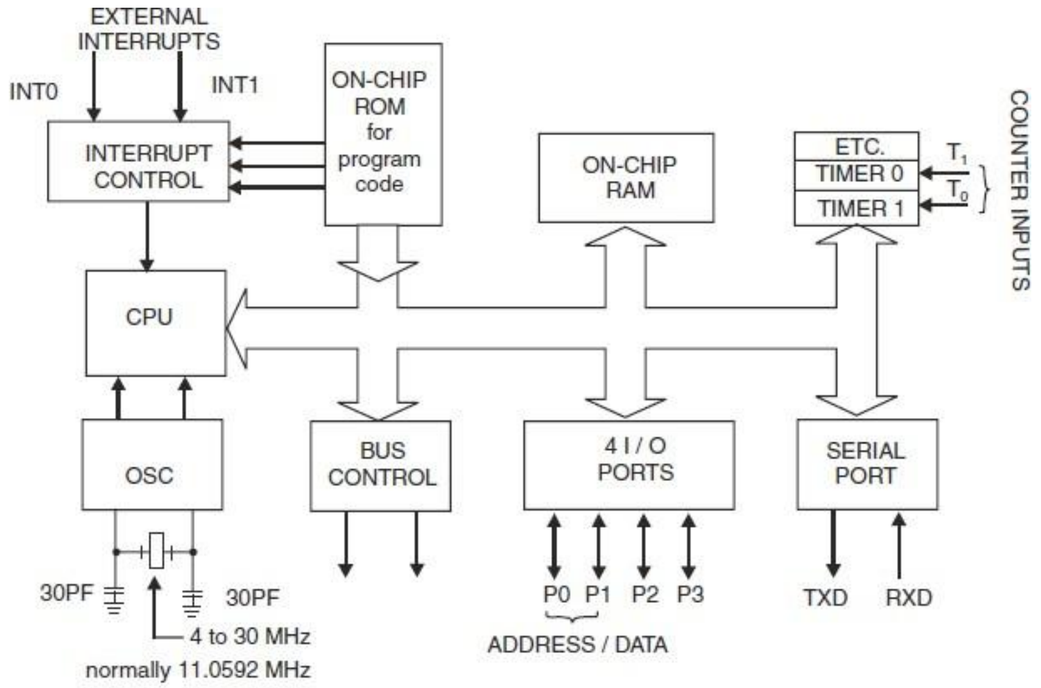


Fig. 1.2 8051 architecture

Pin Diagram of 8051

Port 0 (P0.0 – P0.7) Pins 32 to 39

Works as input/output

Also used for **address/data** in external memory mode

Needs **external pull-up resistors** when used as I/O

Port 1 (P1.0 – P1.7) Pins 1 to 8

Simple input/output port

No alternate functions

Has **internal pull-up resistors**

Port 2 (P2.0 – P2.7) Pins 21 to 28

Works as input/output

Used for **higher address lines** (A8–A15) in external memory mode

Has **internal pull-up resistors**

1.5 PIN DIAGRAM OF 8051

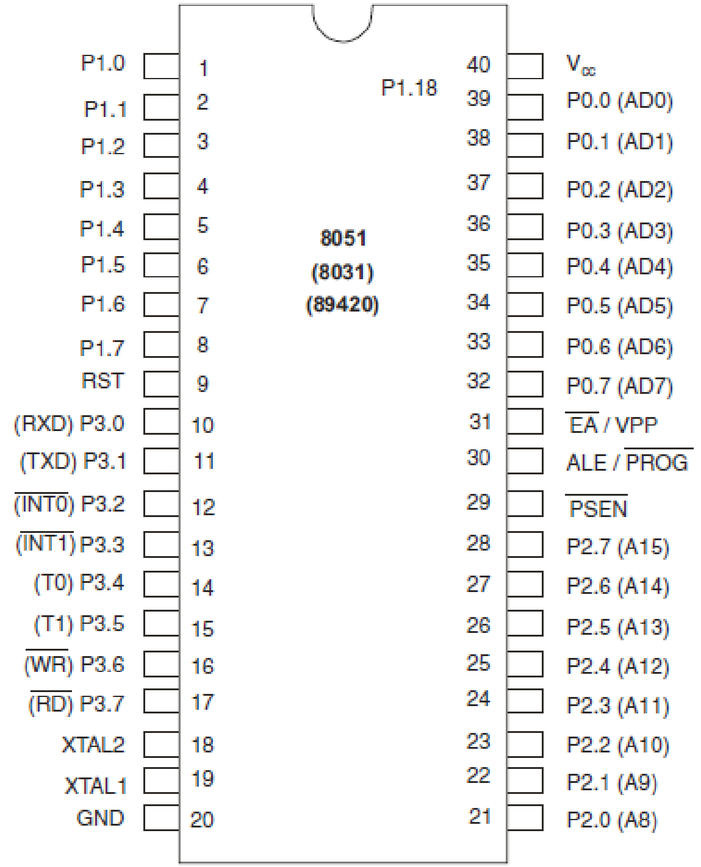


Fig. 1.3 Pin diagram of MC 8051

Port 3 (P3.0 – P3.7) – Simple Description Pins 10 to 17

Port 3 is an **8-bit input/output port**

Each pin also has a **special alternate function**

Commonly used for **interrupts**, **serial communication**, **timers**, and **external memory control**

Has **internal pull-up resistors**

Like other ports, it's also **bit-addressable** (each pin can be controlled individually)

Pin	P3.x	Alternate Function	Purpose
10	P3.0	RXD	Serial data input
11	P3.1	TXD	Serial data output
12	P3.2	INT0	External interrupt 0
13	P3.3	INT1	External interrupt 1
14	P3.4	T0	Timer 0 external input
15	P3.5	T1	Timer 1 external input
16	P3.6	WR	Write signal for external memory
17	P3.7	RD	Read signal for external memory

Port 3 (P3.0 – P3.7) – Simple Description Pins 10 to 17

Power and Clock Pins

Pin	Name	Function
40	Vcc	Power supply (+5V)
20	GND	Ground
19	XTAL1	Input to internal oscillator amplifier
18	XTAL2	Output from oscillator circuit

Control Pins for Memory Interfacing

Pin	Name	Function
9	RST	**Reset input** (active HIGH) – resets the microcontroller
31	EA/VPP	**External Access** – If low, MCU fetches code from external ROM
30	ALE	**Address Latch Enable** – Used to demultiplex address/data from Port 0
29	PSEN	**Program Store Enable** – Active LOW signal for reading external program memory

Architectural Block Diagram of 8051

- **Component of 8051**
- Arithmetic Logic Unit (ALU)
- Accumulator (Register A)
- Register B
- Data Pointer (DPTR)
- Program Counter (PC)
- Input Output ports
- Timer and Counter
- Serial port,Oscillator,Interrupt
- RAM
- ROM
- Program Status Word (PSW)
- Special Function Registers (SFR)

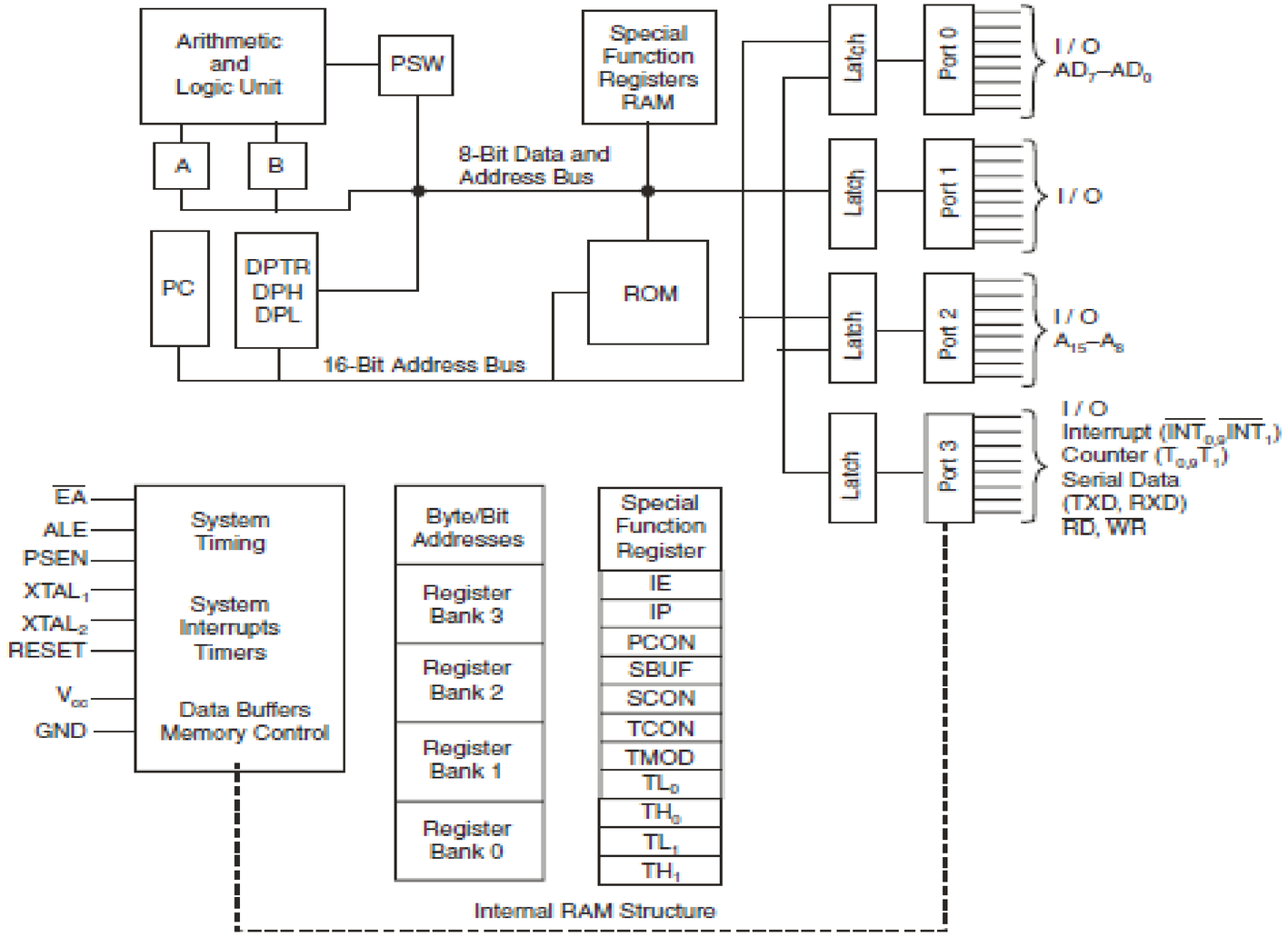


Fig. 1.4 Architectural block diagram of microcontroller 8051

8051 Components: ALU,A Register ,B Register

ALU – Arithmetic Logic Unit

- Performs all arithmetic (+, -, etc.) and logic (AND, OR, etc.) operations
- Works with **A (Accumulator)** and **B Register**

Accumulator (A Register)

- Most important 8-bit register
- Used in **all arithmetic/logic** operations
- **Bit-addressable** and part of SFRs

B Register

- 8-bit, used **only** in MUL AB and DIV AB instructions
- **Bit-addressable**, temporary storage during math

8051 Components: PC,DPTR

Program Counter (PC)

- 16-bit register that points to the **next instruction in ROM**
- Increments automatically after each instruction
- Range: **0000H to FFFFH** (up to 64KB code space)

Data Pointer (DPTR)

- 16-bit register used for **external memory access**
- Divided into:
  - **DPL (Low byte)**
  - **DPH (High byte)**
- Used in MOVX and MOVC instructions to fetch data from code or external memory

8051 Components: – Versatile Data Interfaces

4 Parallel I/O Ports:

- **P0, P1, P2, P3** – Each 8-bit wide = Total **32 I/O lines**
- Each pin is **bit-addressable** (can be individually set/cleared)

Registers Used:

- Each port has a dedicated **Special Function Register (SFR)**:
  - P0, P1, P2, P3

.

Port	Standard Use	Alternate Function
P0	I/O port	Multiplexed address/data bus (AD0–AD7)
P1	I/O only	— (no alternate function)
P2	I/O port	Higher address lines AD8–AD15
P3	I/O + Control signals	RXD, TXD, INT0/1, T0/1, WR, RD

VOID

8051 Components: Timers and Counters

Purpose:

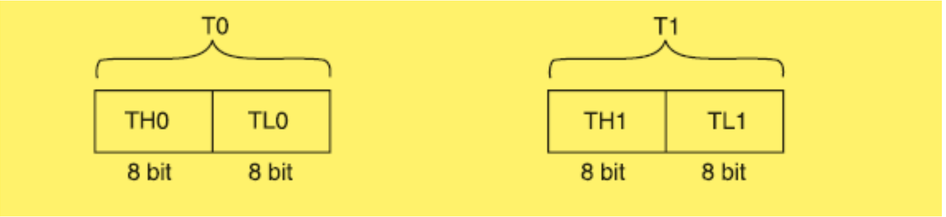
- Generate time delays (Timer Mode)
- Count external events (Counter Mode)

Available Timers:

- **Timer 0 (T0)**

Timer 1 (T1)

- Both are **16-bit** (range: 0000H–FFFFH)



Registers:

- TH0 / TL0, TH1 / TL1 – Hold timer values
- TMOD – Selects mode (Timer/Counter, 8/16-bit)
- TCON – Controls start/stop flags

Modes:

Timer Mode: Internal clock used for time delay

Counter Mode: External pulses counted on T0/T1 pins

Applications:

Delay generation, pulse counting, real-time scheduling

8051 Components: Serial port, Oscillator, Interrupt

VOID

VOID

Serial Port
Enables serial communication between the 8051 and external devices.

- **TXD (P3.1)**: Transmit data
- **RXD (P3.0)**: Receive data
- Controlled by SBUF, SCON
- Supports **4 modes** (synchronous/asynchronous)

Oscillator
Provides the clock signal that drives the microcontroller's operations.

Uses crystal via **XTAL1** & **XTAL2**

Typical frequency: **11.0592 MHz**

Determines **machine cycle speed**

Interrupts
Special signals that interrupt normal program flow to handle urgent tasks.

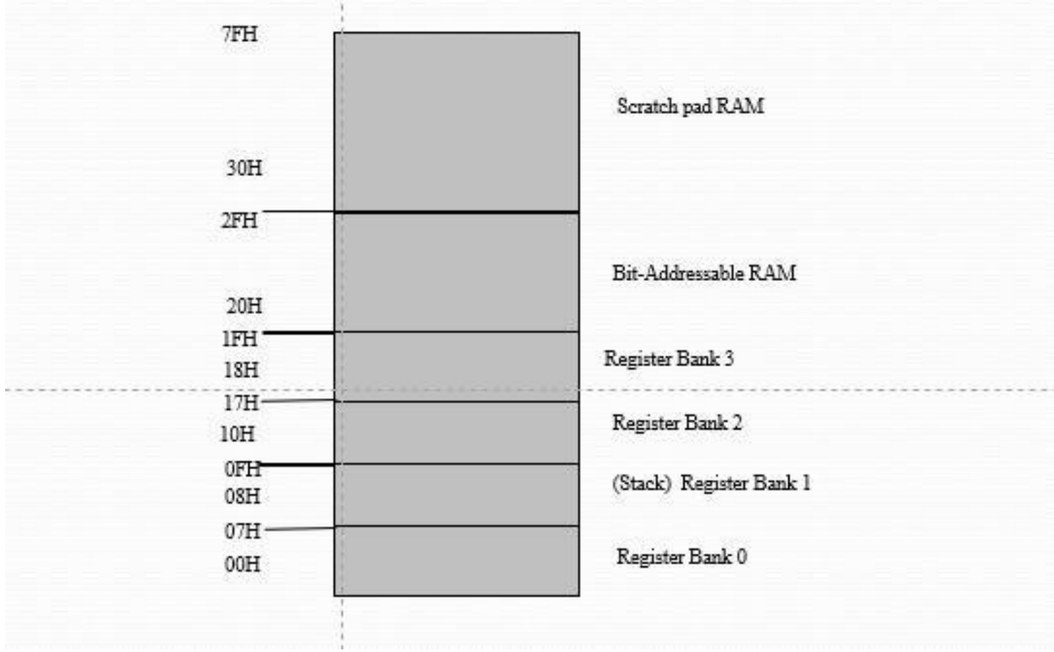
- **5 sources**: INT0, INT1, Timer0, Timer1, Serial
- Controlled by: IE (Enable), IP (Priority)

8051 Components: 128 Byte RAM Overview

Temporary **data storage** during execution

Total RAM:

- **128 bytes (00H–7FH)** user-accessible
- Divided into:
  - **Register banks (R0–R7)**
  - **Bit-addressable area**
  - **Scratchpad RAM**



Divided as:

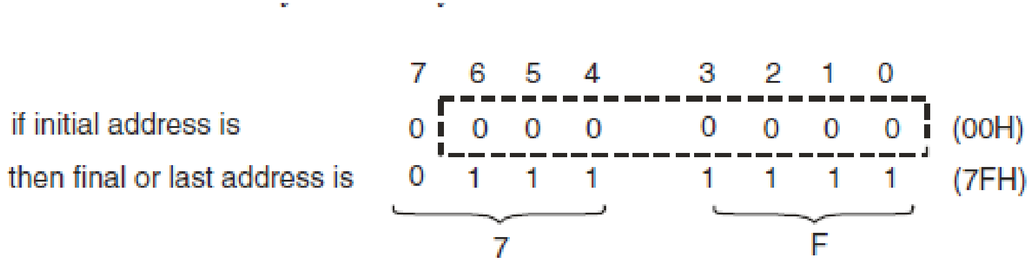
Lower 32 Bytes (00H–1FH): Register banks

Next 16 Bytes (20H–2FH): Bit-addressable RAM

Upper 80 Bytes (30H–7FH): General-purpose scratchpad RAM

Memory Mapping Range:

- Total = **128 locations** = **2^7 = 00H to 7FH**



8051 Components: 128 Byte RAM Overview

RAM Addressing and Structure

Register Banks:

4 banks, each with 8 registers (R0–R7)

Default bank: **Bank 0 (00H–07H)**

Others: Bank 1 (08H–0FH), Bank 2 (10H–17H), Bank 3 (18H–1FH)

Bit-Addressable Area (20H–2FH):

Allows manipulation of individual bits

Useful for flags and control applications

Scratch Pad RAM (30H-7F)

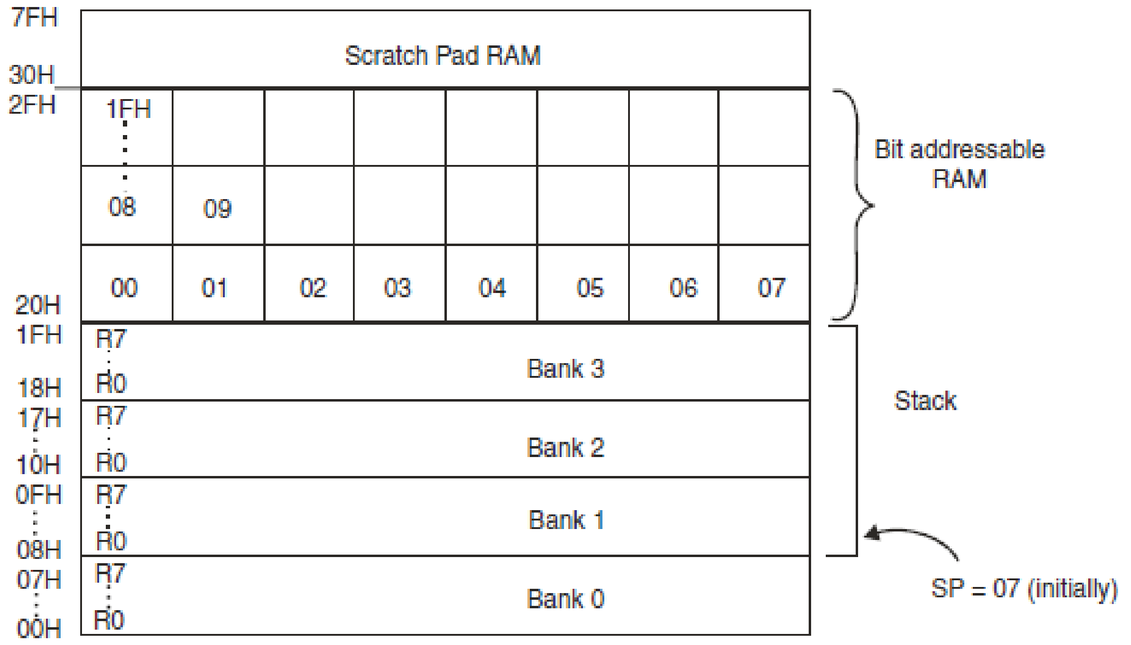


Fig. 1.6 Structure of RAM

Register Bank vs. Stack Conflict:

Problem: Stack (starts at 08H) overlaps with Register Banks 1–3 (08H–1FH) → memory conflict.

Effect: Data corruption and unpredictable behavior during execution.

Solution: Shift stack to safe area using MOV SP, #2FH → uses Scratchpad RAM (30H–7FH).

8051 Components: On-Chip ROM (Program Memory)

8051 - ROM Capacity:

- **4 KB** on-chip **Read-Only Memory**
- Used to store **program code** permanently
- **Non-volatile** – retains data after power off

Purpose:

- Holds fixed program logic/algorithms
- Cannot be modified during execution

Comparison with RAM		
Feature	ROM	RAM
Usage	Program code	Data storage
Volatility	Non-volatile	Volatile
Mutability	Fixed during run	Changeable

ROM Addressing:

Memory Address Range:

- 4 KB = 2^{12} = **0000H to 0FFFH**

Controlled by the **Program Counter (PC)**

PC is **16-bit**, can address up to **64 KB** (0000H–FFFFH)

External ROM Expansion:

Can interface **up to 64 KB** ROM externally

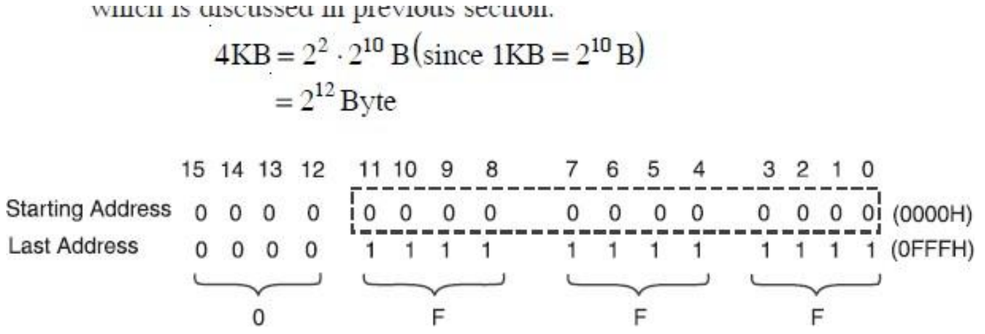
Useful for large applications

- **Example Calculation:**

4 KB = 2^{12} = 0000H to 0FFFH

8 KB = 2^{13} = 0000H to 1FFFH

64 KB = 2^{16} = 0000H to FFFFH



Address range of PC is 0000H to 0FFFH total 4KB locations are available for ROM

from 0000H to 0FFFH.

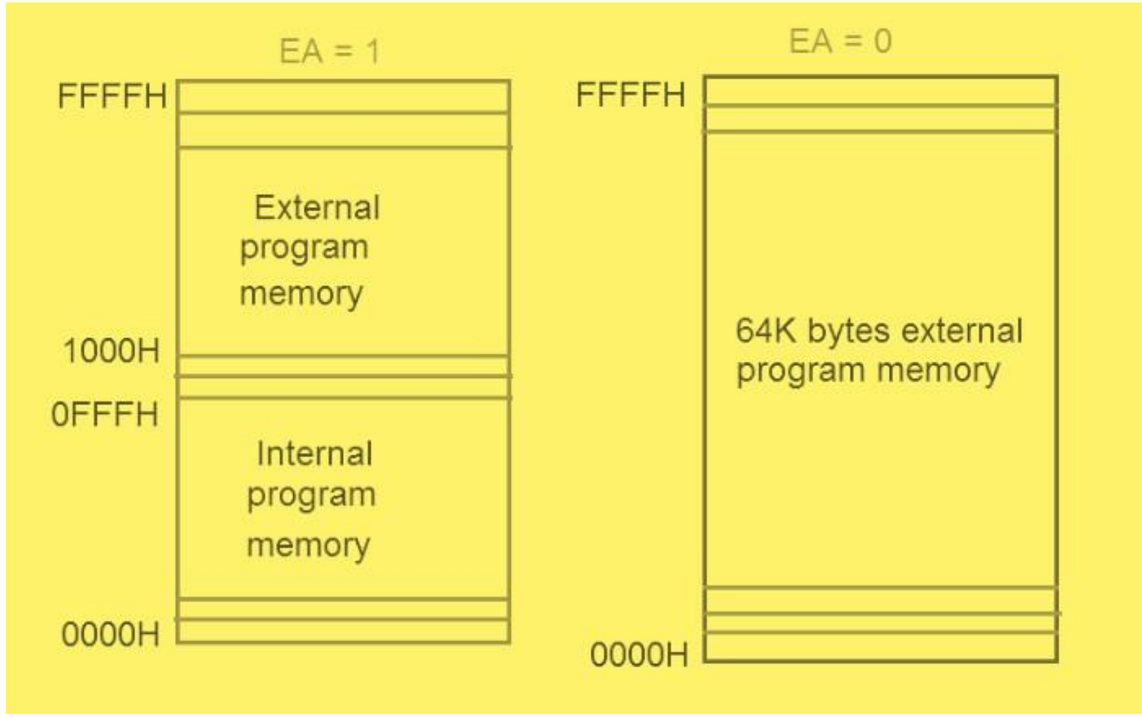
At which we can save the program.

EA Pin – ROM Access Control in 8051 Microcontroller

The **EA (External Access)** pin in the 8051 microcontroller determines whether the microcontroller will use its **internal ROM** or **external program memory** for fetching instructions.

•If **EA is high (connected to Vcc)**, the micr ocontroller **executes code from internal ROM**. This is the standard operating mode when the on-chip ROM is sufficient to hold the entire program.

•If **EA is low (connected to GND)**, the microcontroller **ignores internal ROM** and **fetches all instructions from external memory** This mode is used when larger program memory is required than what is available internally.



8051 Components: Flag Bits & PSW Register

What is PSW?

Program Status Word (PSW) is an 8-bit **Special Function Register**

Reflects **arithmetic status** of operations in the Accumulator (A)

Addressable and **bit-wise accessible**

- **PSW Register**

CY	AC	F0	RS_1	RS_0	OV	--	P

Carry flag	PSW.7	**CY**
Auxiliary carry flag	PSW.6	**AC**
Available to the user for general purpose	PSW.5	**--**
Register Bank selector bit 1	PSW.4	**RS1**
Register Bank selector bit 0	PSW.3	**RS0**
Overflow flag	PSW.2	**OV**
User define bit	PSW.1	**--**
Parity flag Set/Reset odd/even parity	PSW.0	**P**

RS 1	RS 0	Register Bank	Address
0	0	0	00H-07H
0	1	1	08H-0FH
1	0	2	10H-17H
1	1	3	18H-1FH

8051 Components: Special Function Registers (SFRs)

✕ VOID

What are SFRs?

- Registers at addresses **80H to FFH** in internal RAM
- Control and monitor internal features: **I/O ports, timers, serial port, power, etc.**
- Some are **bit-addressable** for precise control

Why SFRs Matter:

- All hardware-level operations go through these registers
- Essential for tasks like **timer setup, serial communication, I/O control**

Special Function Registers

Name	Function	Name	Function
A	Accumulator	SBUF	Serial Port data buffer
B	Arithmetic	SP	Stack Pointer
DPH	Addressing Ext Memory	TMOD	Timer/Counter mode cntrl
DPL	Addressing Ext Memory	TCON	Timer/Counter cntrl
IE	Interrupt enable	TL0	Timer0 lower byte
IP	Interrupt Priority	TH0	Timer0 higher byte
P0	I/O Port Latch	TL1	Timer1 lower byte
P1	I/O Port Latch	TH1	Timer1 higher byte
P2	I/O Port Latch		
P3	I/O Port Latch		
PCON	Power Control		
PSW	Pgm Status Word		
SCON	Serial PortCntrl		

Assembly Language Basics

What is Assembly Language?

- A **low-level language** that uses **mnemonics** instead of binary
- Easier than machine code, but gives **full control** of hardware
- Specific to the processor (8051 in this case)

Instruction Format: [Label:] Mnemonic [Operands] [;Comment]

- **Label**: Optional name for a line (used for jumps/loops)
- **Mnemonic**: The instruction (e.g., MOV, ADD, SJMP)
- **Operand**: Data or registers the instruction works on
- **Comment**: Starts with ;, explains the code

Example:

```
Loop:  MOV R1, #25H   ; Move 25H to R1
       ADD A, R1      ; Add R1 to Accumulator
       SJMP Loop      ; Repeat (jump back to Loop)
```

Addressing Modes in 8051 Microcontroller

Definition: Addressing modes define how the microcontroller accesses data for operations.

1. Immediate Addressing

- Operand is specified in the instruction itself.
- Example: MOV A, #30H

2. Register Addressing

- Data is in registers R0 to R7.
- Example: MOV A, R1

3. Direct Addressing

- Access internal RAM or SFRs by their direct address.
- Example: MOV A, 60H

4. Indirect Addressing

- Register holds the memory address (R0, R1, or DPTR).
- Example: MOV A, @R0

5. Indexed Addressing

- Base address (DPTR or PC) + offset (A). Used in lookup tables.
- Example: MOVC A, @A+DPTR

6. Relative Addressing

- Used with conditional jumps; offset added to PC.
- Example: SJMP LABEL

Arithmetic Instructions in 8051

1. Addition (ADD, ADDC)

- ADD A, #data : Add immediate to A
- ADD A, Rn : Add register to A
- ADD A, direct : Add memory to A
- ADDC is used to include carry in addition.
- Affects CY, AC, OV flags.

2. Multiplication (MUL)

- MUL AB : Multiplies A and B (8-bit each)
- Result: Lower byte in A, higher byte in B

3. Division (DIV)

- DIV AB : Divides A by B (unsigned)
- Quotient in A, Remainder in B

Note: Flags CY and OV are affected in arithmetic operations

Example:Addition (ADD, ADDC)

MOV A, #25H
ADD A, #10H ; A = 35H

Add the contents of A with register Rn with or without ca rry.

iADD A, R5

Add the contents of A with contents of memory with or w ithout carry using direct and indirect addressing

i. ADD A, 51H

ii ADDC A, 75H

iii. ADD A, @R1

Example:Multiplication (MUL)

MOV A, #12H
MOV B, #10H
MUL AB ; A = 20H, B = 01H

Example:Division (DIV)

MOV A, #20H
MOV B, #05H
DIV AB ; A = 04H, B = 00H

VOID

Branch Instructions in 8051

1. Unconditional Branch

- Transfer control without any condition.
- **LJMP address :** Long jump (16-bit address)
- **AJMP address** : Absolute jump (11-bit address within 2KB)
- **SJMP rel** : Short jump (relative jump within −128 to +127)

3. Bit-Level Branch

- Operate on bit values.
- **JB bit, rel** : Jump if bit = 1
- **JNB bit, rel** : Jump if bit = 0
- **JBC bit, rel :** Jump if bit = 1 and clear it

2. Conditional Branch

- Control is transferred only if condition is true.
- **JZ rel** : Jump if A = 0
- **JNZ rel** : Jump if A ≠ 0
- **JC rel** : Jump if carry = 1
- **JNC rel** : Jump if carry = 0
- **CJNE A, #data, rel** : Jump if A ≠ data
- **DJNZ Rn, rel :** Decrement Rn, jump if not zero

Assembler Directives in 8051

1. ORG (Origin)

Sets the starting address for code/data.

Example: ORG 0000H

2. EQU (Equate)

Defines a constant without using memory.

Example: N EQU 10

3. SET

Similar to EQU, but allows redefinition.

Example: COUNT SET 20

4. DB (Define Byte)

Reserves memory and sets 8-bit values.

Example: DATA DB 0FFH

5. END

Marks the end of the source code file.

Example: END

Assembly language programs

Program 1:Problem Statement:

Write a program to add the values of locations 50H and 51H and store the result in locations in 52h and 53H.

```
ORG 0000H              ; Set program counter to 0000H

MOV A, 50H             ; Load contents of memory location 50H into A
ADD A, 51H             ; Add contents of 51H to A
MOV 52H, A             ; Store the LSB of result in 52H

MOV A, #00             ; Load 00H into A to prepare for carry
ADDC A, #00            ; Add carry to A
MOV 53H, A             ; Store MSB (carry) in 53H

END                    ; End of program
```

Program 2:

Write a program Store FFH into memory locations 50H to 58H using indirect addressing mode and a loop.

```
ORG 0000H

MOV A, #0FFH           ; L                tor A
MOV R0, #50H           ; Initialize pointer R0 to 50H
MOV R5, #08H           ; Loop counter for 8 bytes

START:
MOV @R0, A             ; Store FFH at address pointed by R0
INC R0                 ; Move to next address
DJNZ R5, START         ; Decrement counter and repeat if not zero

END
```



Assembly language programs

Program 3

Write a program to Calculate the sum of numbers from 1 to N (N = 15), and store the result at 70H

```
ORG 0000H
N EQU 15              ; Constant N = 15

MOV R0, #00           ; Initialize counter
CLR A                 ; Clear accumulator for sum

AGAIN:
INC R0                ; Increment counter
ADD A, R0             ; Add current count to
accumulator
CJNE R0, #N, AGAIN    ; Continue until R0 == 15

MOV 70H, A            ; Store final result in 70H

END
```

Program 4:

Write a program to Multiply two 8-bit numbers stored at 70H and 71H. Store result in 52H (LSB) and 53H (MSB)




```
ORG 0000H

MOV A, 70H            ; Load multiplicand into A
MOV B, 71H            ; Load multiplier into B
MUL AB                ; Multiply A and B

MOV 52H, A            ; Store LSB of result in 52H
MOV 53H, B            ; Store MSB of result in 53H

END
```

.

Assembly language programs

Program 5:

Write a program to Find the average of five 8-bit numbers (located at 40H to 44H). Store the result in 55H.

```
ORG 0000H

MOV 40H, #05H       ; First number
MOV 41H, #55H       ; Second number
MOV 42H, #06H       ; Third number
MOV 43H, #1AH       ; Fourth number
MOV 44H, #09H       ; Fifth number

MOV R0, #40H        ; Set pointer
MOV R5, #05H        ; Loop counter
MOV B, R5           ; Copy divisor into B
CLR A               ; Clear accumulator

LOOP:
ADD A, @R0          ; Add value to A
INC R0
DJNZ R5, LOOP

DIV AB              ; Divide total by 5
MOV 55H, A          ; Store quotient (average)

END
```

VOID

Interfacing Seven Segment Display with 8051

Introduction to What is a Seven Segment Display?

- Electronic display device for showing decimal numerals and some characters.
- Contains **7 LEDs** arranged in shape of figure '8' + 1 LED for decimal point (dot).

Types:

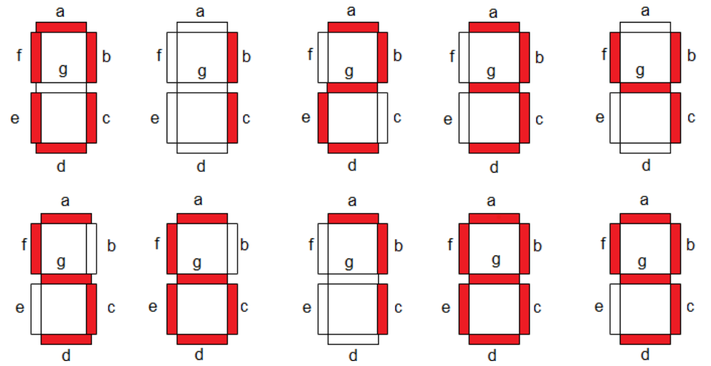
- **Common Cathode (CC):** All cathodes tied together, individual anodes.
- **Common Anode (CA):** All anodes tied together, individual cathodes.

Pin Configuration & Display Patterns

Pin Configuration:

- Pins labeled as: **a, b, c, d, e, f, g** (segments), and **h/dp** (dot point).
- Selection of segments forms numbers or characters.

Digit	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
…							
9	1	1	1	1	0	1	1



VOID

facing Seven Segment Display with 8051

Working Principle & Circuit Description

How It Works:

1.Select LEDs (segments) to **form a digit** by giving high/low signals.

2. **Digit Drive Pattern:** A binary/hex code sent to the microcontroller's port to light up corresponding segments.

3. **Look-Up Table (LUT):** Holds patterns for digits **0-9**.

4. **Program uses MOVC A, @A+PC** to fetch required pattern from LUT.

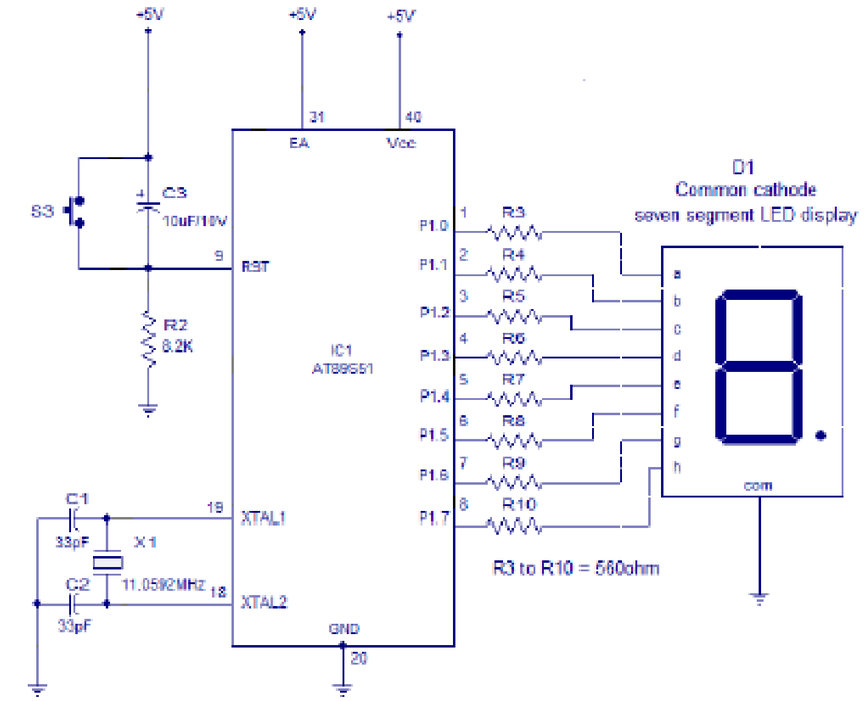
Circuit Diagram

Basic Setup

- **Microcontroller Used:** AT89S51
- **Seven Segment Connected to:** Port 1
- **Components Used:**
  - **R3 - R10:** Current limiting resistors
  - **S3, R2, C3:** Reset + debouncing circuitry
  - **C1, C2, X1:** Oscillator circuit

Purpose:

- Display numbers 0 to 9 repeatedly.



Interfacing 7 segment display to 8051

VOID

Interfacing Seven Segment Display with 8051

Code:

```
ORG 000H          ; Starting address of program memory
START: MOV A,#00001001B   ; Initial value in accumulator (9 decimal)
       MOV B,A            ; Copy A to B for later use
       MOV R0,#0AH        ; Counter R0 initialized to 10 (0AH)

LABEL: MOV A,B            ; Load A with value of B
       INC A              ; Increment A by 1 for next pattern address
       MOV B,A            ; Store back in B
       MOVC A,@A+PC       ; Fetch digit drive pattern from Look-Up Table (LUT)
       MOV P1,A           ; Output pattern to Port 1
       ACALL DELAY        ; Call delay subroutine for visible effect
       DEC R0             ; Decrement counter R0
       MOV A,R0
       JZ START           ; If R0=0, restart count from 0
       SJMP LABEL         ; Otherwise, continue looping

; Look Up Table for digits 0 to 9 (common cathode patterns)
       DB 3FH  ; 0
       DB 06H  ; 1
       DB 5BH  ; 2
       DB 4FH  ; 3
       DB 66H  ; 4
       DB 6DH  ; 5
       DB 7DH  ; 6
       DB 07H  ; 7
       DB 7FH  ; 8
       DB 6FH  ; 9

; Delay Subroutine
DELAY: MOV R4,#05H
WAIT1: MOV R3,#00H
WAIT2: MOV R2,#00H
WAIT3: DJNZ R2,WAIT3
       DJNZ R3,WAIT2
       DJNZ R4,WAIT1
       RET
END
```

Summary & Key Points

Program Highlights:

- Uses **MOVC A,@A+PC** to fetch patterns from **LUT**.
- **Counter (R0)** ensures **0 to 9** display repeatedly.
- LUT simplifies pattern generation.
- Can be extended to **multiple displays** for larger numbers.

Applications:

- **Electronic counters**
- **Digital clocks**
- **Embedded calculators**

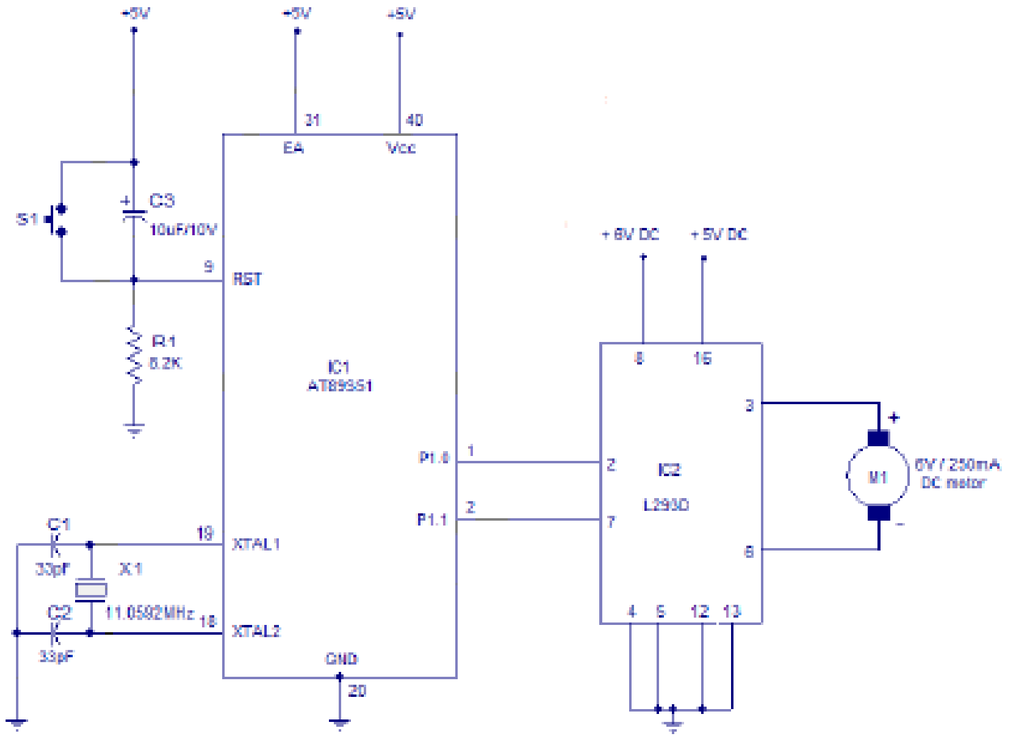
Interfacing DC Motor with 8051 Microcontroller

Introduction & Purpose

- **Goal:** Control the speed and direction of a DC motor using the 8051 microcontroller.
- **Need for Interface:** 8051 cannot supply sufficient current to drive the motor directly, hence a motor driver IC (L293D) is used.

Motor Driver (L293D) & Working Principle

- **L293D Features:**
  - 16-pin IC, controls 2 DC motors.
  - 600mA/channel, voltage range 4.5V–36V.
  - Built-in protection with clamp diodes.
  - Works on H-Bridge concept to control motor direction by changing current flow.



- **Circuit Overview:**
  - AT89S51 (8051) controls L293D via P1.0 & P1.1.
  - Motor connected across output pins 3 & 6 of L293D.
  - Direction is controlled by logic combinations on P1.0 and P1.1.

Interfacing DC Motor with 8051 Microcontroller

Program to Control Motor

```
ORG 00H                ; Program starts here
MAIN: MOV P1,#00000001B    ; Motor rotates clockwise
      ACALL DELAY
      MOV P1,#00000010B    ; Motor rotates anticlockwise
      ACALL DELAY
      SJMP MAIN            ; Repeat cycle forever

DELAY: MOV R4,#0FH
WAIT1: MOV R3,#00H
WAIT2: MOV R2,#00H
WAIT3: DJNZ R2,WAIT3
       DJNZ R3,WAIT2
       DJNZ R4,WAIT1
       RET
END
```

Logic:

- **P1.0 = 1, P1.1 = 0 → Clockwise**
- **P1.0 = 0, P1.1 = 1 → Anticlockwise**

Summary

- **Motor Direction:** Controlled by **P1.0 & P1.1** logic.
- **Current Limit:** ≤ **600mA** per channel.
- **Applications:** Robotics, automation, motorized embedded systems.
- **Caution:** Ensure motor voltage and current are within L293D’s specifications.

CSE_3523

Segment-7 (Part-2)
Embedded system

Microprocessor, microcontroller and embedded system

Course code: CSE-3523
Credit Hour: 3

Md. Mujibur Rahman Maruf

Lecturer,

Dept. of CSE,IIUC

Mujiburmaruf.cuet17@gmail.com

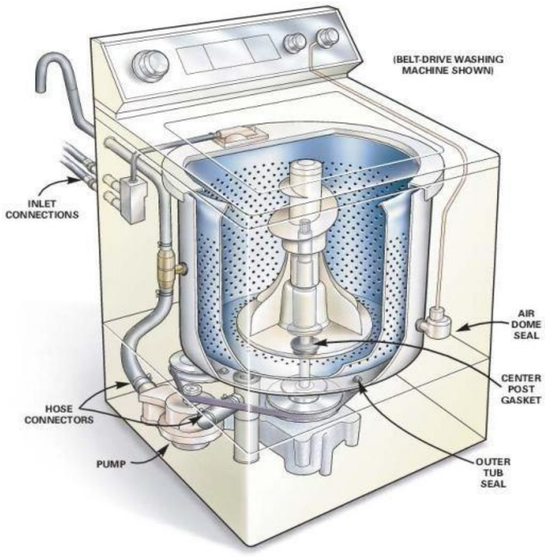
International Islamic University Chittagong(IIUC)

SYSTEM

What is a System?

•A **system** is something that **organizes or performs tasks** according to a **fixed plan, program, or set of rules**.

•It includes **multiple parts** that work **together in coordination** to complete a function.



Example of a System: Washing Machine

A washing machine is an **automatic system** that cleans clothes by performing tasks in a **predefined sequence** using various components

Embedded System

- An **Embedded System** is a **computer system designed to perform a specific, dedicated function**, often as part of a larger system. It is built with a processor, memory, input/output units, and software (firmware)
- **Embedded** refers to a system that is **placed inside** another device or system to perform a **specific task or function**.

Example:

In a **washing machine**, there is a small computer (embedded system) **inside** that controls the washing process.

That computer is **embedded** within the machine — it **is not visible** but **essential** for operation.

Types of Embedded Systems

VOID

Embedded systems are **categorized based on complexity and functionality**. Understanding these classifications helps in choosing the right development tools and architectures:

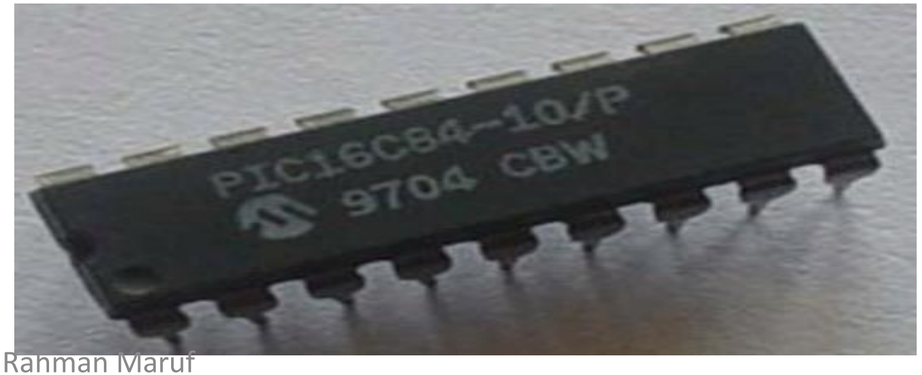
1.Small-Scale Embedded Systems:

Designed using 8-bit or 16-bit microcontrollers, these systems are **simple**, **cost-effective**, and often **battery-powered**. Software is written in C or assembly and must fit within limited memory and power constraints

Example:

Digital watches

Simple calculators



.

Types of Embedded Systems

✕ VOID

2.Medium-Scale Embedded Systems:

These systems use **16 or 32-bit microcontrollers**, **DSPs**, or **RISC processors** and involve moderate hardware and software complexity. They often use **RTOS**, for development.

Example:

Home security systems Industrial automation controllers



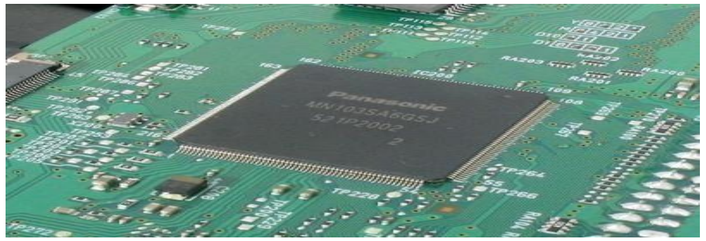
3.Sophisticated Embedded Systems:

Used in **high-performance, real-time applications**, these systems have **large memory**, **powerful processors**, and require **hardware-software co-design**

Example: Self-driving car systems

Medical imaging devices (MRI, CT)

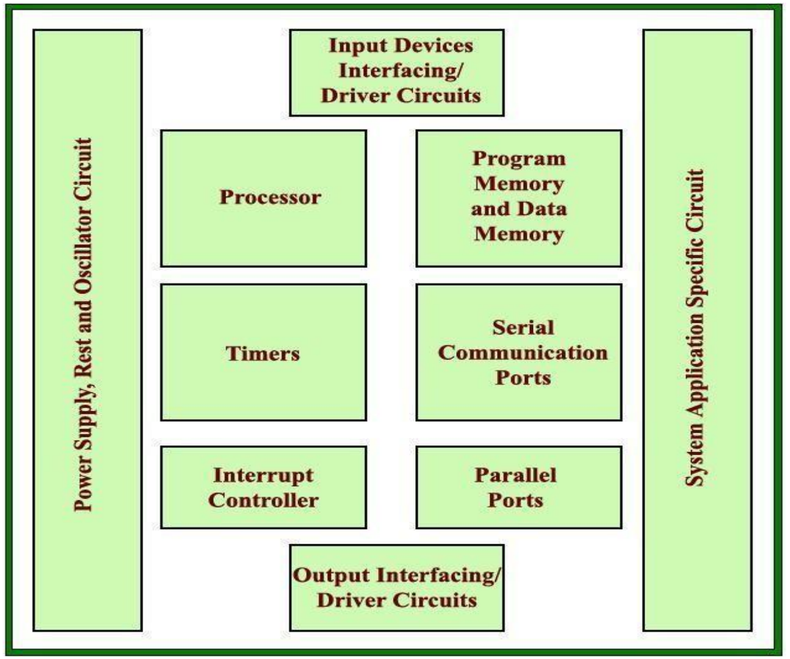
Satellite control systems



Elements of Embedded system

Hardware

The physical part of the embedded system that includes the processor, memory, input/output interfaces, sensors, and other electronic components.



Example: In a **digital thermometer**, the hardware consists of a temperature sensor (to sense temperature), a microcontroller (to process data), display (to show temperature), and power source (battery).

Elements of Embedded system

2.Software

The set of instructions (firmware) that runs on the hardware to perform specific tasks. It includes the embedded application code and device drivers.

Embedded application: The program controlling the embedded system's operation.

Device drivers: Interface between the software and hardware peripherals.

Example:

In the same **digital thermometer**, software reads the sensor data, converts it into temperature values, controls the display output, and manages user input (like switching between °C and °F).

Elements of Embedded system

3.RTOS (Real-Time Operating System)

A specialized operating system designed to run embedded applications that require real-time task scheduling and predictable response times.

Task Scheduling: Manages multiple tasks simultaneously.

Resource management: Efficient use of CPU, memory, and peripherals.

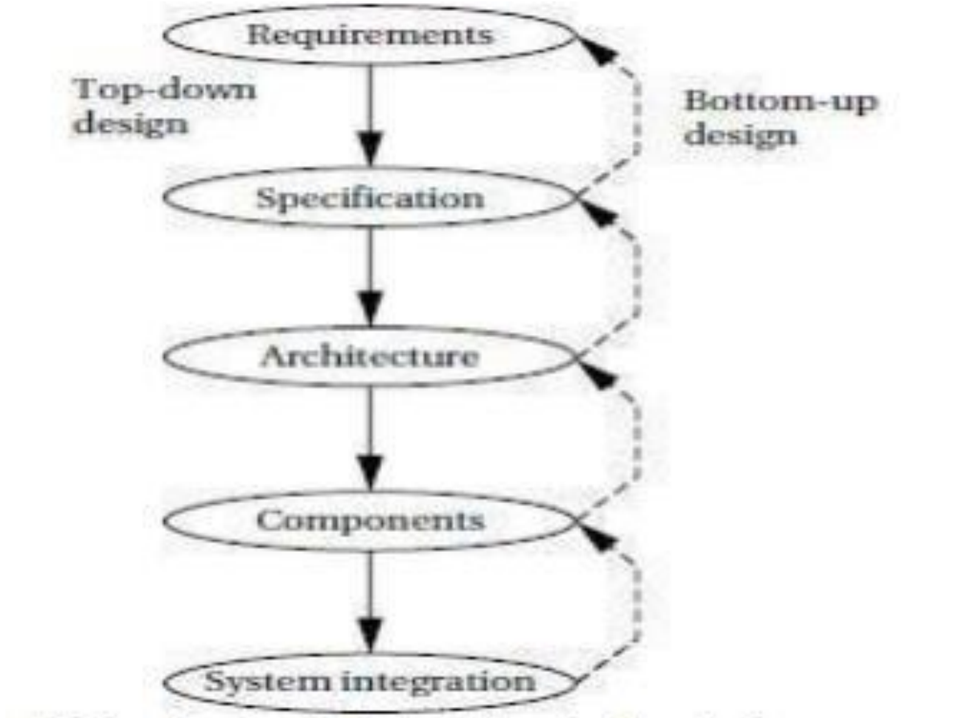
Interrupt handling: Responds quickly to events.

Example:

In a **car's airbag system**, an RTOS ensures the sensor data is processed immediately, and the airbag deploys without delay. It manages multiple real-time tasks like sensor monitoring, system diagnostics, and communication.

steps of embedded system design

- **Requirement**.
- **Specification**
- **Architecture**
- **Component Selection**.
- **Integration & Testing:**



: Major levels of abstraction in the design process

steps of embedded system design

1.Requirements: Define what the system must do

Example: For a smart thermostat, the system should control home temperature automatically, connect to Wi-Fi, display temperature, and consume low power.

2.Specification: Detail inputs, outputs, and performance criteria.

For the smart thermostat:

- Input: Temperature sensor readings.
- Output: Control signal to HVAC system, LCD display updates.
- Performance: Temperature reading update every 1 second.
- Constraints: Operate between 0°C to 50°C, battery life of 6 months.

steps of embedded system design

3.Architecture Design: Plan the hardware and software structure..

Example:

Hardware: Microcontroller, temperature sensor, Wi-Fi module, LCD display,

Software: Sensor reading module, control algorithm, user interface, communication protocol.

4.Component Selection:

Choose specific hardware and software components based on the architecture.

Example:

Microcontroller: ARM Cortex-M4 with built-in ADC.

Temperature sensor: Digital temperature sensor (e.g., DS18B20).

Wi-Fi module: ESP8266. Display: 16x2 LCD.

Software: FreeRTOS for task management, custom firmware.

steps of embedded system design

5. Integration and Testing

Combine components and verify system functionality

Example:

Assemble the PCB with selected components.

Load firmware and test temperature sensing, Wi-Fi connectivity,

display updates, and HVAC control.

Debug and optimize for performance and power consumption.

VOID

Applications of Embedded Systems

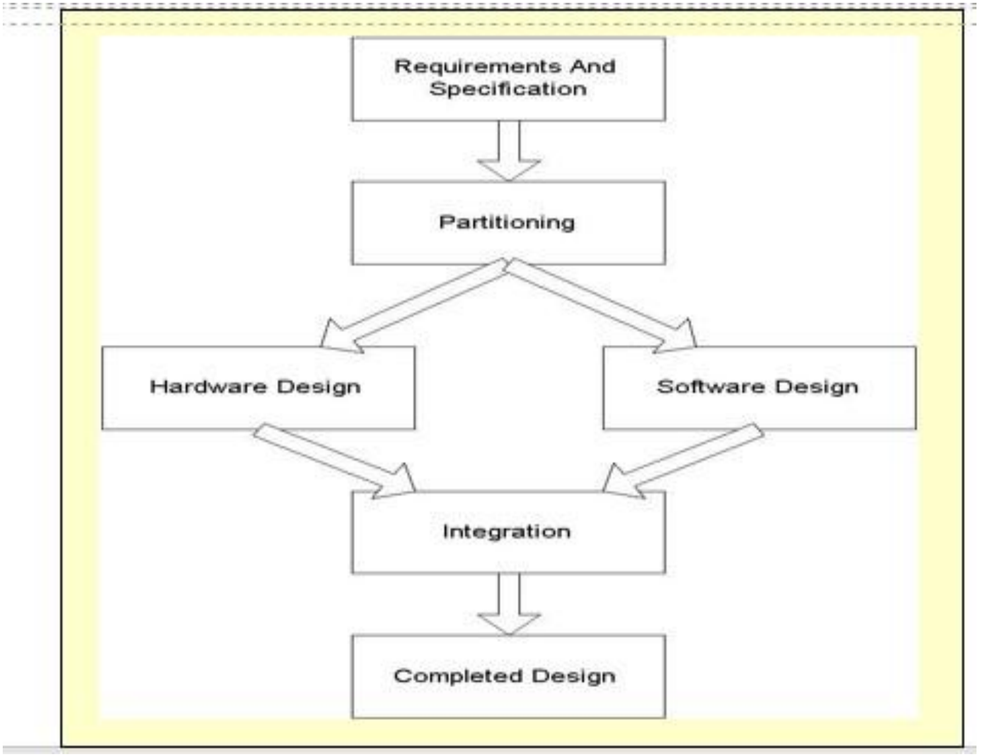
Embedded systems are at the heart of **modern technology**, embedded into a wide range of products. Their applications span across:

- **Telecommunication:** Routers, modems, mobile base stations.
- **Automotive:** Engine control, ABS, GPS systems.
- **Consumer Electronics:** TVs, MP3 players, digital cameras.
- **Industrial Automation:** PLCs, robotic arms, sensors.
- **Medical Devices:** Heart rate monitors, infusion pumps.

These systems are designed to **interact with the physical world**, sense inputs, and control outputs.

Traditional Embedded System Design

In traditional embedded system design, the process follows a **linear and rigid flow**. It begins with **requirements and specifications**, followed by an early **hardware/software (HW/SW) partitioning**. Once partitioning is done, the design proceeds in **two separate tracks** — hardware design and software design — usually handled by different teams. These components are then integrated in the final stage.



Key Characteristics:

- **Early HW/SW Partitioning:**

Partitioning is done at the beginning based on fixed assumptions. If any mistake is made in deciding what part goes to hardware or software, it becomes very hard to correct later in the process.

Traditional Embedded System Design

- **Independent Development:**

Hardware and software teams often work separately, with little interaction or feedback. This lack of synchronization can result in **compatibility issues** during integration.

- **Delayed Integration:**

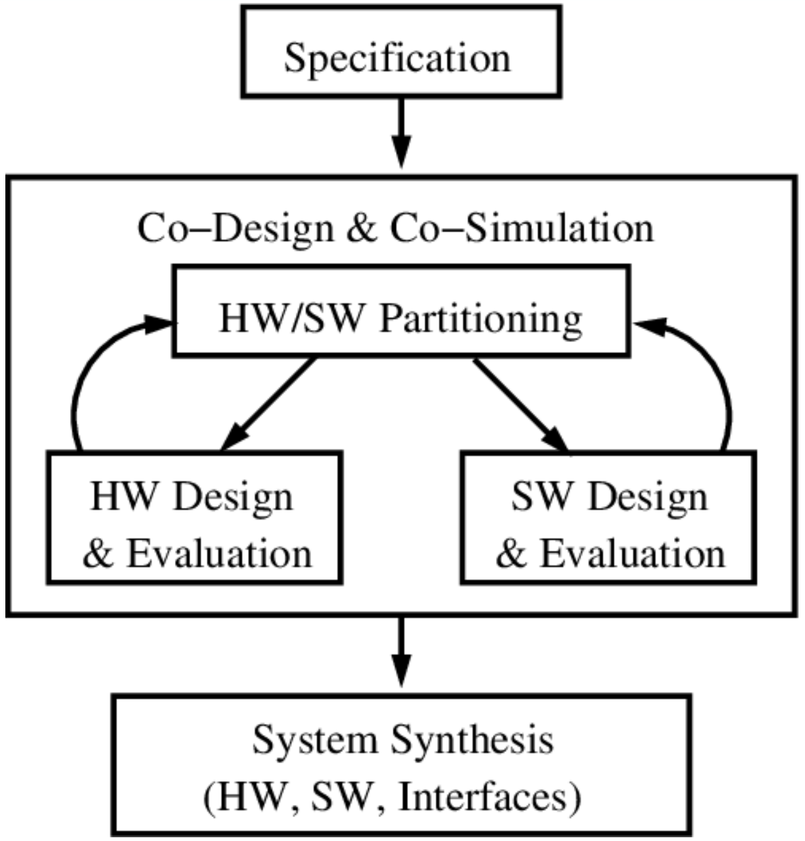
Integration is performed at the end, so **problems are discovered late**, and correcting them is costly and time-consuming.

- **Limited Flexibility:**

Once hardware is designed, it cannot be easily changed to fix a functional or performance issue without major redesign.

Hardware-Software Co-Design

Hardware-Software Co-Design takes a **collaborative and iterative** approach. The process begins with **system-level specification and analysis**, followed by **joint HW/SW architecture planning and partitioning**. Rather than being fixed early, the partitioning evolves throughout development based on functional, performance, and power requirements.



Hardware-Software Co-Design methodology

Hardware-Software Co-Design

Co-design of (embedded) computer systems encompassing the following parts

.

1. **Co-Specification**
2. **HW-SW Partitioning**
3. **HW-SW Co-Synthesis**
4. **HW-SW Co-Simulation**
5. **Co-Verification**

Hardware-Software Co-Design

1.Co specification:

Define the system's intended functionality at a high level using functional models (e.g., task graph).Focuses on **what** the system does, not how it's built.

Example: For home security system

Define key functions: detect motion/door opening, trigger alarm, send alert.

Model tasks like sensing, processing, and actuation in a task graph.

2.HW-SW Partitioning

Analyze the task graph and decide **which tasks to implement in hardware** and which in **software**.

Decisions based on **performance, cost, power, and flexibility**.

Example: Assign sensors (PIR, door contacts) and alarm hardware to hardware.

Assign intrusion logic, alert generation, and cloud communication to software..

Hardware-Software Co-Design

3. Hardware-Software Co-Synthesis

4 principal phase of co synthesis

A.Partitioning: Break the system into smaller parts or tasks.

Example: In a smart home system, Motion sensing → Intrusion logic → Activate alarm → Send notification.

B.Scheduling : Decide the order and timing for each task.

*Example:*Sense motion every second → Process immediately → Trigger alarm & send alert within 1 second.

Hardware-Software Co-Design

Hardware-Software Co-Synthesis

C.Allocation

Assign tasks to specific processing units (hardware or software).

Example: Motion sensing & alarm on hardware; decision logic & alerting on software

D.Mapping

Choose the exact hardware or software components for each assigned task.

- PIR sensor for motion (HW)
- ESP32 MCU running C code for processing (SW)
- Buzzer via GPIO for alarm (HW)
- Wi-Fi module for mobile alert (SW)

Hardware-Software Co-Design

4. HW-SW Co-Simulation

Simulate the combined HW-SW system **before physical prototyping**.

•Validates timing, control logic, and communication between modules.

•**Example:** Simulate sensor activation, alarm response, and alert flow.

Ensure real-time response and correct logic execution under different scenarios.

5. Co-Verification

Ensure that the **final integrated system meets specifications** using testing or formal verification methods.

Verifies functional correctness, timing, and interface reliability.

Example: Test for intruder detection, false alarm handling, and message delivery.

Verify all features work as specified before physical deployment.

Computational Model

What is a Computational Model?

•A computational model is a method to describe how a system behaves and processes information.

•Used to plan and test a system before building it in hardware or software.

•Acts like a **blueprint** for the system's operations

•In embedded system co-design, the computational model helps define whether a functionality should be implemented in **hardware**, **software**, or **both**, based on performance, cost, and other trade-offs.

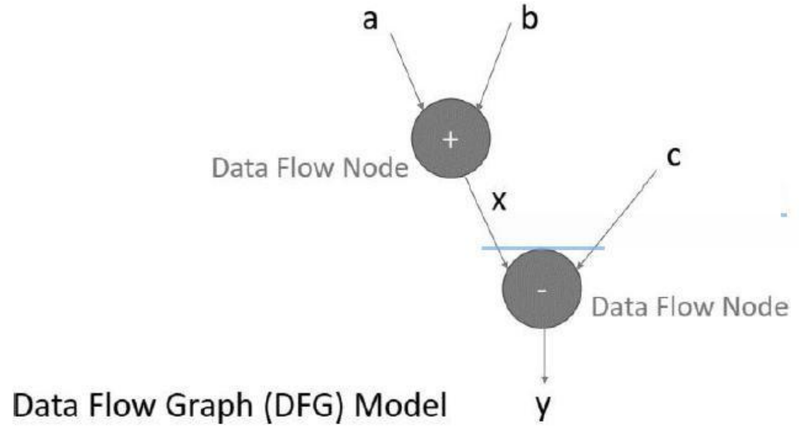
Computational Model

Data Flow Graph (DFG) Model

- **Data-driven execution**: actions occur when data is ready.
- Used in systems where **data processing is key**, like DSP.
- **Example**:

If a function computes x = a + b and y = x - c

- **Graph**:
  - Circles: represent operations (+, -)
  - Arrows: show flow from inputs a, b, c to outputs x, y
- **Use case**: Audio processing, image filtering



Data Flow Graph (DFG) Model

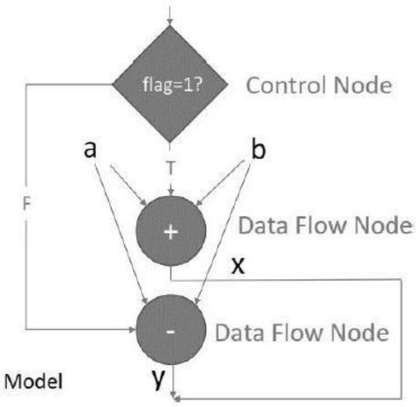
Computational Model

Control Data Flow Graph (CDFG) Model

- Extends DFG by adding **conditional logic** (e.g., if, else).
- Includes **data operations and decision blocks**.
- **Example**:

If flag = 1, then x = a + b; else y = a - b.

- **Graph**:
  - Decision (diamond block) splits flow
  - Different data paths based on condition
- **Use case**: Camera choosing JPEG or PNG format based on settings.



Control Data Flow Graph (CDFG) Model

Computational Model

Finit State Machine Model (FSM)

A **Finite State Machine (FSM)** is a **computational model** used to describe the behavior of systems that go through a sequence of **states**, based on **inputs or events**.

It consists of:

- **State**: A situation or step the system is in (example: "Waiting", "Running").
- **Event/Input**: Something that happens to cause a change (example: "Insert coin").
- **Transition**: The change from one state to another.
- **Action**: What the system does when it changes state (example: "Start alarm").

FSM Model – Example 1

design of an embedded system for driver/passenger 'Seat Belt Warning' in an automotive using the FSM model for following requirements

1. When the vehicle ignition is turned on and the seat belt is not fastened within 10 seconds of ignition ON, the system generates an alarm signal for 5 seconds.
2. The Alarm is turned off when the alarm time (5 seconds) expires or if the driver/passenger fastens the belt or if the ignition switch is turned off, whichever happens first.

Solutions:

Here the states are

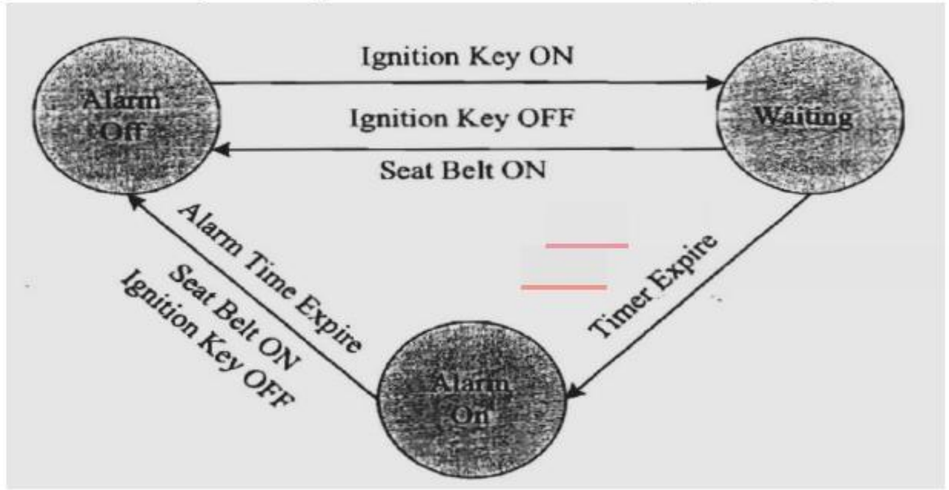
- 'Alarm Off'
- 'Waiting'
- 'Alarm On'

The events are

- 'Ignition Key ON'
- 'Ignition Key OFF'
- 'Timer Expire'
- 'Alarm Time Expire'
- 'Seat Belt ON'

FSM Model – Example 1

Using the FSM, the system requirements can be modeled as given in figure.



State Diagram

- The 'Ignition Key ON' event triggers the 10 second timer and transitions the state to 'Waiting'.
- If a Seat Belt ON' or 'Ignition Key OFF' event occurs during the wait state, the state transitions into 'Alarm Off'.
- When the wait timer expires in the waiting state, the event 'Timer Expire' is generated and it transitions the state to 'Alarm On' from the 'Waiting' state.
- The 'Alarm On' state continues until a 'Seat Belt ON' or 'Ignition Key OFF' event or 'Alarm Time Expire' event, whichever occurs first.
- The occurrence of any of these events transitions the state to 'Alarm Off'.

FSM Model – Example 2

Design an automatic tea/coffee vending machine based on FSM model for the following requirement.

- The tea/coffee vending is initiated by user inserting a 5 taka coin.
- After inserting the coin, the user can either select 'Coffee' or 'Tea' or press 'Cancel' to cancel the order and take back the coin.

Solution

The FSM Model contains four states namely,

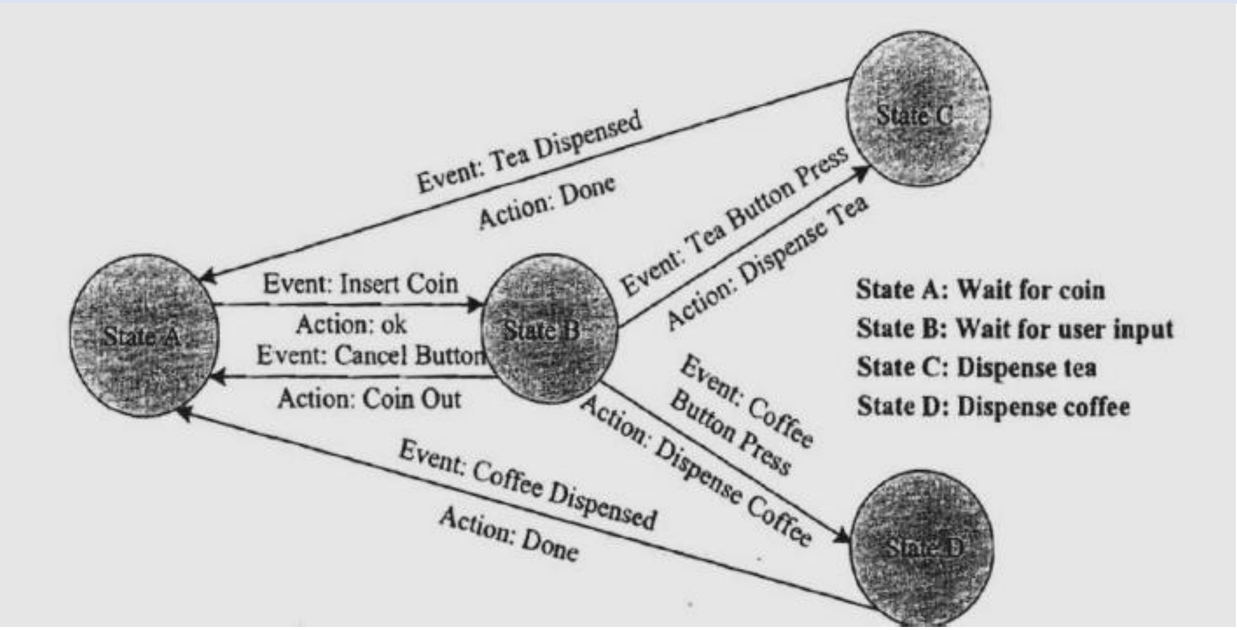
1. 'Wait for coin’
2. Wait for User Input’
3. Dispense Tea’
4. Dispense Coffee'

4 Events:

- Insert Coin
- Press Tea
- Press Coffee
- Press Cancel

FSM Model – Example 2

The event **'Insert Coin'** (5 taka coin insertion), transitions the state to **'Wait for User Input'.**



□ The system stays in this state until a user input is received from the buttons **'Cancel', 'Tea' or 'Coffee'**

□ If the event triggered in **'Wait State**' is **'Cancel'** button press, the coin is pushed out and the state transitions to **'Wait for Coin'.**

□ If the event received in the **'Wait State**' is either '**Tea**' button press, or '**Coffee**' button press, the state changes to '**Dispense Tea**' and '**Dispense Coffee**' respectively.

□ Once the coffee/tea vending is over, the respective states transition back to the **'Wait for Coin' state.**

FSM Model – Example 3

VOID

Design a Finite State Machine (FSM) model for a simple elevator (lift) controller in a multi-story building.

VOID

Requirements:

The elevator responds to floor requests and moves accordingly. When a request is received, the elevator moves either **up** or **down** to reach the requested floor. Once it reaches the requested floor, the **door opens** and stays open for at least **10 seconds**. The door remains open until a new request is received. The elevator must never have its door open while moving. The direction is only changed if there are no Pending requests in the current direction of motion

FSM Model – Example 3

VOID

Solution:

States:

The elevator can be in one of the following four states:

- Idle –
- GoingUp
- GoingDown
- DoorOpen

Events:

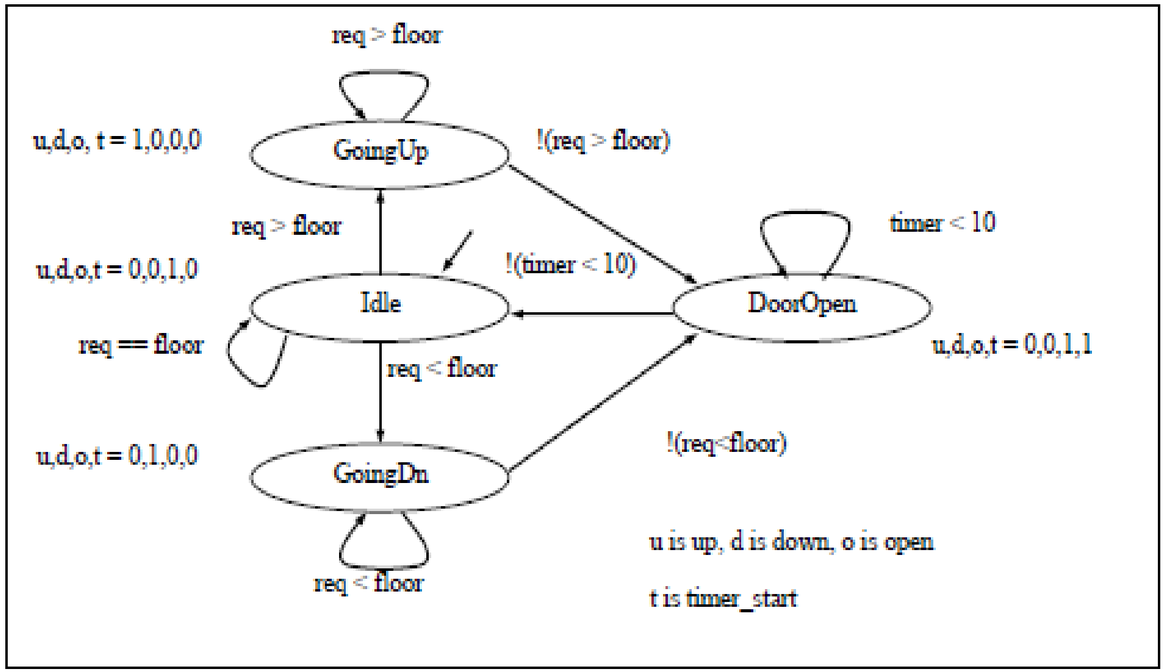
Several events control the transitions between these states:

- req > floor
- req < floor
- req == floor
- !(req > floor)
- !(req < floor)
- timer < 10
- !(timer < 10)

FSM Model – Example 3

VOID

- Initially, the elevator is in the **Idle** state. If a request is made for a floor higher than the current floor (req > floor), the elevator transitions to the **GoingUp** state and begins moving upward.
- If a request is made for a floor lower than the current floor (req < floor), the elevator transitions to the **GoingDown** State and moves downward.
- If the current floor is the requested floor (req == floor), it immediately transitions to the **DoorOpen** state to allow entry or exit.



State Diagram

FSM Model – Example 3

VOID

- While in the **GoingUp** state, the elevator keeps moving upward as long as there is a higher request (req > floor). Once there are no more higher requests (!(req > floor)), it transitions to the **DoorOpen** state.
- Similarly, in the **GoingDown** state, the elevator continues moving down while there is a lower request (req < floor). Once there are no more lower requests (!(req < floor)), it moves to the **DoorOpen** state.
- In the **DoorOpen** state, the door remains open for a minimum of 10 seconds. As long as the timer is less than 10 seconds (timer < 10), the elevator stays in this state. Once the timer reaches 10 seconds or more (!(timer < 10)), the elevator returns to the **Idle** state, waiting for the next request.

Introduction to IoT: Core Concepts and Technology Fundamentals

In a world increasingly connected, where countless devices are linked to the internet, and all collect and share data, understanding the **core concepts**, **underlying technologies**, and wide-ranging applications of the Internet of Things (IoT) is essential. This document will equip you with the foundational knowledge to understand and actively contribute to this rapidly evolving and transformative field, preparing you for deeper dives into its infrastructure, protocols, data management, and security.

What is the Internet of Things (IoT)?

> The Internet of Things (IoT) is a network of physical objects—"things"—embedded with sensors, software, and other technologies for the purpose of connecting and exchanging data with other devices and systems over the internet. These devices range from ordinary household objects to sophisticated industrial tools.

Key Characteristics of IoT

Connectivity

IoT devices are interconnected, enabling communication between themselves and with cloud platforms. This seamless connection is the backbone of any IoT ecosystem.

Data Exchange

Devices collect and exchange vast amounts of data from their environment. This data, once processed, provides valuable insights for decision-making and automation.

Interoperability

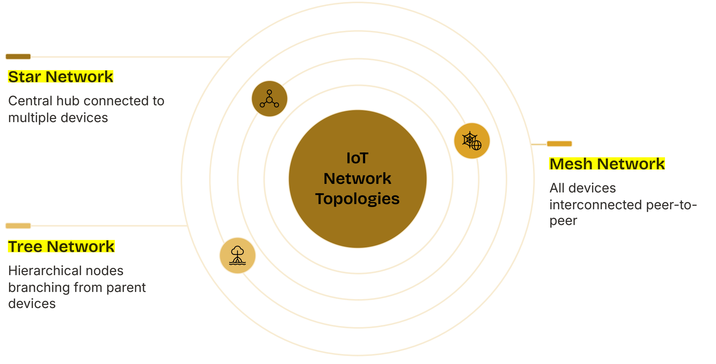
The ability of different IoT devices and systems to work together and exchange information seamlessly, regardless of manufacturer or protocol.

Automation

Based on the data collected and exchanged, IoT systems can automate processes and trigger actions without human intervention, leading to increased efficiency and responsiveness.

IoT Network Topologies

IoT devices connect using various network patterns to ensure reliable communication.



Fundamental Components of IoT

Devices (Things)

Physical objects equipped with sensors to collect data and/or actuators to perform actions in the physical world.

Connectivity

The communication layer that enables devices to connect to each other and to the cloud, utilizing various protocols and network types.

Data Processing

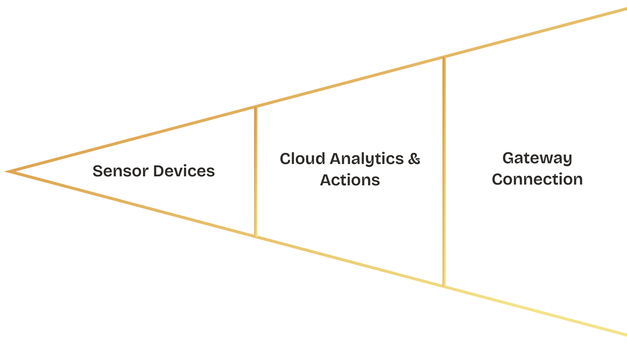
The infrastructure (cloud platforms, edge computing) for collecting, storing, analyzing, and managing the vast amounts of data generated by IoT devices.

User Interface

Applications, dashboards, and other interfaces that allow users to interact with IoT systems, monitor data, and control devices.

IoT Data Flow

Understanding how information moves through an IoT system is crucial.



Core Principles of IoT

1 Scalability

IoT systems must be designed to accommodate a growing number of devices and data volumes without compromising performance.

2 Security

Protecting devices, data, and communication channels from unauthorized access, cyber threats, and vulnerabilities is paramount.

3 Privacy

Ensuring the responsible collection, use, and protection of personal and sensitive data generated by IoT devices.

4 Intelligence

Leveraging data analytics, machine learning, and artificial intelligence to derive actionable insights and enable smart decision-making.

5 Integration

The capability of IoT solutions to seamlessly integrate with existing business processes and IT infrastructure.

IoT Architecture: Core Components & Technology Fundamentals

At its core, IoT relies on several key architectural components working in concert. These components facilitate everything from data collection in the physical world to processing, managing, and presenting insights, forming the "IoT basics and technology fundamentals" that enable connected systems. Each plays a vital role in the seamless operation of IoT ecosystems and represents a layer within the "IoT architecture state-of-the-art".

Sensing & Actuation Layer

This foundational layer comprises physical devices like sensors (collecting environmental data) and actuators (performing physical actions). It's the primary interface with the real world, embodying core "IoT basics and technology fundamentals".

Network & Communication Protocols

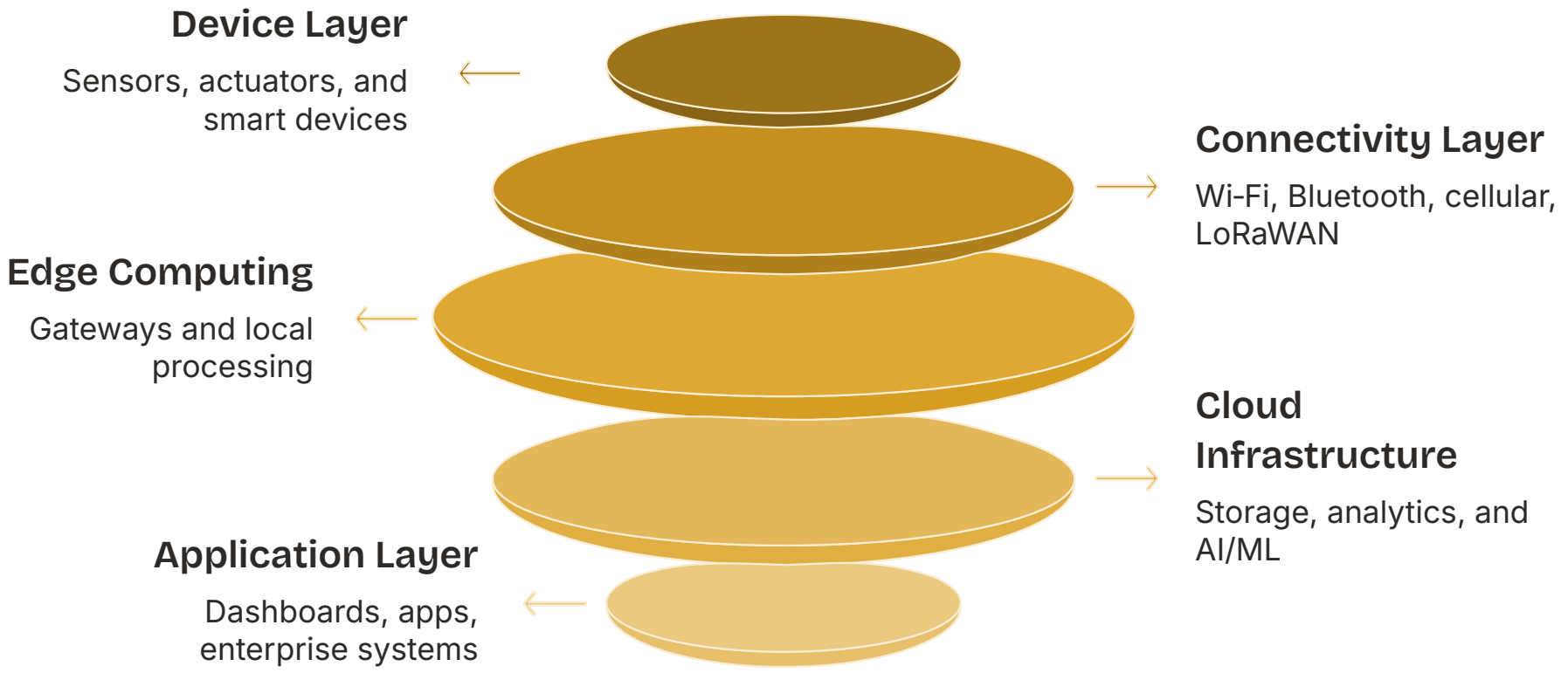
Focusing on "Infrastructure and network architecture" and "Communication protocols and processes", this layer ensures secure and efficient data transmission using technologies like Wi-Fi, Cellular, Bluetooth, and MQTT for device-to-device and device-to-cloud communication.

Data Processing & Management Systems

This layer handles "Data management systems" and "Big Data value creation". It processes raw sensor data, often at the edge or in the cloud, transforming it into actionable insights using analytics, machine learning, and robust storage solutions.

Application & User Interface

This top layer defines "IoT applications across domains". It provides the interfaces and services through which users interact with IoT systems, visualize data, and control devices, enabling practical solutions in smart homes, industries, and cities.



Understanding these foundational elements is crucial for comprehending the vast potential and complexity of the IoT landscape. We will now delve deeper into each of these areas, exploring their functions and significance within a comprehensive IoT ecosystem.

Made with GAMMA

Sensors & Actuators: Fundamental IoT Technology

Sensors and actuators represent the core technological interface between the physical and digital worlds within an IoT system, establishing the foundational elements for data collection and interaction. They enable devices to perceive their surroundings and respond autonomously or based on remote commands, crucial for any IoT application.

Sensors: The IoT's Eyes

Sensors are fundamental devices that detect and measure physical phenomena, converting them into electrical signals or digital data. These signals can then be processed and analyzed as part of the IoT ecosystem. Examples include:

- Temperature sensors (e.g., in thermostats, HVAC systems)
- Humidity sensors (e.g., for agriculture, climate control)
- Motion/Presence sensors (e.g., security systems, smart lighting)
- Light sensors (e.g., automated blinds, streetlights)
- Pressure sensors (e.g., industrial processes, tire monitoring)
- Acoustic sensors (e.g., smart assistants, predictive maintenance)
- GPS modules (e.g., asset tracking, navigation)



Actuators: The IoT's Hands

Actuators are essential components that receive electrical signals and convert them into physical actions, allowing IoT devices to interact with and control their environment. They are critical for implementing the physical responses of IoT systems. Examples include:

- Motors (e.g., robotics, automated valves)
- Relays (e.g., turning lights or machinery on/off)
- Heaters/Coolers (e.g., climate control, industrial ovens)
- Valves (e.g., smart irrigation, fluid control)
- Speakers (e.g., audible alerts, voice feedback)
- Smart Locks (e.g., access control, home security)



Together, sensors and actuators form a closed loop, crucial for the functioning of IoT systems. They enable continuous monitoring of conditions and real-time reactions to changes, underpinning automation and control across countless applications within the IoT landscape.

IoT Infrastructure, Network Architecture & Communication Protocols

Understanding the foundational infrastructure, network architecture, and diverse communication protocols is critical to IoT development. These elements work in concert to enable seamless data exchange between devices, gateways, and cloud platforms, addressing the core technological requirements for any IoT solution.

IoT Infrastructure

IoT infrastructure refers to the physical and virtual components that support the entire IoT ecosystem, from device connectivity to data management and application delivery. It forms the backbone for data flow and processing.

Key Infrastructure Components

- **Edge Devices:** The "things" in IoT, including sensors and actuators that collect data from or act upon the physical world.
- **Gateways:** Bridge the gap between edge devices and the cloud, aggregating data, performing local processing, and translating protocols.
- **Cloud Platforms:** Provide robust services for data storage, advanced analytics, machine learning, and application hosting at scale.
- **Data Centers:** Centralized facilities for hosting cloud platforms, offering high computational power and storage capacity.

Infrastructure Layers

01

Device Layer

Sensors and actuators interacting directly with the physical environment.

02

Connectivity Layer

Enables data transmission from devices to the cloud using various protocols.

03

Data Processing Layer

Manages data storage, analysis, and transformation for insights.

04

Application Layer

Provides user-facing applications and services, delivering value from processed data.

Network Architecture

IoT network architecture defines how devices are connected and how data flows through the system, influencing performance, reliability, and security. Selecting the right topology is crucial for an efficient IoT deployment.

Common Network Topologies

Star Topology

All devices connect to a central hub (e.g., gateway or router). Simple to manage but has a single point of failure.

Mesh Topology

Devices connect directly to multiple other devices, creating redundant paths. Highly reliable and scalable but more complex.

Tree Topology

A hierarchical structure with a root node and multiple branches. Combines aspects of star and bus topologies.

Hybrid Topologies

Combines two or more basic topologies to meet specific system requirements, offering flexibility.

Network Design Considerations

- **Scalability:** Ability to efficiently add more devices and handle increasing data volumes.
- **Reliability:** Ensuring continuous operation and data integrity, even with device or connection failures.
- **Security:** Protecting devices, data, and network from unauthorized access and cyber threats.
- **Latency:** Minimizing delays in data transmission and processing, critical for real-time applications.
- **Power Consumption:** Optimizing energy use for battery-powered devices to extend their lifespan.

Communication Protocols

Communication protocols dictate the rules for data exchange within the IoT network, enabling seamless interaction between devices, gateways, and cloud platforms. Protocol selection depends on factors like range, power consumption, and data rates.

Short-Range Communication Protocols:

- **Wi-Fi:** High bandwidth, suitable for streaming and complex data in local areas (e.g., smart homes, offices). Part of the local network infrastructure.
- **Bluetooth/BLE (Low Energy):** Low power, short-range, ideal for wearables, personal devices, and small sensor networks.
- **Zigbee & Z-Wave:** Low power mesh networking protocols, commonly used for reliable smart home automation, forming local area network architecture.
- **NFC (Near Field Communication):** Very short-range, secure, used for payments, access control, and tag reading.

Long-Range (LPWAN) Communication Protocols:

- **LoRaWAN & NB-IoT:** Low power, long-range solutions for wide-area deployments with small data packets (e.g., smart cities, agriculture), contributing to a broader infrastructure.
- **Cellular (2G/3G/4G/5G):** High bandwidth and range, reliable for mission-critical applications and mobile IoT devices. 5G offers ultra-low latency, crucial for next-gen IoT infrastructure.
- **Satellite:** Global coverage, essential for remote asset tracking and areas without terrestrial network infrastructure.

In addition to these, specialized IoT application protocols like MQTT and CoAP are designed for efficient message exchange over constrained networks, optimizing data transmission for low-power devices, which are essential communication processes. Ethernet also plays a crucial role for reliable, high-speed wired connections in industrial settings, forming a vital part of the wired infrastructure.

From M2M to IoT: Evolution and Value Chains

The Internet of Things (IoT) represents a significant evolution from Machine-to-Machine (M2M) communication, fundamentally transforming how data is collected, processed, and utilized to create new value chains. While M2M laid the groundwork for device interaction, IoT expands connectivity, integrates advanced data processing, and enables user-centric applications, fostering a complex, interconnected ecosystem of intelligent services.

Understanding Machine-to-Machine (M2M)

Machine-to-Machine (M2M) communication refers to the direct communication between devices using wired or wireless channels, without human intervention. It primarily focuses on enabling devices to exchange data and perform actions autonomously. M2M emerged in the late 20th and early 21st centuries, driven by the need for remote monitoring, automation, and telemetry in various industrial and commercial sectors.

Historical Context and Early Applications

The concept of M2M began with simple telemetry applications, such as remote monitoring of utility meters and vending machines. Early M2M solutions were often proprietary, utilizing specialized hardware and software to connect machines over cellular or satellite networks. This allowed businesses to gather data from distant assets, optimize operations, and reduce manual labor, marking the initial steps towards a more connected world.

Key Characteristics of M2M

- **Direct Device Communication:** Devices communicate directly with each other or with a central M2M gateway without explicit human interaction.
- **Automated Data Exchange:** Focuses on the automatic collection and transmission of data for monitoring, tracking, and control purposes.
- **Minimal Human Intervention:** Once deployed, M2M systems are designed to operate autonomously, requiring little to no human oversight for routine tasks.
- **Proprietary Nature:** Often relied on proprietary protocols and hardware, leading to siloed solutions and limited interoperability.
- **Specific Applications:** Tailored for narrow, often single-purpose applications within closed environments.

Typical M2M Applications

- **Fleet Management:** Tracking vehicle locations, monitoring engine performance, and optimizing delivery routes.
- **Utility Meters:** Automated reading of electricity, gas, and water meters for billing and consumption analysis (Smart Metering).
- **Industrial Monitoring:** Supervising manufacturing equipment, detecting anomalies, and predicting maintenance needs.
- **Vending Machines:** Monitoring inventory levels and reporting operational status for timely restocking and maintenance.
- **Security Systems:** Remote surveillance, alarm triggering, and access control.

M2M Architecture Components

A typical M2M architecture comprises several layers:

- **M2M Devices:** Sensors, actuators, and smart modules embedded in machines or objects.
- **Communication Networks:** Connectivity options like cellular (2G/3G/4G), satellite, Wi-Fi, and short-range wireless technologies (e.g., Zigbee, Z-Wave).
- **M2M Middleware:** Software that enables communication and data exchange between devices and applications, often handling data routing, security, and device management.
- **M2M Applications:** Software that processes the collected data and presents it to end-users for analysis, control, or decision-making.

Limitations Leading to IoT Evolution

Despite its utility, M2M faced several limitations that spurred the development of IoT. Its proprietary nature often resulted in vendor lock-in and a lack of interoperability between different systems. M2M solutions were typically closed, isolated, and designed for specific tasks, which hindered broader integration and the creation of value across diverse datasets. Scalability was also a challenge, as expanding M2M deployments often required significant infrastructure overhauls. These limitations highlighted the need for a more open, interconnected, and scalable framework capable of leveraging internet technologies and advanced data processing, paving the way for the Internet of Things.

M2M: Direct Device Communication

- **Point-to-Point Communication:** Devices communicate directly with each other or a central M2M gateway, often using proprietary hardware and software.
- **Limited Scope:** Focused on specific, isolated tasks like remote monitoring, asset tracking, or telemetry.
- **Data Telemetry:** Primarily involves automated measurement and transmission of data from remote sources.
- **Technical Focus:** Emphasizes hardware, connectivity, and direct data transfer for a singular purpose.
- **Closed Systems:** Often implemented within a closed, dedicated network environment, limiting broader value creation.
- **Legacy Protocols:** Often utilized specialized, proprietary communication protocols and hardware, leading to integration challenges.
- **Limited Scalability:** Scalability was often restricted by the physical infrastructure and proprietary nature of the solutions, making large-scale deployments complex and costly.
- **Vendor Lock-in:** Solutions were frequently tied to specific vendors, limiting flexibility and increasing dependence on single providers for hardware, software, and services.
- **Basic Automation:** Primarily focused on automating simple, repetitive tasks such as meter reading, equipment monitoring, or remote control of machinery without complex decision-making.

IoT: Ecosystems of Connected Intelligence & Value

- **Internet Connectivity:** Devices connect to the broader internet, leveraging cloud platforms for data aggregation, analysis, and management, enabling global reach.
- **Broad Applications:** Supports a vast array of applications across diverse sectors, fostering new business models and expanded value chains (e.g., smart cities, connected health).
- **Data Analytics & AI:** Integrates advanced analytics and machine learning to derive actionable insights from massive datasets, driving predictive capabilities and automation.
- **User-Centric:** Focuses on intuitive user interfaces, applications, and seamless integration into daily life, enhancing user experience and engagement.
- **Open Standards:** Relies on open communication protocols and APIs to ensure interoperability and scalability across diverse platforms and devices.
- **Edge Computing:** Integrates computation and data processing closer to the data sources (the 'edge') to reduce latency, conserve bandwidth, and enable faster decision-making for critical applications.
- **Real-time Processing:** Capable of processing vast streams of data in real-time, enabling immediate responses to events and supporting dynamic operational adjustments.
- **Ecosystem Integration:** Fosters a broad ecosystem of devices, platforms, and services through open APIs and cloud infrastructure, facilitating seamless interoperability and collaboration.
- **Standardization Efforts:** Driven by industry-wide efforts towards common standards and protocols (e.g., MQTT, CoAP, Zigbee, Thread) to ensure device compatibility and accelerate market adoption.
- **Economic Impact:** Generates significant economic value through increased efficiency, new service offerings, predictive maintenance, and optimized resource utilization across industries.

IoT Applications Across Domains

IoT is transforming industries and daily life, creating unprecedented opportunities for efficiency, innovation, and convenience. From automating homes to revolutionizing industrial processes, the breadth of IoT applications is vast and continuously expanding across numerous sectors.

Smart Homes & Buildings

IoT in homes and buildings focuses on creating intelligent, interconnected environments that enhance comfort, safety, and energy efficiency.

- **Use Cases:** Automated lighting and climate control (smart thermostats), advanced security systems (connected cameras, smart locks), smart appliances, predictive maintenance for HVAC, air quality monitoring, intelligent access control.
- **Technologies Used:** Z-Wave, Zigbee, Wi-Fi, Bluetooth, Matter protocol, occupancy and motion sensors, smart meters, AI-powered control hubs, cloud-based IoT platforms (e.g., AWS IoT Core, Google Cloud IoT).
- **Benefits Achieved:** Energy savings (up to 30% reduction in utility bills), enhanced security with real-time alerts, increased occupant comfort and convenience, remote management capabilities, proactive maintenance reducing downtime.
- **Implementation Challenges:** Interoperability issues between diverse devices and ecosystems, data privacy and security vulnerabilities, high initial investment, complex installation and configuration, reliance on stable internet connectivity.
- **Metrics/ROI:** 15-30% energy cost reduction, improved building operational efficiency, reduced maintenance calls, higher occupant satisfaction, increased property valuation.

Smart Agriculture

Revolutionizing farming through precision agriculture, IoT optimizes resource use and maximizes yield.

- **Use Cases:** Soil moisture and nutrient monitoring, automated irrigation systems, drone-based crop health monitoring, livestock tracking and health management, weather forecasting and microclimate analysis, smart greenhouses.
- **Technologies Used:** LoRaWAN, NB-IoT, satellite connectivity, environmental sensors, GPS, autonomous farm equipment, AI for yield prediction and disease detection, data analytics platforms.
- **Benefits Achieved:** Up to 20% increase in crop yield, 30% reduction in water usage, optimized fertilizer application, early detection of plant diseases, improved animal welfare and productivity, reduced operational costs.
- **Implementation Challenges:** Limited connectivity in remote areas, high cost of sensor deployment, data interpretation complexity, ruggedization of devices for harsh environments, lack of skilled personnel.
- **Metrics/ROI:** 10-20% higher crop yields, 20-35% reduction in input costs (water, fertilizer, pesticides), improved livestock health leading to higher output.

Smart Cities

IoT transforms urban environments into more sustainable, efficient, and livable spaces for citizens.

- **Use Cases:** Intelligent traffic management (smart traffic lights, parking sensors), smart street lighting with adaptive illumination, efficient waste management (smart bins), environmental monitoring (air and water quality), public safety surveillance, smart public transport systems.
- **Technologies Used:** LPWAN (LoRa, Sigfox), 5G networks, CCTV cameras, environmental sensors, LiDAR, smart meters, GIS mapping, cloud computing, urban data platforms.
- **Benefits Achieved:** Reduced traffic congestion (up to 25% decrease in travel time), 30-50% energy savings from smart lighting, improved waste collection efficiency, real-time pollution monitoring, enhanced public safety.
- **Implementation Challenges:** High infrastructure costs, data privacy concerns with public surveillance, cybersecurity risks, integration of legacy systems, gaining public acceptance, fragmented governance.
- **Metrics/ROI:** Reduction in urban pollution (e.g., 10-15% improvement in air quality), optimized city services leading to operational savings, improved citizen quality of life.

Wearable Technology

IoT wearables offer continuous personal health monitoring and enhanced user interaction.

- **Use Cases:** Fitness trackers (activity, sleep, heart rate), smartwatches with notification and payment functions, continuous glucose monitors (CGMs), smart patches for vital sign tracking, augmented reality (AR) glasses for industrial training.
- **Technologies Used:** Bluetooth Low Energy (BLE), NFC, miniaturized sensors (accelerometers, gyroscopes, optical heart rate sensors), bio-impedance sensors, edge computing, cloud-based health platforms.
- **Benefits Achieved:** Proactive health management, personalized fitness insights, early detection of health anomalies, enhanced safety for lone workers, hands-free operation in industrial settings.
- **Implementation Challenges:** Battery life limitations, data accuracy and calibration, user adoption and comfort, data security and privacy of sensitive health information, regulatory hurdles for medical-grade devices.
- **Metrics/ROI:** Improved chronic disease management, increased user engagement in health activities, reduced insurance costs through preventative care.

Industrial IoT (IIoT)

IIoT optimizes industrial processes, boosts productivity, and ensures operational safety in manufacturing and heavy industries.

- **Use Cases:** Predictive maintenance for machinery, asset tracking and inventory management, factory automation and robotics integration, quality control and defect detection, worker safety monitoring, supply chain visibility.
- **Technologies Used:** PLC/SCADA integration, OPC UA, MQTT, Ethernet/IP, industrial sensors (vibration, temperature, pressure), AI/ML for anomaly detection, digital twin technology, edge gateways.
- **Benefits Achieved:** Up to 20% reduction in unplanned downtime, 15% increase in operational efficiency, significant cost savings on maintenance, improved product quality, enhanced worker safety.
- **Implementation Challenges:** Integration with legacy operational technology (OT) systems, robust cybersecurity requirements, harsh industrial environments, high capital expenditure, data silos across operations.
- **Metrics/ROI:** 10-40% reduction in maintenance costs, 5-15% improvement in overall equipment effectiveness (OEE), increased throughput and reduced waste.

Healthcare & RPM (Remote Patient Monitoring)

IoT in healthcare enhances patient care, streamlines clinical workflows, and enables remote medical services.

- **Use Cases:** Remote monitoring of chronic conditions (diabetes, hypertension, heart disease), smart drug dispensers for medication adherence, connected medical devices (smart inhalers, wearable ECGs), elderly care and fall detection, hospital asset tracking.
- **Technologies Used:** BLE, Wi-Fi, cellular (4G/5G), medical-grade sensors, secure cloud platforms (HIPAA compliant), AI for diagnostic support, telehealth platforms.
- **Benefits Achieved:** 15-20% improvement in patient outcomes, reduced hospital readmissions, lower healthcare costs, increased patient engagement and adherence, expanded access to care, proactive health interventions.
- **Implementation Challenges:** Data privacy (HIPAA, GDPR) and security compliance, regulatory approvals for medical devices, interoperability between systems, false alarms and data overload, user acceptance and digital literacy.
- **Metrics/ROI:** Reduced hospital stays by 20%, 30% fewer emergency room visits, improved medication adherence rates, enhanced quality of life for patients.

Transportation & Logistics

IoT optimizes fleet management, enhances safety, and ensures efficient delivery in the movement of goods and people.

- **Use Cases:** Fleet tracking and management, cold chain monitoring for perishable goods, predictive maintenance for vehicles, smart parking solutions, traffic flow optimization, autonomous vehicles (long-term).
- **Technologies Used:** GPS, cellular (LTE-M, NB-IoT), RFID, environmental sensors, telematics devices, blockchain for supply chain transparency, route optimization software.
- **Benefits Achieved:** 10-25% reduction in fuel costs, improved delivery times, reduced spoilage of goods, enhanced driver safety, better asset utilization, real-time visibility across the supply chain.
- **Implementation Challenges:** Data security and hacking risks (vehicle systems), integration with diverse legacy logistics systems, global connectivity challenges, cost of large-scale deployment, data overload for fleet managers.
- **Metrics/ROI:** 15% reduction in operational costs, 20% improvement in delivery efficiency, enhanced customer satisfaction through real-time tracking.

Energy & Utilities

IoT enables intelligent grids, optimizes energy consumption, and enhances infrastructure monitoring for utilities.

- **Use Cases:** Smart grids for demand-response management, remote monitoring of power lines and pipelines, smart metering for precise billing, renewable energy integration, predictive maintenance for infrastructure (e.g., transformers, wind turbines), water leakage detection.
- **Technologies Used:** PLC, SCADA, AMI (Advanced Metering Infrastructure), smart sensors, wide area networks (WAN), data analytics for load forecasting, distributed ledger technology for energy trading.
- **Benefits Achieved:** 10-15% reduction in energy waste, improved grid stability and reliability, faster outage detection and restoration, optimized resource allocation, reduced operational costs, enhanced environmental sustainability.
- **Implementation Challenges:** Cybersecurity threats to critical infrastructure, regulatory compliance, data privacy for consumption patterns, high cost of grid modernization, integration with existing utility systems.
- **Metrics/ROI:** 5-10% reduction in energy losses, 20% faster outage response, improved grid efficiency and reliability, reduced carbon footprint.

Retail & Supply Chain

IoT transforms retail operations, optimizes inventory, and enhances the customer shopping experience.

- **Use Cases:** Smart inventory management (RFID tags), asset tracking, customer behavior analytics (in-store heatmaps), personalized promotions (beacons), cold chain monitoring for food safety, automated checkout systems, supply chain visibility.
- **Technologies Used:** RFID, NFC, BLE beacons, computer vision, pressure sensors, robotics for warehousing, AI for demand forecasting, blockchain for traceability.
- **Benefits Achieved:** Up to 99% inventory accuracy, reduced stockouts and overstocking, enhanced customer personalization, improved operational efficiency, reduced theft, better product traceability.
- **Implementation Challenges:** Data privacy concerns regarding customer tracking, high cost of RFID/beacon deployment, integration with existing POS and ERP systems, cybersecurity for customer data, rapid technological obsolescence.
- **Metrics/ROI:** 15-30% reduction in inventory carrying costs, 5-10% increase in sales through personalization, improved customer satisfaction, reduced shrinkage.

Environmental Monitoring

IoT provides critical data for understanding, protecting, and managing our natural environment.

- **Use Cases:** Air quality monitoring (PM2.5, CO2, ozone), water quality monitoring (pH, turbidity, dissolved oxygen), wildlife tracking and conservation, forest fire detection, flood prediction, smart waste management.
- **Technologies Used:** Environmental sensors, satellite imagery, LoRaWAN, NB-IoT, acoustic sensors for wildlife, drone technology, AI for data analysis and pattern recognition, GIS.
- **Benefits Achieved:** Early warning systems for natural disasters, improved conservation efforts, data-driven environmental policy making, better resource management, enhanced public health.
- **Implementation Challenges:** Sensor maintenance in harsh environments, power supply for remote deployments, data interpretation and modeling, regulatory compliance, public engagement, funding for large-scale projects.
- **Metrics/ROI:** Reduced economic losses from natural disasters, improved ecological indicators, enhanced public health outcomes, better regulatory compliance.

Education

IoT in education creates smart learning environments and enhances administrative efficiency.

- **Use Cases:** Smart classrooms (interactive whiteboards, connected projectors), asset tracking for school equipment, environmental monitoring for optimal learning conditions, smart attendance systems, campus safety and security, personalized learning experiences.
- **Technologies Used:** Wi-Fi 6, BLE, occupancy sensors, RFID, digital displays, learning management systems (LMS) integration, AI for adaptive learning.
- **Benefits Achieved:** Enhanced student engagement, improved resource management, safer campus environments, optimized energy usage, streamlined administrative tasks, personalized educational content delivery.
- **Implementation Challenges:** Data privacy concerns for student information, cybersecurity risks, budget constraints, teacher training and adoption, ensuring equitable access, managing device proliferation.
- **Metrics/ROI:** Improved student outcomes, reduced operational costs for facilities, enhanced campus safety, better asset utilization, increased administrative efficiency.

These diverse applications highlight IoT's profound potential to drive significant advancements across virtually every sector, improving quality of life and business operations alike through connected intelligence.